



Data Warehouses

Lecture 3

Lecture Goals

Agenda:

OLTP/OLAP

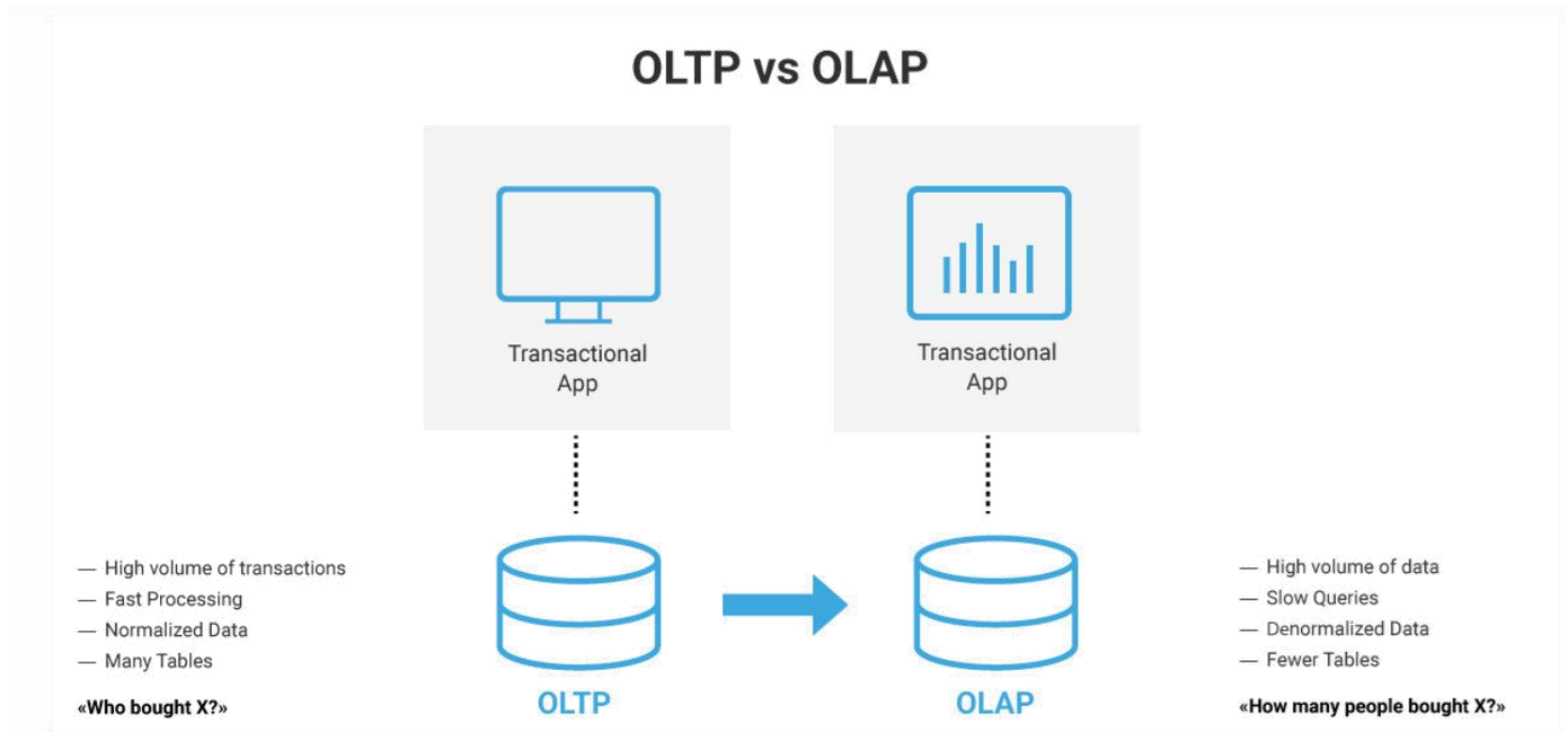
Querying data using SQL – data
analysis perspective

OVER

Execution plans

*a very brief introduction to execution
plans in SSMS*

OLTP/OLAP



OLAP: Understanding the Past and Present

Purpose:

- OLAP is primarily focused on analyzing historical and current data to understand trends, patterns, and relationships. It's about gaining insights from existing data to inform business decisions.

Methodology:

- OLAP uses multidimensional data models (like data cubes) to organize and summarize data, allowing users to quickly query and analyze data from different perspectives (dimensions).
- Common OLAP operations include: Slicing and Dicing, Drill-down and Roll-up, Aggregation

Output:

- OLAP typically produces reports, dashboards, and visualizations that help business users understand past performance, current status, and identify areas for improvement.

Focus:

- Descriptive analytics - "What happened?" and "Why did it happen?"

BI and BA



The Evolution of Data Usage:

Organizations progress through different stages of data maturity, from understanding the past to predicting and automating the future.



Business Intelligence (BI):

Focuses on Descriptive and Diagnostic analytics.

Questions answered: "What happened?" and "Why did it happen?,"

Outputs: Dashboards, KPIs, standard reports.



Business Analytics (BA):

Bridges the gap between BI and Data Science. Focuses on Predictive analytics using statistical methods applied to business problems.

Questions answered: "What is likely to happen next quarter?,"

Outputs: Trend analysis, forecasting models.

OLAP in BI and BA

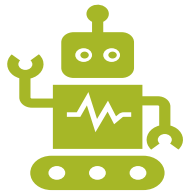
OLAP and Business Intelligence (The Perfect Match):

- OLAP cubes and columnar databases are the foundational backbone of BI.
- BI tools (Power BI, Tableau) generate the complex GROUP BY, ROLLUP, and CUBE SQL queries we will discuss and send them to the OLAP engine.
- OLAP provides the sub-second response times required for interactive BI dashboards.

OLAP and Business Analytics:

- Business Analysts rely on OLAP systems to aggregate massive datasets before exporting them into statistical tools (like Excel, SPSS, or R) for trend forecasting.

DS and ML



Data Science (DS):

A multidisciplinary field (computer science, statistics, domain knowledge) used to extract complex insights from both structured and unstructured data (text, images, logs).

Focus:

- *Experimentation, discovering hidden patterns, and building complex data pipelines.*



Machine Learning (ML):

A subset of Artificial Intelligence (and a core tool of Data Science).

Algorithms that learn from data to make predictions or decisions without being explicitly programmed.

Focus:

- *Prescriptive analytics and automation.*

Questions answered:

- *"How can we automate the detection of fraudulent transactions in real-time?"*

ML: Predicting the Future and Automating Decisions



Purpose:

Machine Learning is focused on building models that can learn from data to make predictions, classify data, or automate tasks. It's about using data to anticipate future outcomes and make intelligent decisions.



Methodology:

ML algorithms learn patterns from training data and then use these patterns to make predictions or classifications on new, unseen data.

Common ML tasks include: Classification, Regression, Clustering, Anomaly Detection



Output:

ML models produce predictions, classifications, recommendations, and automated actions



Focus:

Predictive and Prescriptive analytics - "What will happen?" and "What should we do?"

OLAP in DS and ML

Is OLAP used for Machine Learning?

- Not directly for model training, but it plays a crucial supporting role.

OLAP as a "Feature Store":

- Data Scientists need high-quality, cleansed, and historically accurate data to train Machine Learning models.
- The Data Warehouse (via OLAP) provides "Golden Records" (single source of truth) for customer histories, sales aggregates, and financial metrics.

The Workflow:

- DS/ML algorithms (e.g., neural networks) are poor at doing heavy relational joins.
- Instead, data scientists use OLAP queries to extract wide, flattened, and aggregated datasets to feed into their ML models.



Operational database is CRUCIAL

Operational database
is NOT ENOUGH



Necessity Is the Mother of Invention

At first, the same databases were used for both transactional processing and analytic queries

SQL is quite flexible in this regard

Late 80's/90's move to dedicated solutions focused on data analysis

Information wise - **Data warehousing**

Data warehousing - DW

Nowadays

DW now exists in almost all larger enterprises

- multiple OLTP systems – often geographically distributed
- large amounts of data

In smaller companies DW are less popular

- do not have many OLTP systems
- smaller amounts of data
- some tools are really expensive

Recent trends

SQL on Hadoop

Apache Hive, Apache Spark, Apache Kudu, Cloudera Impala, Facebook Presto, Apache Drill, Apache Kylin, etc.

Cloud solutions

AWS Redshift, Google BigQuery, Azure Synapse Analytics, Snowflake, etc.

Data Lake and Data Lakehouses

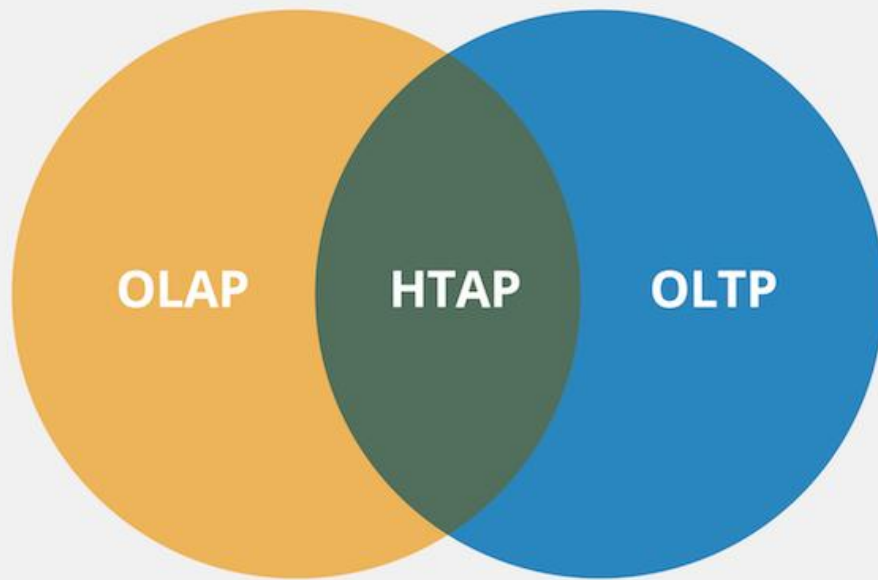
Dremio, Kylo, CDP, etc.

Virtualization

Denodo, Querona, etc.

Real-time and streaming data:

ClickHouse, Apache Druid, Materialise



HTAP

Handling so-called operational analytical processing workloads

In a way real-time analytics

Online analytical processing

A profile for software applications that enables users to analyze data multidimensionally, via rolling up, slicing, dicing and drilling down. OLAP databases are for historical analytics, frequently on transactional data.

Hybrid transaction/analytical processing

A technology for managing live operational data and historical data within the same database management system, instead of using separate OLTP databases that feed into an OLAP data warehouse. HTAP enables both real-time analytics for processing transactions and historical analytics.

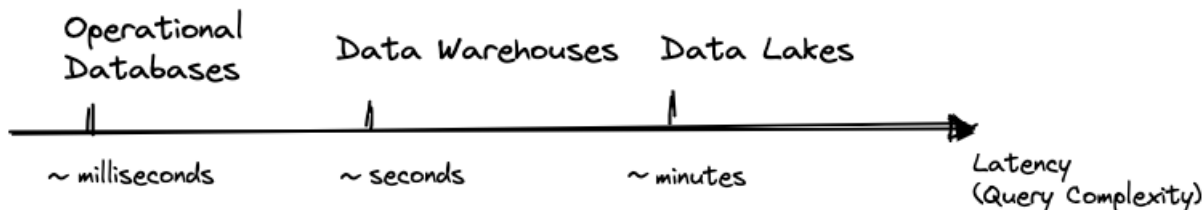
Online transaction processing

OLTP databases are built to process mission-critical transactions for the business's operations (e.g., logging orders, pulling customer account information etc.). Generally, OLTP databases feed information to and receive data from mobile applications, point of sale systems, web applications, websites, ATM machines etc.

Do not confuse with operational reporting

act of detailing a company's day-to-day operations using visuals that show operations workflow to make fast, informed decisions about daily operations.

Example – TiDB, AlloyDB



source:
www.softwareadvice.com/resources/olap-data-warehouse-alternatives/

ISO/ANSI SQL Standard



SQL:2003 – Windowing functions officially introduced



SQL:2006 – No news on windowing functions, XML related features



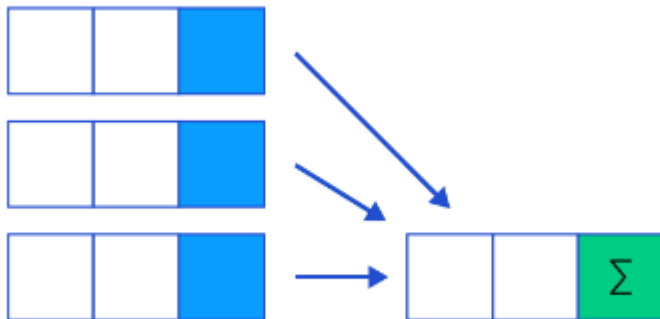
SQL:2008 – Some improvements on windows functions

Window functions

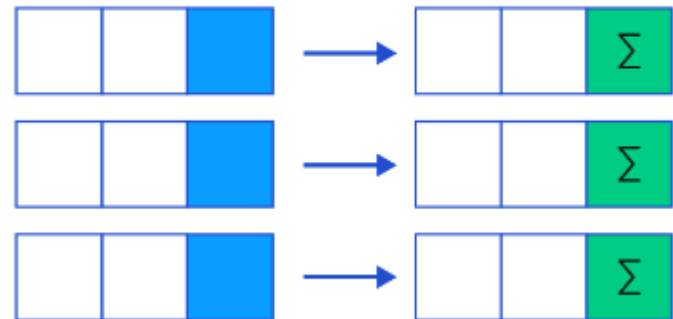
Window functions are often used in data warehouse and other OLAP applications

Window

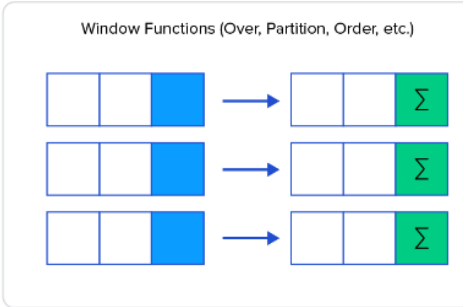
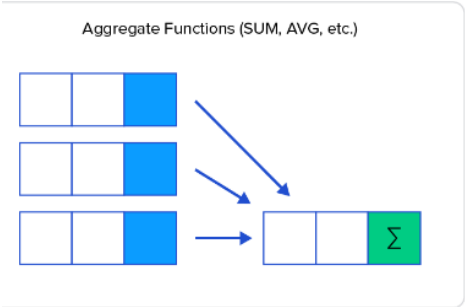
Aggregate Functions (SUM, AVG, etc.)



Window Functions (Over, Partition, Order, etc.)



Window



	count
1	504

	name	count
1...	HL Road Seat Assembly	504
1...	HL Road Seat/Saddle	504
1...	HL Road Tire	504
1...	HL Shell	504
1...	HL Spindle/Axle	504
1...	HL Touring Frame - Bl...	504
1...	HL Touring Frame - Bl...	504
1...	HL Touring Frame - Bl...	504
1...	HL Touring Frame - Bl...	504
1...	HL Touring Frame - Ye...	504
1	HL Touring Frame - Ye	504

Window

How to define a window
with the OVER Clause

*you define the window over which the function will be applied
by default the window is defined for the entire table*

the result is the same as if you had used an aggregation function without a GROUP BY clause

Syntax

<window_function_type> OVER <window_specification>

Details

window_function (expression)

OVER (

[<PARTITION BY partition_list >]

[<ORDER BY order_list >]

[<ROW or RANGE clause>]

)

Notions

The WHERE clause

returns a certain number of rows.

They constitute the **result set**.

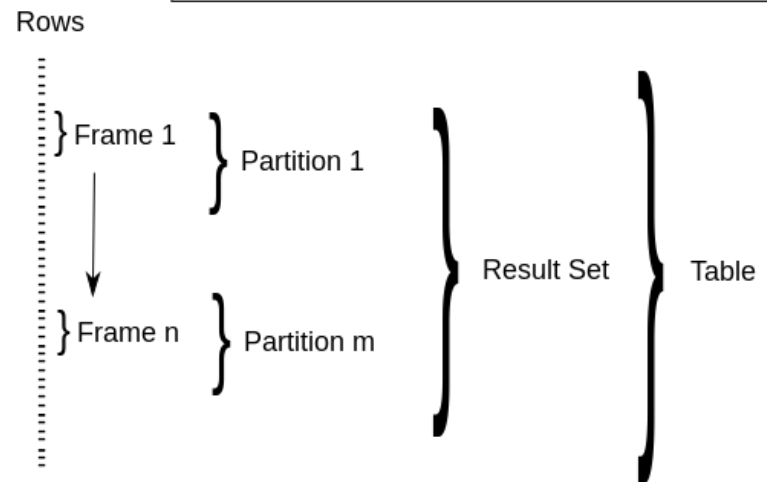
The ORDER BY clause

re-orders the result set into a certain sequence

this sequence determines the order in which the rows are passed to the SELECT clause.

*The row, which is actually given to the SELECT clause, is called the **current row**.*

SQL Window Functions act on (sliding) Window Frames, which are part of Window Partitions, Result Sets, Tables.



Notions

The WINDOW PARTITION

clause divides the result set into
window partitions

*If there is no WINDOW PARTITION
clause*

all rows of the **result set** constitutes
one **partition**

equivalent to groups created by the
GROUP BY clause.

Partitions are distinct from each
other:

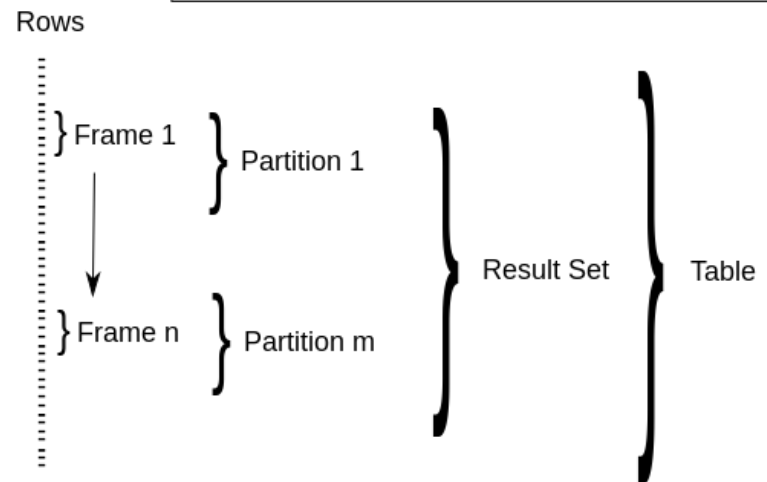
there is no overlapping as every row
of the result set belongs to one and
only one partition.

The WINDOW ORDER clause

orders the **rows** of each **partition**

*which may differ from the ORDER
BY clause*

SQL Window Functions act on (sliding)
Window Frames, which are part of
Window Partitions, Result Sets, Tables.



Notions

The WINDOW FRAME clause

defines which rows of the actual partition belong to the actual **window frame**

defines one frame for every row of the result set.

This is done by determining the **lower** and **upper boundary** of affected rows.

In consequence, there are as many (mostly different) frames as number of rows in the result set.

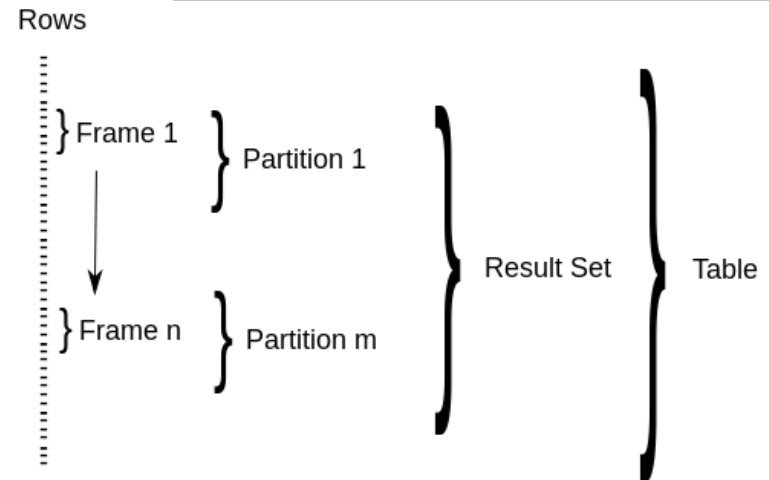
The upper and lower boundaries are newly determined with every row of the result set!

Single rows may be part of more than one frame.

The actual frame is the instantiation of the 'sliding window'.

Its rows are ordered according to the WINDOW ORDER clause.

SQL Window Functions act on (sliding) Window Frames, which are part of Window Partitions, Result Sets, Tables.



Notions

If there is no WINDOW FRAME clause

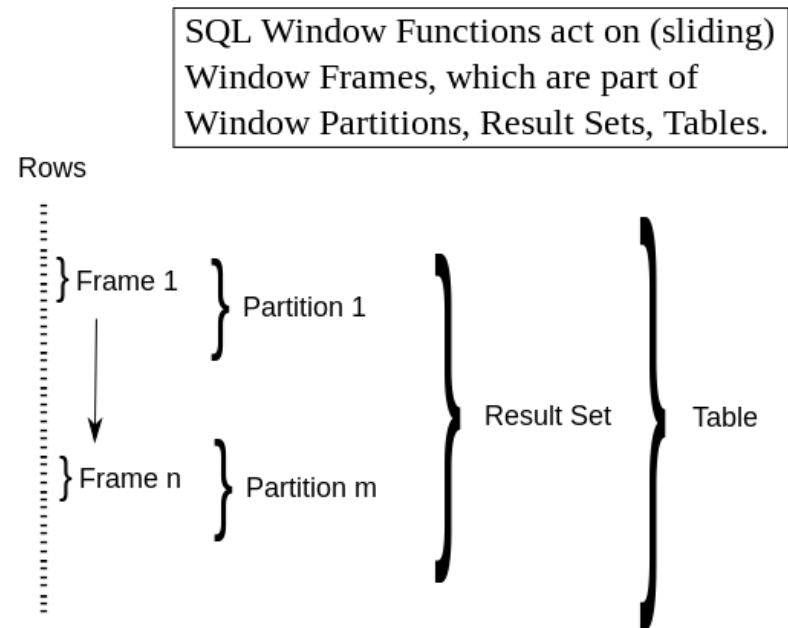
the rows of the actual partition constitute frames with the following default boundaries:

The first row of the actual partition is their lower boundary and the current row is their upper boundary.

If there is no WINDOW FRAME clause and no WINDOW ORDER clause

the upper boundary switches to the last row of the actual partition.

The <window_function> acts on the rows of the actual **frame**.



Window

Window functions

allows you to segment a set of rows and then apply a function to that set of rows

You get,

ability to choose an aggregation function

use other functions

rankings, analytics

Types of Window functions

Aggregate Window Functions

*SUM(), MAX(), MIN(), AVG().
COUNT()*

Ranking Window Functions

*RANK(), DENSE_RANK(),
ROW_NUMBER(), NTILE()*

Value Window Functions

*LAG(), LEAD(), FIRST_VALUE(),
LAST_VALUE()*

Window

```
select CustomerName  
      SUM(OrderAmt) OVER () as NoParm  
from XXX;
```

	Order...	OrderDate	OrderA...	CustomerNa...
1	1	2012-03-01	10.0000	Joe
2	2	2012-03-01	11.0000	Sam
3	3	2012-03-02	10.0000	Beth
4	4	2012-03-02	15.0000	Joe
5	5	2012-03-02	17.0000	Sam
6	6	2012-03-03	12.0000	Joe
7	7	2012-03-04	10.0000	Beth
8	8	2012-03-04	18.0000	Sam
9	9	2012-03-04	12.0000	Joe
10	10	2012-03-04	11.0000	Beth
11	11	2012-03-05	14.0000	Sam
12	12	2012-03-06	17.0000	Beth
13	13	2012-03-06	19.0000	Joe
14	14	2012-03-07	13.0000	Beth
15	15	2012-03-07	16.0000	Sam



	CustomerNa...	NoParm
1	Joe	205.0000
2	Sam	205.0000
3	Beth	205.0000
4	Joe	205.0000
5	Sam	205.0000
6	Joe	205.0000
7	Beth	205.0000
8	Sam	205.0000
9	Joe	205.0000
10	Beth	205.0000
11	Sam	205.0000
12	Beth	205.0000
13	Joe	205.0000
14	Beth	205.0000
15	Sam	205.0000

Sum = 205

Window

Query

```
select CustomerName  
      SUM(OrderAmt) OVER () as NoParm  
from XXX
```

Can be rewritten:

```
select CustomerName, (select  
  sum(OrderAmt) from XXX) as  
  NoParm  
from XXX
```

	Order...	OrderDate	OrderA...	CustomerNa...
1	1	2012-03-01	10.0000	Joe
2	2	2012-03-01	11.0000	Sam
3	3	2012-03-02	10.0000	Beth
4	4	2012-03-02	15.0000	Joe
5	5	2012-03-02	17.0000	Sam
6	6	2012-03-03	12.0000	Joe
7	7	2012-03-04	10.0000	Beth
8	8	2012-03-04	18.0000	Sam
9	9	2012-03-04	12.0000	Joe
10	10	2012-03-04	11.0000	Beth
11	11	2012-03-05	14.0000	Sam
12	12	2012-03-06	17.0000	Beth
13	13	2012-03-06	19.0000	Joe
14	14	2012-03-07	13.0000	Beth
15	15	2012-03-07	16.0000	Sam



	CustomerNa...	NoParm
1	Joe	205.0000
2	Sam	205.0000
3	Beth	205.0000
4	Joe	205.0000
5	Sam	205.0000
6	Joe	205.0000
7	Beth	205.0000
8	Sam	205.0000
9	Joe	205.0000
10	Beth	205.0000
11	Sam	205.0000
12	Beth	205.0000
13	Joe	205.0000
14	Beth	205.0000
15	Sam	205.0000



Window

The OVER clause is what specifies a window function and must always be included in the statement.

The default in an OVER clause is the entire rowset.

Window

Example

select

P.name, PS.Name, PC.Name, AVG(P.ListPrice) OVER () as AVG2

from

[Production].[Product] AS P

inner join [Production].[ProductSubcategory] AS PS

on P.ProductSubcategoryID = PS.ProductSubcategoryID

inner join [Production].[ProductCategory] AS PC

on PS.ProductCategoryID = PC.ProductCategoryID

Window

	name	Name	Name	AVG2
1	HL Road Frame - Black, 58	Road Frames	Components	744,5952
2	HL Road Frame - Red, 58	Road Frames	Components	744,5952
3	Sport-100 Helmet, Red	Helmets	Accessories	744,5952
4	Sport-100 Helmet, Black	Helmets	Accessories	744,5952
5	Mountain Bike Socks, M	Socks	Clothing	744,5952
6	Mountain Bike Socks, L	Socks	Clothing	744,5952
7	Sport-100 Helmet, Blue	Helmets	Accessories	744,5952
8	AWC Logo Cap	Caps	Clothing	744,5952
9	Long-Sleeve Logo Jersey, S	Jerseys	Clothing	744,5952
10	Long-Sleeve Logo Jersey, M	Jerseys	Clothing	744,5952
11	Long-Sleeve Logo Jersey, L	Jerseys	Clothing	744,5952
12	Long-Sleeve Logo Jersey, XL	Jerseys	Clothing	744,5952
13	HL Road Frame - Red, 62	Road Frames	Components	744,5952
14	HL Road Frame - Red, 44	Road Frames	Components	744,5952
15	HL Road Frame - Red, 48	Road Frames	Components	744,5952
16	HL Road Frame - Red, 52	Road Frames	Components	744,5952
17	HL Road Frame - Red, 56	Road Frames	Components	744,5952

Example

select

*P.name, PS.Name, PC.Name,
AVG(P.ListPrice) OVER () as AVG2*

from

[Production].[Product] AS P

inner join

*[Production].[ProductSubcategory] AS
PS*

*on P.ProductSubcategoryID =
PS.ProductSubcategoryID*

inner join

[Production].[ProductCategory] AS PC

*on PS.ProductCategoryID =
PC.ProductCategoryID*

Window

The OVER clause

takes three additional arguments to change the scope of the clause

PARTITION BY,

ORDER BY,

and ROWS or RANGE.

SQL

OVER (

[PARTITION BY value_expression , ... [n]]

[ORDER BY order_by_expression]

[<ROWS or RANGE clause>]

)

Window

OVER clause

defines the partitioning and ordering of rows (i.e. a window) for the above functions to operate on – hence, called window functions.

The OVER clause accepts the following arguments to define a window:

PARTITION BY:

Divides the query result set into partitions.

The window function is applied to each partition separately.

Multiple fields need to be separated by a comma.

If the PARTITION BY is not specified

then the grouping will be done on the entire table and values will be aggregated accordingly.

Window

OVER clause

defines the partitioning and ordering of rows (i.e. a window) for the above functions to operate on – hence, called window functions.

The OVER clause accepts the following arguments to define a window:

ORDER BY:

Sorts the rows within each partition.

If ORDER BY is not specified

ORDER BY uses the entire table.

Window

OVER clause

defines the partitioning and ordering of rows (i.e. a window) for the above functions to operate on – hence, called window functions.

The OVER clause accepts the following arguments to define a window:

ROWS or RANGE clause:

Further subsets the rows within the partition by specifying start and endpoints within the partition.

The default for ROWS or RANGE clause is

RANGE BETWEEN UNBOUND PRECEDING AND CURRENT ROW

Window

OVER with PARTITION BY

is used to reduce the scope of the window to which the aggregation is applied

divides the result set into partitions based on the criteria specified

the aggregation function is then applied to each partition independently

similar to the GROUP BY clause

the PARTITION BY clause allows you to specify different partitions within the same select statement – multiple groupings

you can use different partitions for different result set attributes



OVER
WITH PARTITION BY

Window

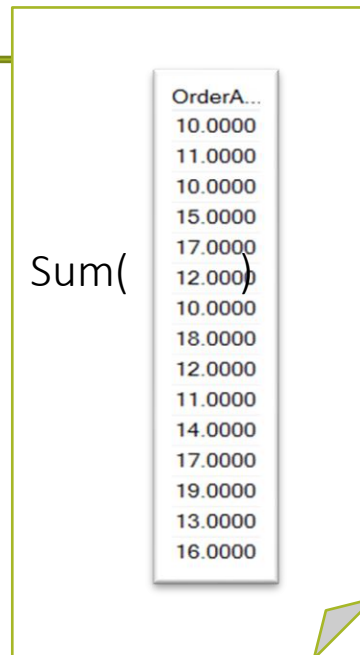
select CustomerName

1. *,SUM(OrderAmt) OVER () as NoParm*
2. *,SUM(OrderAmt) OVER (PARTITION BY CustomerName) as PartByName*

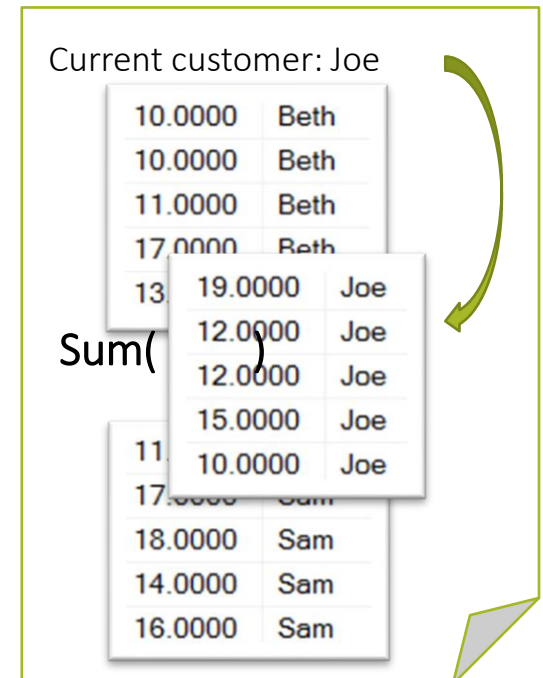
From XXX order by OrderID;

	Order...	OrderDate	OrderA...	CustomerNa...
1	1	2012-03-01	10.0000	Joe
2	2	2012-03-01	11.0000	Sam
3	3	2012-03-02	10.0000	Beth
4	4	2012-03-02	15.0000	Joe
5	5	2012-03-02	17.0000	Sam
6	6	2012-03-03	12.0000	Joe
7	7	2012-03-04	10.0000	Beth
8	8	2012-03-04	18.0000	Sam
9	9	2012-03-04	12.0000	Joe
10	10	2012-03-04	11.0000	Beth
11	11	2012-03-05	14.0000	Sam
12	12	2012-03-06	17.0000	Beth
13	13	2012-03-06	19.0000	Joe
14	14	2012-03-07	13.0000	Beth
15	15	2012-03-07	16.0000	Sam

1 - window



2 - window



Window

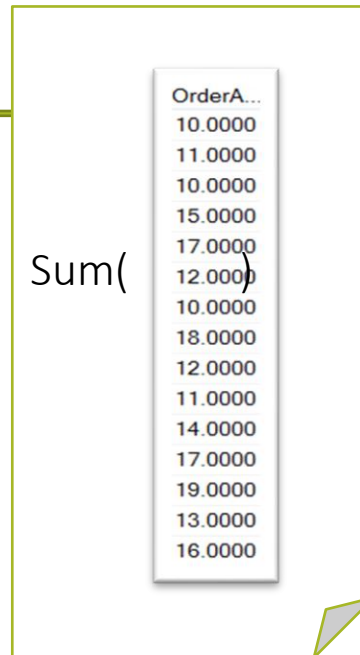
select CustomerName

1. *,SUM(OrderAmt) OVER () as NoParm*
2. *,SUM(OrderAmt) OVER (PARTITION BY CustomerName) as PartByName*

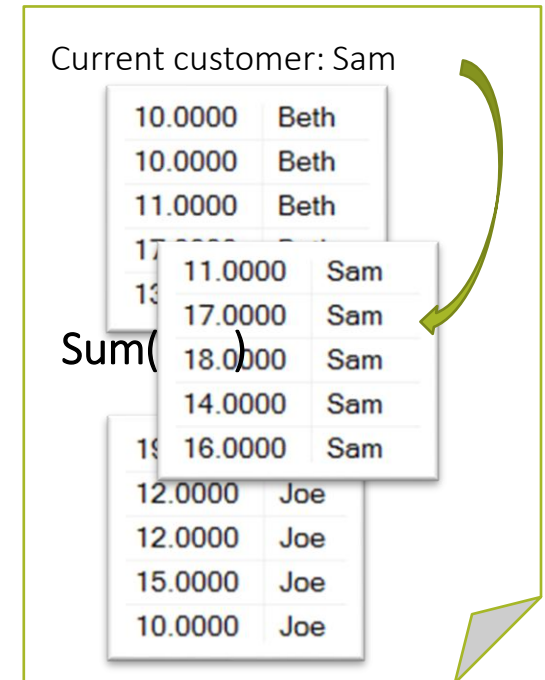
From XXX order by OrderID;

	Order...	OrderDate	OrderA...	CustomerNa...
1	1	2012-03-01	10.0000	Joe
2	2	2012-03-01	11.0000	Sam
3	3	2012-03-02	10.0000	Beth
4	4	2012-03-02	15.0000	Joe
5	5	2012-03-02	17.0000	Sam
6	6	2012-03-03	12.0000	Joe
7	7	2012-03-04	10.0000	Beth
8	8	2012-03-04	18.0000	Sam
9	9	2012-03-04	12.0000	Joe
10	10	2012-03-04	11.0000	Beth
11	11	2012-03-05	14.0000	Sam
12	12	2012-03-06	17.0000	Beth
13	13	2012-03-06	19.0000	Joe
14	14	2012-03-07	13.0000	Beth
15	15	2012-03-07	16.0000	Sam

1 - window



2 - window



Window

```
select CustomerName  
      ,SUM(OrderAmt) OVER () as NoParm  
      ,SUM(OrderAmt) OVER (PARTITION BY CustomerName) as PartByName  
From XXX order by OrderID;
```

	Order...	OrderDate	OrderA...	CustomerNa...
1	1	2012-03-01	10.0000	Joe
2	2	2012-03-01	11.0000	Sam
3	3	2012-03-02	10.0000	Beth
4	4	2012-03-02	15.0000	Joe
5	5	2012-03-02	17.0000	Sam
6	6	2012-03-03	12.0000	Joe
7	7	2012-03-04	10.0000	Beth
8	8	2012-03-04	18.0000	Sam
9	9	2012-03-04	12.0000	Joe
10	10	2012-03-04	11.0000	Beth
11	11	2012-03-05	14.0000	Sam
12	12	2012-03-06	17.0000	Beth
13	13	2012-03-06	19.0000	Joe
14	14	2012-03-07	13.0000	Beth
15	15	2012-03-07	16.0000	Sam



	CustomerNa...	NoParm	PartByNa...
1	Joe	205.0000	68.0000
2	Sam	205.0000	76.0000
3	Beth	205.0000	61.0000
4	Joe	205.0000	68.0000
5	Sam	205.0000	76.0000
6	Joe	205.0000	68.0000
7	Beth	205.0000	61.0000
8	Sam	205.0000	76.0000
9	Joe	205.0000	68.0000
10	Beth	205.0000	61.0000
11	Sam	205.0000	76.0000
12	Beth	205.0000	61.0000
13	Joe	205.0000	68.0000
14	Beth	205.0000	61.0000
15	Sam	205.0000	76.0000

Window

```
select CustomerName, OrderDate  
      ,SUM(OrderAmt) OVER (PARTITION BY CustomerName, OrderDate) as  
      PartByName  
from XXX
```

	Order...	OrderDate	OrderA...	CustomerNa...
1	1	2012-03-01	10.0000	Joe
2	2	2012-03-01	11.0000	Sam
3	3	2012-03-02	10.0000	Beth
4	4	2012-03-02	15.0000	Joe
5	5	2012-03-02	17.0000	Sam
6	6	2012-03-03	12.0000	Joe
7	7	2012-03-04	10.0000	Beth
8	8	2012-03-04	18.0000	Sam
9	9	2012-03-04	12.0000	Joe
10	10	2012-03-04	11.0000	Beth
11	11	2012-03-05	14.0000	Sam
12	12	2012-03-06	17.0000	Beth
13	13	2012-03-06	19.0000	Joe
14	14	2012-03-07	13.0000	Beth
15	15	2012-03-07	16.0000	Sam



	CustomerNa...	OrderDate	PartByNa...
1	Beth	2012-03-02	10.0000
2	Beth	2012-03-04	21.0000
3	Beth	2012-03-04	21.0000
4	Beth	2012-03-06	17.0000
5	Beth	2012-03-07	13.0000
6	Joe	2012-03-01	10.0000
7	Joe	2012-03-02	15.0000
8	Joe	2012-03-03	12.0000
9	Joe	2012-03-04	12.0000
10	Joe	2012-03-06	19.0000
11	Sam	2012-03-01	11.0000
12	Sam	2012-03-02	17.0000
13	Sam	2012-03-04	18.0000
14	Sam	2012-03-05	14.0000
15	Sam	2012-03-07	16.0000

Window

Example

select

*P.name, PS.Name, PC.Name, AVG(P.ListPrice) OVER
() as AVG2,
AVG(P.ListPrice) OVER (Partition by P.Name) as
AVG3*

from

*[Production].[Product] AS P
inner join [Production].[ProductSubcategory] AS PS
on P.ProductSubcategoryID =
PS.ProductSubcategoryID
inner join [Production].[ProductCategory] AS PC
on PS.ProductCategoryID = PC.ProductCategoryID*

Window

Results Messages					
	name	Name	Name	AVG2	AVG3
1	All-Purpose Bike Stand	Bike Stands	Accessories	744,5952	159,00
2	AWC Logo Cap	Caps	Clothing	744,5952	8,99
3	Bike Wash - Dissolver	Cleaners	Accessories	744,5952	7,95
4	Cable Lock	Locks	Accessories	744,5952	25,00
5	Chain	Chains	Components	744,5952	20,24
6	Classic Vest, L	Vests	Clothing	744,5952	63,50
7	Classic Vest, M	Vests	Clothing	744,5952	63,50
8	Classic Vest, S	Vests	Clothing	744,5952	63,50
9	Fender Set - Mountain	Fenders	Accessories	744,5952	21,98
10	Front Brakes	Brakes	Components	744,5952	106,50
11	Front Derailleur	Derailleurs	Components	744,5952	91,49
12	Full-Finger Gloves, L	Gloves	Clothing	744,5952	37,99
13	Full-Finger Gloves, M	Gloves	Clothing	744,5952	37,99
14	Full-Finger Gloves, S	Gloves	Clothing	744,5952	37,99
15	Half-Finger Gloves, L	Gloves	Clothing	744,5952	24,49
16	Half-Finger Gloves, M	Gloves	Clothing	744,5952	24,49
17	Half-Finger Gloves, S	Gloves	Clothing	744,5952	24,49
18	Headlights - Dual-Beam	Lights	Accessories	744,5952	34,99

Example

select

```
P.name, PS.Name, PC.Name,  
AVG(P.ListPrice) OVER () as AVG2,  
AVG(P.ListPrice) OVER (Partition by P.Name)  
as AVG3
```

from

```
[Production].[Product] AS P  
inner join [Production].[ProductSubcategory]  
AS PS  
on P.ProductSubcategoryID =  
PS.ProductSubcategoryID  
inner join [Production].[ProductCategory] AS  
PC  
on PS.ProductCategoryID =  
PC.ProductCategoryID
```

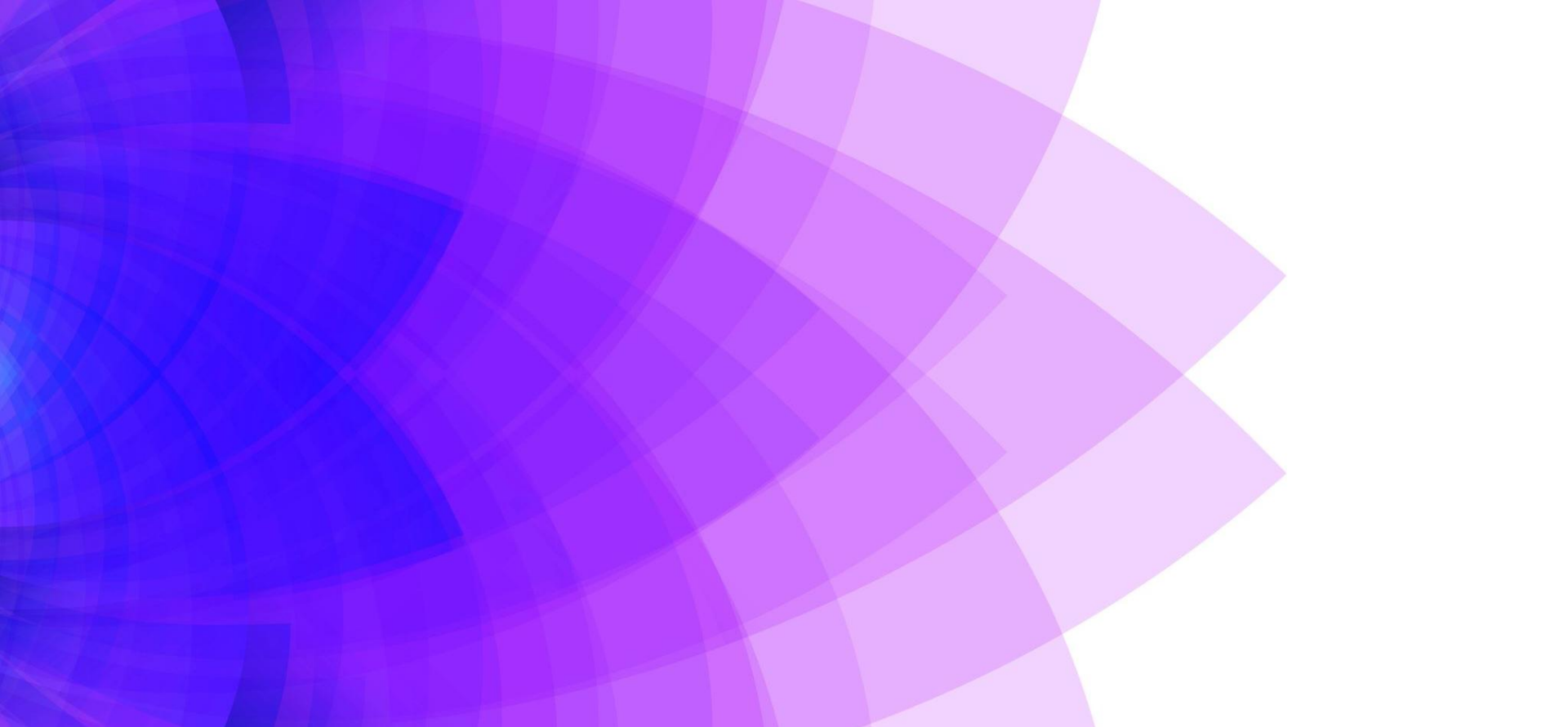
Window

```
select CustomerName, OrderDate
      ,100*OrderAmt/SUM(OrderAmt) OVER (PARTITION BY CustomerName) as PrcCustomer
      ,100*OrderAmt/SUM(OrderAmt) OVER (PARTITION BY OrderDate) as PrcDay
from XXX
```

	Order...	OrderDate	OrderA...	CustomerNa...
1	1	2012-03-01	10.0000	Joe
2	2	2012-03-01	11.0000	Sam
3	3	2012-03-02	10.0000	Beth
4	4	2012-03-02	15.0000	Joe
5	5	2012-03-02	17.0000	Sam
6	6	2012-03-03	12.0000	Joe
7	7	2012-03-04	10.0000	Beth
8	8	2012-03-04	18.0000	Sam
9	9	2012-03-04	12.0000	Joe
10	10	2012-03-04	11.0000	Beth
11	11	2012-03-05	14.0000	Sam
12	12	2012-03-06	17.0000	Beth
13	13	2012-03-06	19.0000	Joe
14	14	2012-03-07	13.0000	Beth
15	15	2012-03-07	16.0000	Sam



	CustomerNa...	OrderDate	PrcCustomer	PrcDay
1	Joe	2012-03-01	14.705882352941176	47.619047619047619
2	Sam	2012-03-01	14.473684210526315	52.380952380952380
3	Sam	2012-03-02	22.368421052631578	40.476190476190476
4	Beth	2012-03-02	16.393442622950819	23.809523809523809
5	Joe	2012-03-02	22.058823529411764	35.714285714285714
6	Joe	2012-03-03	17.647058823529411	100.000000000000000
7	Joe	2012-03-04	17.647058823529411	23.529411764705882
8	Beth	2012-03-04	16.393442622950819	19.607843137254901
9	Beth	2012-03-04	18.032786885245901	21.568627450980392
10	Sam	2012-03-04	23.684210526315789	35.294117647058823
11	Sam	2012-03-05	18.421052631578947	100.000000000000000
12	Beth	2012-03-06	27.868852459016393	47.222222222222222
13	Joe	2012-03-06	27.941176470588235	52.777777777777777
14	Beth	2012-03-07	21.311475409836065	44.827586206896551
15	Sam	2012-03-07	21.052631578947368	55.172413793103448



OVER
+ ORDER BY

Window

OVER with ORDER BY

is used to order how the partitions apply the window function.

If no partition is specified then the partition is still the whole table

If no additional constraints are defined the window is just

Beginning – the first element

End – the current element (last instance)

equal elements are indistinguishable

Window

```
select CustomerName, OrderDate  
      , SUM(OrderAmt) OVER (PARTITION BY CustomerName ORDER BY OrderDate) as  
      VAL
```

(A) from XXX order by OrderID;

(B) from XXX order by CustomerName, OrderDate;

	Order...	OrderDate	OrderA...	CustomerNa...
1	1	2012-03-01	10.0000	Joe
2	2	2012-03-01	11.0000	Sam
3	3	2012-03-02	10.0000	Beth
4	4	2012-03-02	15.0000	Joe
5	5	2012-03-02	17.0000	Sam
6	6	2012-03-03	12.0000	Joe
7	7	2012-03-04	10.0000	Beth
8	8	2012-03-04	18.0000	Sam
9	9	2012-03-04	12.0000	Joe
10	10	2012-03-04	11.0000	Beth
11	11	2012-03-05	14.0000	Sam
12	12	2012-03-06	17.0000	Beth
13	13	2012-03-06	19.0000	Joe
14	14	2012-03-07	13.0000	Beth
15	15	2012-03-07	16.0000	Sam



	CustomerName	OrderDate	VAL
1	Joe	2012-03-01	10.0000
2	Sam	2012-03-01	11.0000
3	Beth	2012-03-02	10.0000
4	Joe	2012-03-02	25.0000
5	Sam	2012-03-02	28.0000
6	Joe	2012-03-03	37.0000
7	Beth	2012-03-04	31.0000
8	Sam	2012-03-04	46.0000
9	Joe	2012-03-04	49.0000
10	Beth	2012-03-04	31.0000
11	Sam	2012-03-05	60.0000
12	Beth	2012-03-06	48.0000
13	Joe	2012-03-06	68.0000
14	Beth	2012-03-07	61.0000
15	Sam	2012-03-07	76.0000

A

	CustomerName	OrderDate	VAL
1	Beth	2012-03-02	10.0000
2	Beth	2012-03-04	31.0000
3	Beth	2012-03-04	31.0000
4	Beth	2012-03-06	48.0000
5	Beth	2012-03-07	61.0000
6	Joe	2012-03-01	10.0000
7	Joe	2012-03-02	25.0000
8	Joe	2012-03-03	37.0000
9	Joe	2012-03-04	49.0000
10	Joe	2012-03-06	68.0000
11	Sam	2012-03-01	11.0000
12	Sam	2012-03-02	28.0000
13	Sam	2012-03-04	46.0000
14	Sam	2012-03-05	60.0000
15	Sam	2012-03-07	76.0000

B

Window

select CustomerName

SUM(OrderAmt) OVER (ORDER BY CustomerName) as OrderByName

(A) from XXX order by OrderID;

(B) from XXX order by OrderByName

	Order...	OrderDate	OrderA...	CustomerNa...
1	1	2012-03-01	10.0000	Joe
2	2	2012-03-01	11.0000	Sam
3	3	2012-03-02	10.0000	Beth
4	4	2012-03-02	15.0000	Joe
5	5	2012-03-02	17.0000	Sam
6	6	2012-03-03	12.0000	Joe
7	7	2012-03-04	10.0000	Beth
8	8	2012-03-04	18.0000	Sam
9	9	2012-03-04	12.0000	Joe
10	10	2012-03-04	11.0000	Beth
11	11	2012-03-05	14.0000	Sam
12	12	2012-03-06	17.0000	Beth
13	13	2012-03-06	19.0000	Joe
14	14	2012-03-07	13.0000	Beth
15	15	2012-03-07	16.0000	Sam



	CustomerName	OrderByName
1	Joe	129.0000
2	Sam	205.0000
3	Beth	61.0000
4	Joe	129.0000
5	Sam	205.0000
6	Joe	129.0000
7	Beth	61.0000
8	Sam	205.0000
9	Joe	129.0000
10	Beth	61.0000
11	Sam	205.0000
12	Beth	61.0000
13	Joe	129.0000
14	Beth	61.0000
15	Sam	205.0000

A

	CustomerName	OrderByName
1	Beth	61.0000
2	Beth	61.0000
3	Beth	61.0000
4	Beth	61.0000
5	Beth	61.0000
6	Joe	129.0000
7	Joe	129.0000
8	Joe	129.0000
9	Joe	129.0000
10	Joe	129.0000
11	Sam	205.0000
12	Sam	205.0000
13	Sam	205.0000
14	Sam	205.0000
15	Sam	205.0000

B

Window

select CustomerName

1. ,SUM(OrderAmt) OVER (PARTITION BY CustomerName) as PartByName

2. ,SUM(OrderAmt) OVER (ORDER BY CustomerName) as OrderByName

from XXX order by OrderID;

	Order...	OrderDate	OrderA...	CustomerNa...
1	1	2012-03-01	10.0000	Joe
2	2	2012-03-01	11.0000	Sam
3	3	2012-03-02	10.0000	Beth
4	4	2012-03-02	15.0000	Joe
5	5	2012-03-02	17.0000	Sam
6	6	2012-03-03	12.0000	Joe
7	7	2012-03-04	10.0000	Beth
8	8	2012-03-04	18.0000	Sam
9	9	2012-03-04	12.0000	Joe
10	10	2012-03-04	11.0000	Beth
11	11	2012-03-05	14.0000	Sam
12	12	2012-03-06	17.0000	Beth
13	13	2012-03-06	19.0000	Joe
14	14	2012-03-07	13.0000	Beth
15	15	2012-03-07	16.0000	Sam

1 - window

Current customer: Joe

Sum(

10.0000	Beth
10.0000	Beth
11.0000	Beth
17.0000	Beth
13.0000	Joe
12.0000	Joe
12.0000	Joe
15.0000	Joe
10.0000	Joe

)

2 - window

Current customer: Joe

Sum(

10.0000	Beth
10.0000	Beth
11.0000	Beth
17.0000	Beth
13.0000	Beth
19.0000	Joe
12.0000	Joe
12.0000	Joe
15.0000	Joe
10.0000	Joe

)

Window

select CustomerName

1. *SUM(OrderAmt) OVER (PARTITION BY CustomerName) as PartByName*

2. *SUM(OrderAmt) OVER (ORDER BY CustomerName) as OrderByName*

from XXX order by OrderID;

	Order...	OrderDate	OrderA...	CustomerNa...
1	1	2012-03-01	10.0000	Joe
2	2	2012-03-01	11.0000	Sam
3	3	2012-03-02	10.0000	Beth
4	4	2012-03-02	15.0000	Joe
5	5	2012-03-02	17.0000	Sam
6	6	2012-03-03	12.0000	Joe
7	7	2012-03-04	10.0000	Beth
8	8	2012-03-04	18.0000	Sam
9	9	2012-03-04	12.0000	Joe
10	10	2012-03-04	11.0000	Beth
11	11	2012-03-05	14.0000	Sam
12	12	2012-03-06	17.0000	Beth
13	13	2012-03-06	19.0000	Joe
14	14	2012-03-07	13.0000	Beth
15	15	2012-03-07	16.0000	Sam

1 - window

Current customer: Sam

10.0000	Beth
10.0000	Beth
11.0000	Beth
11.0000	Sam
17.0000	Sam
18.0000	Sam
14.0000	Sam
16.0000	Sam
12.0000	Joe
12.0000	Joe
15.0000	Joe
10.0000	Joe

Sum()

2 - window

Current customer: Sam

OrderA...	CustomerNa...
10.0000	Beth
10.0000	Beth
11.0000	Beth
17.0000	Beth
13.0000	Beth
19.0000	Joe
12.0000	Joe
12.0000	Joe
15.0000	Joe
10.0000	Joe
11.0000	Sam
17.0000	Sam
18.0000	Sam
14.0000	Sam
16.0000	Sam

Sum()

Window

```
select CustomerName
      ,SUM(OrderAmt) OVER () as NoParm
      ,SUM(OrderAmt) OVER (PARTITION BY CustomerName) as PartByName
      ,SUM(OrderAmt) OVER (ORDER BY CustomerName) as OrderByName
from XXX order by CustomerName
```

	Order...	OrderDate	OrderA...	CustomerNa...
1	1	2012-03-01	10.0000	Joe
2	2	2012-03-01	11.0000	Sam
3	3	2012-03-02	10.0000	Beth
4	4	2012-03-02	15.0000	Joe
5	5	2012-03-02	17.0000	Sam
6	6	2012-03-03	12.0000	Joe
7	7	2012-03-04	10.0000	Beth
8	8	2012-03-04	18.0000	Sam
9	9	2012-03-04	12.0000	Joe
10	10	2012-03-04	11.0000	Beth
11	11	2012-03-05	14.0000	Sam
12	12	2012-03-06	17.0000	Beth
13	13	2012-03-06	19.0000	Joe
14	14	2012-03-07	13.0000	Beth
15	15	2012-03-07	16.0000	Sam



	CustomerNa...	NoParm	PartByNa...	OrderByNa...
1	Beth	205.0000	61.0000	61.0000
2	Beth	205.0000	61.0000	61.0000
3	Beth	205.0000	61.0000	61.0000
4	Beth	205.0000	61.0000	61.0000
5	Beth	205.0000	61.0000	61.0000
6	Joe	205.0000	68.0000	129.0000
7	Joe	205.0000	68.0000	129.0000
8	Joe	205.0000	68.0000	129.0000
9	Joe	205.0000	68.0000	129.0000
10	Joe	205.0000	68.0000	129.0000
11	Sam	205.0000	76.0000	205.0000
12	Sam	205.0000	76.0000	205.0000
13	Sam	205.0000	76.0000	205.0000
14	Sam	205.0000	76.0000	205.0000
15	Sam	205.0000	76.0000	205.0000

Window

```
select CustomerName, OrderDate  
      ,SUM(OrderAmt) OVER (PARTITION BY CustomerName) as PartByName  
      ,SUM(OrderAmt) OVER (PARTITION BY CustomerName,OrderDate) as PartByNameDate  
      ,SUM(OrderAmt) OVER (ORDER BY CustomerName,OrderDate) as OrderByName  
from XXX
```

	Order...	OrderDate	OrderA...	CustomerNa...
1	1	2012-03-01	10.0000	Joe
2	2	2012-03-01	11.0000	Sam
3	3	2012-03-02	10.0000	Beth
4	4	2012-03-02	15.0000	Joe
5	5	2012-03-02	17.0000	Sam
6	6	2012-03-03	12.0000	Joe
7	7	2012-03-04	10.0000	Beth
8	8	2012-03-04	18.0000	Sam
9	9	2012-03-04	12.0000	Joe
10	10	2012-03-04	11.0000	Beth
11	11	2012-03-05	14.0000	Sam
12	12	2012-03-06	17.0000	Beth
13	13	2012-03-06	19.0000	Joe
14	14	2012-03-07	13.0000	Beth
15	15	2012-03-07	16.0000	Sam



	CustomerNa...	OrderDate	PartByNa...	PartByNameD...	OrderByNa...
1	Beth	2012-03-02	61.0000	10.0000	10.0000
2	Beth	2012-03-04	61.0000	21.0000	31.0000
3	Beth	2012-03-04	61.0000	21.0000	31.0000
4	Beth	2012-03-06	61.0000	17.0000	48.0000
5	Beth	2012-03-07	61.0000	13.0000	61.0000
6	Joe	2012-03-01	68.0000	10.0000	71.0000
7	Joe	2012-03-02	68.0000	15.0000	86.0000
8	Joe	2012-03-03	68.0000	12.0000	98.0000
9	Joe	2012-03-04	68.0000	12.0000	110.0000
10	Joe	2012-03-06	68.0000	19.0000	129.0000
11	Sam	2012-03-01	76.0000	11.0000	140.0000
12	Sam	2012-03-02	76.0000	17.0000	157.0000
13	Sam	2012-03-04	76.0000	18.0000	175.0000
14	Sam	2012-03-05	76.0000	14.0000	189.0000
15	Sam	2012-03-07	76.0000	16.0000	205.0000

WINDOW

```
select OrderDate, OrderAmount,  
       SUM(OrderAmount) OVER (ORDER BY OrderDate) as VAL  
from XXX order by OrderDate;
```

	Order...	OrderDate	OrderA...	CustomerNa...
1	1	2012-03-01	10.0000	Joe
2	2	2012-03-01	11.0000	Sam
3	3	2012-03-02	10.0000	Beth
4	4	2012-03-02	15.0000	Joe
5	5	2012-03-02	17.0000	Sam
6	6	2012-03-03	12.0000	Joe
7	7	2012-03-04	10.0000	Beth
8	8	2012-03-04	18.0000	Sam
9	9	2012-03-04	12.0000	Joe
10	10	2012-03-04	11.0000	Beth
11	11	2012-03-05	14.0000	Sam
12	12	2012-03-06	17.0000	Beth
13	13	2012-03-06	19.0000	Joe
14	14	2012-03-07	13.0000	Beth
15	15	2012-03-07	16.0000	Sam



	OrderDate	OrderAmount	VAL
1	2012-03-01	10.0000	21.0000
2	2012-03-01	11.0000	21.0000
3	2012-03-02	10.0000	63.0000
4	2012-03-02	15.0000	63.0000
5	2012-03-02	17.0000	63.0000
6	2012-03-03	12.0000	75.0000
7	2012-03-04	10.0000	126.0000
8	2012-03-04	18.0000	126.0000
9	2012-03-04	12.0000	126.0000
10	2012-03-04	11.0000	126.0000
11	2012-03-05	14.0000	140.0000
12	2012-03-06	17.0000	176.0000
13	2012-03-06	19.0000	176.0000
14	2012-03-07	13.0000	205.0000
15	2012-03-07	16.0000	205.0000



OVER
+ ORDER BY
+ ROWS/RANGE

Window

OVER with ROWS or RANGE

specify a set of rows based on the position of the current row

Because each partition is anchored to the current row, the rows or range specifications are based on their proximity to the current row.

both functions require the use of the ORDER BY clause

ROWS clause

limits the rows within a partition by specifying a fixed number of rows preceding or following the current row.

RANGE clause

logically limits the rows within a partition by specifying a range of values with respect to the value in the current row.

Window

Some describe ROWS as a physical operator while RANGE is a logical operator.

The ROWS clause limits the rows within a partition by specifying a fixed number of rows preceding or following the current row.

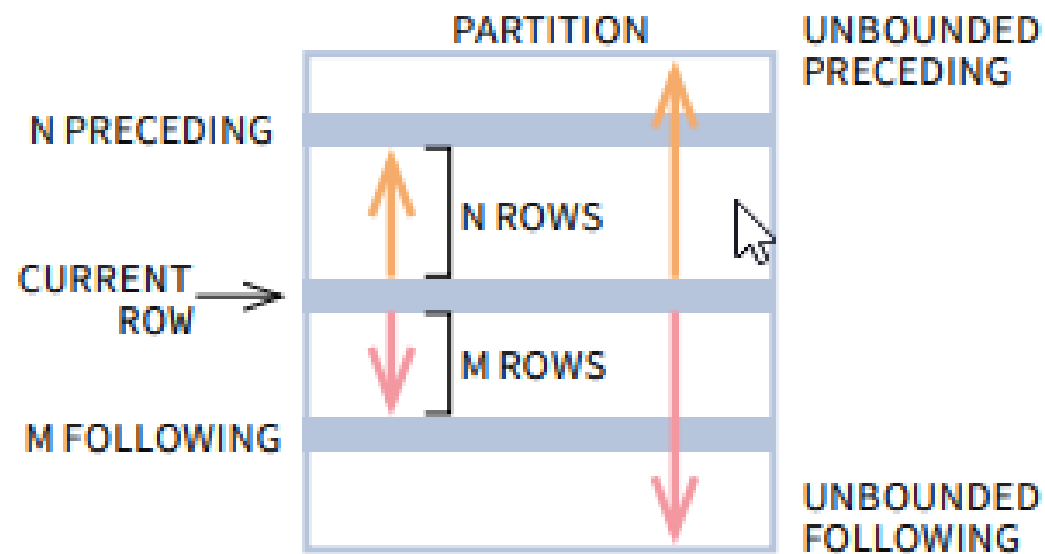
The RANGE clause logically limits the rows within a partition by specifying a range of values with respect to the value in the current row.

Range will include those rows in the window frame which have the same ORDER BY values as the current row.

Thus, it's possible that you can get duplicates with RANGE if the ORDER BY is not unique.

Preceding and following rows are defined based on the ordering in the ORDER BY clause.

Window



Window

ROWS BETWEEN 1 PRECEDING
AND 1 FOLLOWING

	city	sold	month
	Paris	300	1
	Rome	200	1
	Paris	500	2
	Rome	100	4
current row →	Paris	200	4
	Paris	300	5
	Rome	200	5
	London	200	5
	London	100	6
	Rome	300	6

1 row before the current row and
1 row after the current row

RANGE BETWEEN 1 PRECEDING
AND 1 FOLLOWING

	city	sold	month
	Paris	300	1
	Rome	200	1
	Paris	500	2
	Rome	100	4
current row →	Paris	200	4
	Paris	300	5
	Rome	200	5
	London	200	5
	London	100	6
	Rome	300	6

values in the range between 3 and 5
ORDER BY must contain a single expression

GROUPS BETWEEN 1 PRECEDING
AND 1 FOLLOWING

	city	sold	month
	Paris	300	1
	Rome	200	1
	Paris	500	2
	Rome	100	4
current row →	Paris	200	4
	Paris	300	5
	Rome	200	5
	London	200	5
	London	100	6
	Rome	300	6

1 group before the current row and 1 group
after the current row regardless of the value

Code

```
SELECT
  House,
  FullName,
  PhysicalSkill,
  SUM(PhysicalSkill)
    OVER (PARTITION BY House) AS TotalPhysicalSkillByHouse,
  MentalSkill,
  SUM(MentalSkill)
    OVER (PARTITION BY House ORDER BY House ROWS BETWEEN UNBOUNDED PRECEDING AND
CURRENT ROW) AS MentalSkillRows,
  SUM(MentalSkill)
    OVER (PARTITION BY House ORDER BY House RANGE BETWEEN UNBOUNDED PRECEDING
AND CURRENT ROW) AS MentalSkillRange,
  SUM(MentalSkill)
    OVER (PARTITION BY House) AS TotalMentalSkillByHouse
FROM WindowTable
```

House	FullName	PhysicalSkill	TotalPhysicalSkillByHouse	MentalSkill	MentalSkillRows	MentalSkillRange	TotalMentalSkillByHouse
Lannister	Jamie Lannister	7.00	13.00	5.00	5.00	23.00	23.00
Lannister	Tyrian Lannister	3.00	13.00	10.00	15.00	23.00	23.00
Lannister	Tywin Lannister	3.00	13.00	8.00	23.00	23.00	23.00
Stark	Robb Stark	6.00	20.00	6.00	6.00	28.00	28.00
Stark	Ned Stark	8.00	20.00	7.00	13.00	28.00	28.00
Stark	Bran Stark	2.00	20.00	9.00	22.00	28.00	28.00
Stark	Arya Stark	4.00	20.00	6.00	28.00	28.00	28.00

Window

At a first glance, the RANGE syntax is similar to the ROWS syntax.

But the semantic is very different!

Numbers <n> given in this syntax did not specify any counter.

They specify the distance from the value in the current row to the lower or upper boundary.

Therefore the ORDER BY column shall be of type NUMERIC, DATE, or INTERVAL.

Window

```

SELECT  FName,
        Salary,
        SumByRows = SUM(Salary) OVER (ORDER BY Salary
                                      ROWS UNBOUNDED PRECEDING),
        SumByRange = SUM(Salary) OVER (ORDER BY Salary
                                      RANGE UNBOUNDED PRECEDING)
FROM    (VALUES (1, 'George',      800),
          (2, 'Sam',              950),
          (3, 'Diane',           1100),
          (4, 'Nicholas',        1250),
          (5, 'Samuel',          1250),
          (6, 'Patricia',        1300),
          (7, 'Brian',           1500),
          (8, 'Thomas',          1600),
          (9, 'Fran',           2450),
          (10, 'Debbie',         2850),
          (11, 'Mark',           2975),
          (12, 'James',          3000),
          (13, 'Cynthia',        3000),
          (14, 'Christopher',    5000)
        ) dt(RowID, FName, Salary);

```

	FName	Salary	SumByRows	SumByRange
1	George	800	800	800
2	Sam	950	1750	1750
3	Diane	1100	2850	2850
4	Samuel	1250	4100	5350
5	Nicholas	1250	5350	5350
6	Patricia	1300	6650	6650
7	Brian	1500	8150	8150
8	Thomas	1600	9750	9750
9	Fran	2450	12200	12200
10	Debbie	2850	15050	15050
11	Mark	2975	18025	18025
12	Cynthia	3000	21025	24025
13	James	3000	24025	24025
14	Christopher	5000	29025	29025

Window

Example

```
SELECT
  id, dep_name, salary,
  SUM(salary) OVER (
    PARTITION BY dep_name
    ORDER BY salary
    RANGE BETWEEN 100 PRECEDING
    AND 50 FOLLOWING
  ) AS sum_over_range
FROM employee;
```

Example

Leads to the sum over all rows
which have
*a salary from 100 below and 50
over the actual row.*

Window

A number of key words can be used with this clause to define the window frame used by the window functions.

CURRENT ROW:

identifies the current row

supported by ROWS and RANGE

UNBOUNDED PRECEDING:

the beginning of the partition

supported by both ROWS and RANGE.

UNBOUNDED FOLLOWING:

the end of the partition

supported by both ROWS and RANGE.

Window

A number of key words can be used with this clause to define the window frame used by the window functions.

n PRECEDING:

This is used to specify the number of rows before the current row supported by the ROWS clause only.

n FOLLOWING:

This is used to specify the number of rows after the current row supported by the ROWS clause only.

Preceding and following rows

are defined based on the ordering in the ORDER BY clause

Window

A number of key words can be used with this clause to define the window frame used by the window functions.

BETWEEN X AND Y

Used with either ROWS or RANGE to specify the lower (starting) and upper (ending) boundary points of the window.

X - <window frame bound> defines the boundary starting point

Y - <window frame bound> defines the boundary end point

The upper bound cannot be smaller than the lower bound.

BETWEEN rows

defined based on the ordering in the ORDER BY clause

Window

For example,

ROWS BETWEEN 2 PRECEDING AND CURRENT ROW

defines a window that has three rows in size, starting with 2 rows preceding, until and including the current row.

ROWS BETWEEN 2 FOLLOWING AND 10 FOLLOWING

defines a window that starts with the second row that follows the current row and ends with the tenth row that follows the current row

RANGE BETWEEN CURRENT ROW AND UNBOUNDED FOLLOWING

defines a window that starts with the current row and ends with the last row of the partition

The window frame “RANGE ... CURRENT ROW ...”

include all rows that have the same values in the ORDER BY expression as the current row.

Window

select CustomerName,OrderDate,OrderAmt
 , *SUM(OrderAmt) OVER (PARTITION BY CustomerName ORDER BY OrderDate
 RANGE BETWEEN 2 PRECEDING AND CURRENT ROW) as RangeRow*
from XXX

	Order...	OrderDate	OrderA...	CustomerNa...
1	1	2012-03-01	10.0000	Joe
2	2	2012-03-01	11.0000	Sam
3	3	2012-03-02	10.0000	Beth
4	4	2012-03-02	15.0000	Joe
5	5	2012-03-02	17.0000	Sam
6	6	2012-03-03	12.0000	Joe
7	7	2012-03-04	10.0000	Beth
8	8	2012-03-04	18.0000	Sam
9	9	2012-03-04	12.0000	Joe
10	10	2012-03-04	11.0000	Beth
11	11	2012-03-05	14.0000	Sam
12	12	2012-03-06	17.0000	Beth
13	13	2012-03-06	19.0000	Joe
14	14	2012-03-07	13.0000	Beth
15	15	2012-03-07	16.0000	Sam

Current customer: Sam

10.0000	Beth
10.0000	Beth
11.0000	Beth
11.0000	Sam
17.0000	Sam
18.0000	Sam
14.0000	Sam
16.0000	Sam
12.0000	Joe
12.0000	Joe
15.0000	Joe
10.0000	Joe

Sum(

11.0000	Sam
17.0000	Sam
18.0000	Sam

Window

```
select CustomerName,OrderDate,OrderAmt  
      ,SUM(OrderAmt) OVER (PARTITION BY CustomerName ORDER BY OrderDate  
                           RANGE CURRENT ROW) as RangeRow  
      ,SUM(OrderAmt) OVER (PARTITION BY CustomerName ORDER BY OrderDate  
                           ROWS CURRENT ROW) as RowRow  
from XXX
```

	Order...	OrderDate	OrderA...	CustomerNa...
1	1	2012-03-01	10.0000	Joe
2	2	2012-03-01	11.0000	Sam
3	3	2012-03-02	10.0000	Beth
4	4	2012-03-02	15.0000	Joe
5	5	2012-03-02	17.0000	Sam
6	6	2012-03-03	12.0000	Joe
7	7	2012-03-04	10.0000	Beth
8	8	2012-03-04	18.0000	Sam
9	9	2012-03-04	12.0000	Joe
10	10	2012-03-04	11.0000	Beth
11	11	2012-03-05	14.0000	Sam
12	12	2012-03-06	17.0000	Beth
13	13	2012-03-06	19.0000	Joe
14	14	2012-03-07	13.0000	Beth
15	15	2012-03-07	16.0000	Sam



	CustomerNa...	OrderDate	OrderA...	RangeR...	RowRow
1	Beth	2012-03-02	10.0000	10.0000	10.0000
2	Beth	2012-03-04	10.0000	21.0000	10.0000
3	Beth	2012-03-04	11.0000	21.0000	11.0000
4	Beth	2012-03-06	17.0000	17.0000	17.0000
5	Beth	2012-03-07	13.0000	13.0000	13.0000
6	Joe	2012-03-01	10.0000	10.0000	10.0000
7	Joe	2012-03-02	15.0000	15.0000	15.0000
8	Joe	2012-03-03	12.0000	12.0000	12.0000
9	Joe	2012-03-04	12.0000	12.0000	12.0000
10	Joe	2012-03-06	19.0000	19.0000	19.0000
11	Sam	2012-03-01	11.0000	11.0000	11.0000
12	Sam	2012-03-02	17.0000	17.0000	17.0000
13	Sam	2012-03-04	18.0000	18.0000	18.0000
14	Sam	2012-03-05	14.0000	14.0000	14.0000
15	Sam	2012-03-07	16.0000	16.0000	16.0000

Window

select CustomerName

,SUM(OrderAmt) OVER (PARTITION BY CustomerName ORDER BY OrderDate RANGE BETWEEN UNBOUNDED PRECEDING and CURRENT ROW) as RangeByDate

,SUM(OrderAmt) OVER (PARTITION BY CustomerName ORDER BY OrderDate ROWS BETWEEN UNBOUNDED PRECEDING and CURRENT ROW) as RangeByDate

from XXX

	Order...	OrderDate	OrderA...	CustomerNa...
1	1	2012-03-01	10.0000	Joe
2	2	2012-03-01	11.0000	Sam
3	3	2012-03-02	10.0000	Beth
4	4	2012-03-02	15.0000	Joe
5	5	2012-03-02	17.0000	Sam
6	6	2012-03-03	12.0000	Joe
7	7	2012-03-04	10.0000	Beth
8	8	2012-03-04	18.0000	Sam
9	9	2012-03-04	12.0000	Joe
10	10	2012-03-04	11.0000	Beth
11	11	2012-03-05	14.0000	Sam
12	12	2012-03-06	17.0000	Beth
13	13	2012-03-06	19.0000	Joe
14	14	2012-03-07	13.0000	Beth
15	15	2012-03-07	16.0000	Sam



	CustomerNa...	RangeByD...	RangeByD...
1	Beth	10.0000	10.0000
2	Beth	31.0000	20.0000
3	Beth	31.0000	31.0000
4	Beth	48.0000	48.0000
5	Beth	61.0000	61.0000
6	Joe	10.0000	10.0000
7	Joe	25.0000	25.0000
8	Joe	37.0000	37.0000
9	Joe	49.0000	49.0000
10	Joe	68.0000	68.0000
11	Sam	11.0000	11.0000
12	Sam	28.0000	28.0000
13	Sam	46.0000	46.0000
14	Sam	60.0000	60.0000
15	Sam	76.0000	76.0000

Window

select CustomerName

,SUM(OrderAmt) OVER (PARTITION BY CustomerName ORDER BY OrderDate ROWS BETWEEN 1 PRECEDING and CURRENT ROW) as RangeByDate

,SUM(OrderAmt) OVER (PARTITION BY CustomerName ORDER BY OrderDate ROWS BETWEEN CURRENT ROW and 1 FOLLOWING) as RangeByDate

from XXX

	Order...	OrderDate	OrderA...	CustomerNa...
1	1	2012-03-01	10.0000	Joe
2	2	2012-03-01	11.0000	Sam
3	3	2012-03-02	10.0000	Beth
4	4	2012-03-02	15.0000	Joe
5	5	2012-03-02	17.0000	Sam
6	6	2012-03-03	12.0000	Joe
7	7	2012-03-04	10.0000	Beth
8	8	2012-03-04	18.0000	Sam
9	9	2012-03-04	12.0000	Joe
10	10	2012-03-04	11.0000	Beth
11	11	2012-03-05	14.0000	Sam
12	12	2012-03-06	17.0000	Beth
13	13	2012-03-06	19.0000	Joe
14	14	2012-03-07	13.0000	Beth
15	15	2012-03-07	16.0000	Sam



	CustomerNa...	RangeByD...	RangeByD...
1	Beth	10.0000	20.0000
2	Beth	20.0000	21.0000
3	Beth	21.0000	28.0000
4	Beth	28.0000	30.0000
5	Beth	30.0000	13.0000
6	Joe	10.0000	25.0000
7	Joe	25.0000	27.0000
8	Joe	27.0000	24.0000
9	Joe	24.0000	31.0000
10	Joe	31.0000	19.0000
11	Sam	11.0000	28.0000
12	Sam	28.0000	35.0000
13	Sam	35.0000	32.0000
14	Sam	32.0000	30.0000
15	Sam	30.0000	16.0000

Window

GROUP BY over a query with OVER:

```
SELECT Sum(OrderAmt) AS SAMT, OrderDate  
      ,SUM(SUM(OrderAmt)) OVER (ORDER BY OrderDate ROWS BETWEEN 1  
      PRECEDING AND CURRENT ROW) as RangeRow
```

From XXX

group by OrderDate

	Order...	OrderDate	OrderA...	CustomerNa...
1	1	2012-03-01	10.0000	Joe
2	2	2012-03-01	11.0000	Sam
3	3	2012-03-02	10.0000	Beth
4	4	2012-03-02	15.0000	Joe
5	5	2012-03-02	17.0000	Sam
6	6	2012-03-03	12.0000	Joe
7	7	2012-03-04	10.0000	Beth
8	8	2012-03-04	18.0000	Sam
9	9	2012-03-04	12.0000	Joe
10	10	2012-03-04	11.0000	Beth
11	11	2012-03-05	14.0000	Sam
12	12	2012-03-06	17.0000	Beth
13	13	2012-03-06	19.0000	Joe
14	14	2012-03-07	13.0000	Beth
15	15	2012-03-07	16.0000	Sam



	SAMT	OrderDate	RangeR...
1	21.0000	2012-03-01	21.0000
2	42.0000	2012-03-02	63.0000
3	12.0000	2012-03-03	54.0000
4	51.0000	2012-03-04	63.0000
5	14.0000	2012-03-05	65.0000
6	36.0000	2012-03-06	50.0000
7	29.0000	2012-03-07	65.0000

Window

```
SELECT SalesOrderID, ProductID, OrderQty
      ,SUM(OrderQty) OVER(PARTITION BY SalesOrderID) AS Total
      ,AVG(OrderQty) OVER(PARTITION BY SalesOrderID) AS "Avg"
      ,COUNT(OrderQty) OVER(PARTITION BY SalesOrderID) AS "Count"
      ,MIN(OrderQty) OVER(PARTITION BY SalesOrderID) AS "Min"
      ,MAX(OrderQty) OVER(PARTITION BY SalesOrderID) AS "Max"
FROM Sales.SalesOrderDetail
WHERE SalesOrderID = 43659;
```

SalesOrderID	ProductID	OrderQty	Total	Avg	Count	Min	Max
43659	776	1	26	2	12	1	6
43659	777	3	26	2	12	1	6
43659	778	1	26	2	12	1	6
43659	771	1	26	2	12	1	6
43659	772	1	26	2	12	1	6
43659	773	2	26	2	12	1	6
43659	774	1	26	2	12	1	6
43659	714	3	26	2	12	1	6
43659	716	1	26	2	12	1	6
43659	709	6	26	2	12	1	6
43659	712	2	26	2	12	1	6
43659	711	4	26	2	12	1	6

Window

```
SELECT SalesOrderID, ProductID, OrderQty
      ,SUM(OrderQty) OVER(PARTITION BY SalesOrderID) AS Total
      ,CAST(1.*OrderQty/SUM(OrderQty) OVER(PARTITION BY SalesOrderID)
      *100 AS DECIMAL(5,2))AS "Percent by ProductID"
FROM Sales.SalesOrderDetail
WHERE SalesOrderID = 43659;
```

SalesOrderID	ProductID	OrderQty	Total	Percent by ProductID
43659	776	1	26	3.85
43659	777	3	26	11.54
43659	778	1	26	3.85
43659	771	1	26	3.85
43659	772	1	26	3.85
43659	773	2	26	7.69
43659	774	1	26	3.85
43659	714	3	26	11.54
43659	716	1	26	3.85
43659	709	6	26	23.08
43659	712	2	26	7.69
43659	711	4	26	15.38

Window

```
SELECT BusinessEntityID, TerritoryID, DATEPART(yy,ModifiedDate) AS SalesYear,  
SalesYTD  
    , AVG(SalesYTD) OVER (PARTITION BY TerritoryID  
                        ORDER BY DATEPART(yy,ModifiedDate)  
                        ) AS MovingAvg  
    , SUM(SalesYTD) OVER (PARTITION BY TerritoryID  
                        ORDER BY DATEPART(yy,ModifiedDate)  
                        ) AS CumulativeTotal  
FROM Sales.SalesPerson  
WHERE TerritoryID IS NULL OR TerritoryID < 5  
ORDER BY TerritoryID,SalesYear;
```

BusinessEntityID	TerritoryID	SalesYear	SalesYTD	MovingAvg	CumulativeTotal
274	NULL	2005	559,697.56	559,697.56	559,697.56
287	NULL	2006	519,905.93	539,801.75	1,079,603.50
285	NULL	2007	172,524.45	417,375.98	1,252,127.95
283	1	2005	1,573,012.94	1,462,795.04	2,925,590.07
280	1	2005	1,352,577.13	1,462,795.04	2,925,590.07
284	1	2006	1,576,562.20	1,500,717.42	4,502,152.27
275	2	2005	3,763,178.18	3,763,178.18	3,763,178.18
277	3	2005	3,189,418.37	3,189,418.37	3,189,418.37
276	4	2005	4,251,368.55	3,354,952.08	6,709,904.17
281	4	2005	2,458,535.62	3,354,952.08	6,709,904.17



Ranking

Ranking

Ranking

Ranking functions return a ranking value for each row in a partition

Depending on the function that is used

some rows might receive the same value as other rows.

Ranking functions are nondeterministic

Examples:

ROW_NUMBER, RANK, DENSE_RANK, NTILE

Ranking

ROW_NUMBER

- return a row number for each row within the partition based on the partition and order.

- requires the use of the ORDER BY clause.

- If PARTITION BY is used, then the row numbering starts over within the partition.

Ranking

```
select CustomerName,OrderDate,OrderAmt
      ,ROW_NUMBER() OVER(ORDER BY OrderAmt DESC) RowNum
      ,ROW_NUMBER() OVER(PARTITION BY OrderDate ORDER BY
      CustomerName) RowNum
FROM lab.CTEOrders ORDER BY OrderDate
```

	Order...	OrderDate	OrderA...	CustomerNa...
1	1	2012-03-01	10.0000	Joe
2	2	2012-03-01	11.0000	Sam
3	3	2012-03-02	10.0000	Beth
4	4	2012-03-02	15.0000	Joe
5	5	2012-03-02	17.0000	Sam
6	6	2012-03-03	12.0000	Joe
7	7	2012-03-04	10.0000	Beth
8	8	2012-03-04	18.0000	Sam
9	9	2012-03-04	12.0000	Joe
10	10	2012-03-04	11.0000	Beth
11	11	2012-03-05	14.0000	Sam
12	12	2012-03-06	17.0000	Beth
13	13	2012-03-06	19.0000	Joe
14	14	2012-03-07	13.0000	Beth
15	15	2012-03-07	16.0000	Sam



	CustomerNa...	OrderDate	OrderA...	RowNumByC...	RowNumP...
1	Joe	2012-03-01	10.0000	14	1
2	Sam	2012-03-01	11.0000	12	2
3	Beth	2012-03-02	10.0000	13	1
4	Joe	2012-03-02	15.0000	6	2
5	Sam	2012-03-02	17.0000	3	3
6	Joe	2012-03-03	12.0000	9	1
7	Beth	2012-03-04	10.0000	15	1
8	Beth	2012-03-04	11.0000	11	2
9	Joe	2012-03-04	12.0000	10	3
10	Sam	2012-03-04	18.0000	2	4
11	Sam	2012-03-05	14.0000	7	1
12	Beth	2012-03-06	17.0000	4	1
13	Joe	2012-03-06	19.0000	1	2
14	Beth	2012-03-07	13.0000	8	1
15	Sam	2012-03-07	16.0000	5	2

Ranking

RANK and DENSE_RANK

order the rows based on the specified partition and apply a rank or number to them.

assign the same rank to “ties”.

The difference is how it handles the next rank number in the series.

RANK does a “true” ordering and will apply the ranking based on the number of rows and skip numbers that are ties.

DENSE_RANK keeps the tie as well, however it does not skip any numbers in the sequence.

Ranking

```
select CustomerName,OrderDate,OrderAmt
  ,RANK() OVER(ORDER BY CustomerName) RankByCust
  ,DENSE_RANK() OVER(ORDER BY CustomerName) DenseByCust
  ,RANK() OVER(PARTITION BY CustomerName ORDER BY OrderDate) RankByCustDt
  ,DENSE_RANK() OVER(PARTITION BY CustomerName ORDER BY OrderDate) DenseByCustDt
from lab.CTEOrders order by CustomerName
```

	Order...	OrderDate	OrderA...	CustomerNa...
1	1	2012-03-01	10.0000	Joe
2	2	2012-03-01	11.0000	Sam
3	3	2012-03-02	10.0000	Beth
4	4	2012-03-02	15.0000	Joe
5	5	2012-03-02	17.0000	Sam
6	6	2012-03-03	12.0000	Joe
7	7	2012-03-04	10.0000	Beth
8	8	2012-03-04	18.0000	Sam
9	9	2012-03-04	12.0000	Joe
10	10	2012-03-04	11.0000	Beth
11	11	2012-03-05	14.0000	Sam
12	12	2012-03-06	17.0000	Beth
13	13	2012-03-06	19.0000	Joe
14	14	2012-03-07	13.0000	Beth
15	15	2012-03-07	16.0000	Sam



	CustomerNa...	OrderDate	OrderA...	RankByC...	DenseByC...	RankByCust...	DenseByCust...
1	Beth	2012-03-02	10.0000	1	1	1	1
2	Beth	2012-03-04	10.0000	1	1	2	2
3	Beth	2012-03-04	11.0000	1	1	2	2
4	Beth	2012-03-06	17.0000	1	1	4	3
5	Beth	2012-03-07	13.0000	1	1	5	4
6	Joe	2012-03-01	10.0000	6	2	1	1
7	Joe	2012-03-02	15.0000	6	2	2	2
8	Joe	2012-03-03	12.0000	6	2	3	3
9	Joe	2012-03-04	12.0000	6	2	4	4
10	Joe	2012-03-06	19.0000	6	2	5	5
11	Sam	2012-03-01	11.0000	11	3	1	1
12	Sam	2012-03-02	17.0000	11	3	2	2
13	Sam	2012-03-04	18.0000	11	3	3	3
14	Sam	2012-03-05	14.0000	11	3	4	4
15	Sam	2012-03-07	16.0000	11	3	5	5

Ranking

NTILE

groups the data into ordered and ranked groups

based on the ORDER BY clause

based on the number of groups used

specified as function's argument

if you specify 4 groups to produce a quartile ranking,

4 ranked values (1-4) will be assigned to each group based on the order

In many cases, the total numbers of rows is not divisible by the number of groups chosen

The function will always frontload the results so the earliest groups will have the "extra" rows.

Ranking

NTILE

If a PARTITION BY clause is used, the NTILE ranking will be applied within each partition

As with the other ranking functions, the ORDER BY clause is required to use this function

Example:

NTILE(4) – quartile

Each group represents $\frac{1}{4}$ of the dataset

NTILE(100) – percentage

Each group represents 1% of the dataset

Ranking

```
select CustomerName,OrderDate,OrderAmt  
      ,NTILE(4) OVER(ORDER BY OrderDate) NtileByDt  
      ,NTILE(2) OVER(PARTITION BY OrderDate ORDER BY OrderAmt)  
      NtileByDtAmt  
from XXX order by OrderDate;
```

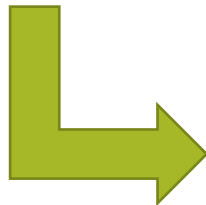
	Order...	OrderDate	OrderA...	CustomerNa...
1	1	2012-03-01	10.0000	Joe
2	2	2012-03-01	11.0000	Sam
3	3	2012-03-02	10.0000	Beth
4	4	2012-03-02	15.0000	Joe
5	5	2012-03-02	17.0000	Sam
6	6	2012-03-03	12.0000	Joe
7	7	2012-03-04	10.0000	Beth
8	8	2012-03-04	18.0000	Sam
9	9	2012-03-04	12.0000	Joe
10	10	2012-03-04	11.0000	Beth
11	11	2012-03-05	14.0000	Sam
12	12	2012-03-06	17.0000	Beth
13	13	2012-03-06	19.0000	Joe
14	14	2012-03-07	13.0000	Beth
15	15	2012-03-07	16.0000	Sam



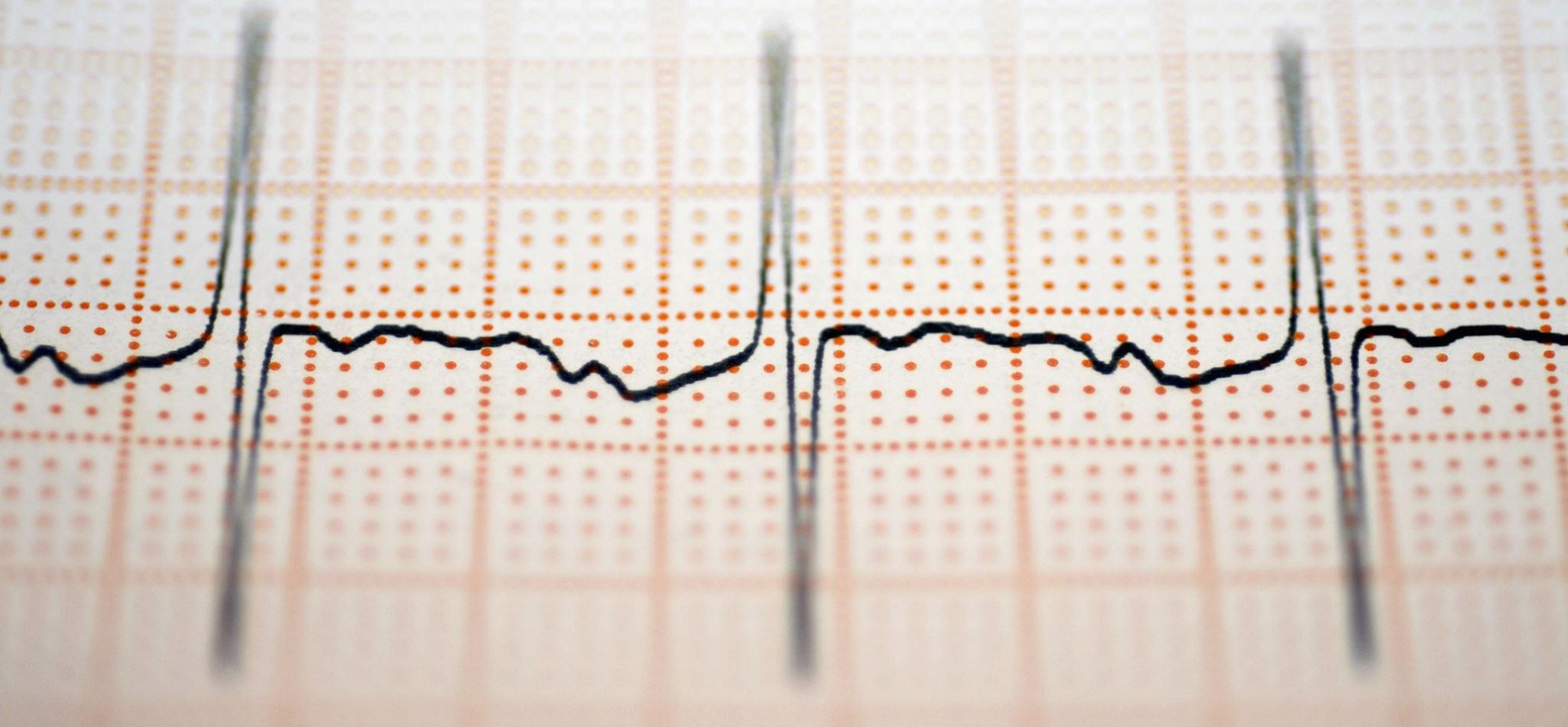
	CustomerNa...	OrderDate	OrderA...	NtileBy...	NtileByDtAmt
1	Joe	2012-03-01	10.0000	1	1
2	Sam	2012-03-01	11.0000	1	2
3	Beth	2012-03-02	10.0000	1	1
4	Joe	2012-03-02	15.0000	1	1
5	Sam	2012-03-02	17.0000	2	2
6	Joe	2012-03-03	12.0000	2	1
7	Beth	2012-03-04	10.0000	2	1
8	Beth	2012-03-04	11.0000	2	1
9	Joe	2012-03-04	12.0000	3	2
10	Sam	2012-03-04	18.0000	3	2
11	Sam	2012-03-05	14.0000	3	1
12	Beth	2012-03-06	17.0000	3	1
13	Joe	2012-03-06	19.0000	4	2
14	Beth	2012-03-07	13.0000	4	1
15	Sam	2012-03-07	16.0000	4	2

Ranking

```
USE AdventureWorks2012;
GO
SELECT p.FirstName, p.LastName
      ,ROW_NUMBER() OVER (ORDER BY a.PostalCode) AS "Row Number"
      ,RANK() OVER (ORDER BY a.PostalCode) AS Rank
      ,DENSE_RANK() OVER (ORDER BY a.PostalCode) AS "Dense Rank"
      ,NTILE(4) OVER (ORDER BY a.PostalCode) AS Quartile
      ,s.SalesYTD
      ,a.PostalCode
FROM Sales.SalesPerson AS s
     INNER JOIN Person.Person AS p
          ON s.BusinessEntityID = p.BusinessEntityID
     INNER JOIN Person.Address AS a
          ON a.AddressID = p.BusinessEntityID
WHERE TerritoryID IS NOT NULL AND SalesYTD <> 0;
```



FirstName	LastName	Row Number	Rank	Dense Rank	Quartile	SalesYTD	PostalCode
Michael	Blythe	1	1	1	1	4557045.0459	98027
Linda	Mitchell	2	1	1	1	5200475.2313	98027
Jillian	Carson	3	1	1	1	3857163.6332	98027
Garrett	Vargas	4	1	1	1	1764938.9859	98027
Tsvi	Reiter	5	1	1	2	2811012.7151	98027
Shu	Ito	6	6	2	2	3018725.4858	98055
José	Saraiva	7	6	2	2	3189356.2465	98055
David	Campbell	8	6	2	3	3587378.4257	98055
Tete	Mensa-Annan	9	6	2	3	1931620.1835	98055
Lynn	Tsoflias	10	6	2	3	1758385.926	98055
Rachel	Valdez	11	6	2	4	2241204.0424	98055
Jae	Pak	12	6	2	4	5015682.3752	98055
Ranjit	Varkey Chudukatil	13	6	2	4	3827950.238	98055



Analytical

Analytical

Analytic functions fall in to two primary categories:
values at a position and percentiles

Functions:

Position Value Functions:

LAG, LEAD, FIRST_VALUE and LAST_VALUE

find a row in the partition and return the desired value from that row.

Percentile Functions:

CUME_DIST and PERCENT_RANK

break the partition into percentiles and return a rank value for each row.

Position

Position Value Functions:

LAG, LEAD, FIRST_VALUE, LAST_VALUE

Allow the usage of values from other rows in the current row

The LAG and LEAD functions

allow to specify the offset or how many rows to look forward (LEAD) or backward (LAG)

support a default value in cases where the value returned would be null.

do not support the use of ROWS or RANGE in the OVER clause.

The FIRST_VALUE and LAST_VALUE

allow you to further define the partition using ROWS or RANGE if desired.

lag(sold) OVER(ORDER BY month)

order by month ↓	month	sold	
	1	500	NULL
	2	300	500
	3	400	300
	4	100	400
	5	500	100

lead(sold) OVER(ORDER BY month)

order by month ↓	month	sold	
	1	500	300
	2	300	400
	3	400	100
	4	100	500
	5	500	NULL

lag(sold, 2, 0) OVER(ORDER BY month)

order by month ↓	month	sold	
	1	500	0
	2	300	0
	3	400	500
	4	100	300
	5	500	400

offset=2
↓

lead(sold, 2, 0) OVER(ORDER BY month)

order by month ↓	month	sold	
	1	500	400
	2	300	100
	3	400	500
	4	100	0
	5	500	0

offset=2
↑

Position

```
SELECT OrderID,OrderDate,OrderAmt,CustomerName
,LAG(OrderAmt) OVER (PARTITION BY CustomerName ORDER BY OrderID) as PrevOrdAmt
,LEAD(OrderAmt, 2) OVER (PARTITION BY CustomerName ORDER BY OrderID) as
NextTwoOrdAmt
,LEAD(OrderDate, 2, '9999-12-31') OVER (PARTITION BY CustomerName ORDER BY OrderID)
as NextTwoOrdDtNoNull
FROM lab.CTEOrders order by CustomerName
```

	Order...	OrderDate	OrderA...	CustomerNa...
1	1	2012-03-01	10.0000	Joe
2	2	2012-03-01	11.0000	Sam
3	3	2012-03-02	10.0000	Beth
4	4	2012-03-02	15.0000	Joe
5	5	2012-03-02	17.0000	Sam
6	6	2012-03-03	12.0000	Joe
7	7	2012-03-04	10.0000	Beth
8	8	2012-03-04	18.0000	Sam
9	9	2012-03-04	12.0000	Joe
10	10	2012-03-04	11.0000	Beth
11	11	2012-03-05	14.0000	Sam
12	12	2012-03-06	17.0000	Beth
13	13	2012-03-06	19.0000	Joe
14	14	2012-03-07	13.0000	Beth
15	15	2012-03-07	16.0000	Sam



	Order...	OrderDate	OrderA...	CustomerNa...	PrevOrdA...	NextTwoOrdA...	NextTwoOrdDtNo...
1	3	2012-03-02	10.0000	Beth	NULL	11.0000	2012-03-04
2	7	2012-03-04	10.0000	Beth	10.0000	17.0000	2012-03-06
3	10	2012-03-04	11.0000	Beth	10.0000	13.0000	2012-03-07
4	12	2012-03-06	17.0000	Beth	11.0000	NULL	9999-12-31
5	14	2012-03-07	13.0000	Beth	17.0000	NULL	9999-12-31
6	1	2012-03-01	10.0000	Joe	NULL	12.0000	2012-03-03
7	4	2012-03-02	15.0000	Joe	10.0000	12.0000	2012-03-04
8	6	2012-03-03	12.0000	Joe	15.0000	19.0000	2012-03-06
9	9	2012-03-04	12.0000	Joe	12.0000	NULL	9999-12-31
10	13	2012-03-06	19.0000	Joe	12.0000	NULL	9999-12-31
11	2	2012-03-01	11.0000	Sam	NULL	18.0000	2012-03-04
12	5	2012-03-02	17.0000	Sam	11.0000	14.0000	2012-03-05
13	8	2012-03-04	18.0000	Sam	17.0000	16.0000	2012-03-07
14	11	2012-03-05	14.0000	Sam	18.0000	NULL	9999-12-31
15	15	2012-03-07	16.0000	Sam	14.0000	NULL	9999-12-31

Position

```
SELECT OrderID,OrderDate,OrderAmt,CustomerName
      ,FIRST_VALUE(OrderDate) OVER (PARTITION BY CustomerName ORDER BY
      OrderID) as FirstOrdDt
      ,LAST_VALUE(CustomerName) OVER (PARTITION BY OrderDate ORDER BY
      OrderID ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED
      FOLLOWING) as LastCustToOrdByDay
FROM lab.CTEOrders order by OrderID
```

	Order...	OrderDate	OrderA...	CustomerNa...
1	1	2012-03-01	10.0000	Joe
2	2	2012-03-01	11.0000	Sam
3	3	2012-03-02	10.0000	Beth
4	4	2012-03-02	15.0000	Joe
5	5	2012-03-02	17.0000	Sam
6	6	2012-03-03	12.0000	Joe
7	7	2012-03-04	10.0000	Beth
8	8	2012-03-04	18.0000	Sam
9	9	2012-03-04	12.0000	Joe
10	10	2012-03-04	11.0000	Beth
11	11	2012-03-05	14.0000	Sam
12	12	2012-03-06	17.0000	Beth
13	13	2012-03-06	19.0000	Joe
14	14	2012-03-07	13.0000	Beth
15	15	2012-03-07	16.0000	Sam



	Order...	OrderDate	OrderA...	CustomerNa...	FirstOrdDt	LastCustToOrdBy...
1	1	2012-03-01	10.0000	Joe	2012-03-01	Sam
2	2	2012-03-01	11.0000	Sam	2012-03-01	Sam
3	3	2012-03-02	10.0000	Beth	2012-03-02	Sam
4	4	2012-03-02	15.0000	Joe	2012-03-01	Sam
5	5	2012-03-02	17.0000	Sam	2012-03-01	Sam
6	6	2012-03-03	12.0000	Joe	2012-03-01	Joe
7	7	2012-03-04	10.0000	Beth	2012-03-02	Beth
8	8	2012-03-04	18.0000	Sam	2012-03-01	Beth
9	9	2012-03-04	12.0000	Joe	2012-03-01	Beth
10	10	2012-03-04	11.0000	Beth	2012-03-02	Beth
11	11	2012-03-05	14.0000	Sam	2012-03-01	Sam
12	12	2012-03-06	17.0000	Beth	2012-03-02	Joe
13	13	2012-03-06	19.0000	Joe	2012-03-01	Joe
14	14	2012-03-07	13.0000	Beth	2012-03-02	Sam
15	15	2012-03-07	16.0000	Sam	2012-03-01	Sam

Position

```
SELECT id,
       emp_name,
       dep_name,
       ROW_NUMBER()      OVER (PARTITION BY dep_name ORDER BY id) AS row_number_in_frame,
       NTH_VALUE(emp_name, 2) OVER (PARTITION BY dep_name ORDER BY id) AS second_row_in_frame,
       LEAD(emp_name, 2)   OVER (PARTITION BY dep_name ORDER BY id) AS two_rows_ahead
FROM   employee;
```

ID	EMP_NAME	DEP_NAME	ROW_NUMBER_IN_FRAME	SECOND_ROW_IN_FRAME	TWO_ROWS_AHEAD
1	Matthew	Management	1	-	Grace
2	Olivia	Management	2	Olivia	-
3	Grace	Management	3	Olivia	-
4	Jim	Production	1	-	Michael
5	Alice	Production	2	Alice	Tom
6	Michael	Production	3	Alice	Kevin
7	Tom	Production	4	Alice	-
8	Kevin	Production	5	Alice	-
10	Sophia	Sales	1	-	-
11	Samantha	Sales	2	Samantha	-
9	Elvis	Service	1	-	-

Percentile

Percentile Functions:

CUME_DIST, PERCENT_RANK

return a ranking value

as the ranking percentages move from lowest to highest, each group's percent value includes all of the earlier values in the calculation as well

CUME_DIST

(0,1] that represents the percent of rows that have a value less than or equal to the current in the group

first group is the percentage

PERCENT_RANK

computes the rank of the value within a group as a percentage

[0,1] with the first group is always 0

Percentile

```
SELECT Department, LastName, Rate,
       CUME_DIST ()
         OVER (PARTITION BY Department ORDER BY Rate) AS CumeDist,
       PERCENT_RANK()
         OVER (PARTITION BY Department ORDER BY Rate ) AS PctRank
FROM HumanResources.vEmployeeDepartmentHistory AS
edh INNER JOIN HumanResources.EmployeePayHistory AS e
ON e.BusinessEntityID = edh.BusinessEntityID
WHERE Department IN (N'Information Services',N'Document
Control') ORDER BY Department, Rate DESC;
```

Department	LastName	Rate	CumeDist	PctRank
Document Control	Arifin	17.7885	1	1
Document Control	Norred	16.8269	0.8	0.5
Document Control	Kharatishvili	16.8269	0.8	0.5
Document Control	Chai	10.25	0.4	0
Document Control	Berge	10.25	0.4	0
Information Services	Trenary	50.4808	1	1
Information Services	Conroy	39.6635	0.9	0.888888888
Information Services	Ajenstat	38.4615	0.8	0.666666666
Information Services	Wilson	38.4615	0.8	0.666666666
Information Services	Sharma	32.4519	0.6	0.444444444
Information Services	Connelly	32.4519	0.6	0.444444444
Information Services	Berg	27.4038	0.4	0
Information Services	Meyyappan	27.4038	0.4	0
Information Services	Bacon	27.4038	0.4	0
Information Services	Bueno	27.4038	0.4	0
(15 row(s) affected)				

Percentile

```
SELECT OrderDate, CustomerName  
      , CUME_DIST() OVER (Partition by CustomerName ORDER BY OrderDate)  
      CumDist  
      , PERCENT_RANK() OVER (Partition by CustomerName ORDER BY OrderDate)  
      PctRank  
from lab.CTEOrders order by CustomerName, OrderDate
```

	Order...	OrderDate	OrderA...	CustomerNa...
1	1	2012-03-01	10.0000	Joe
2	2	2012-03-01	11.0000	Sam
3	3	2012-03-02	10.0000	Beth
4	4	2012-03-02	15.0000	Joe
5	5	2012-03-02	17.0000	Sam
6	6	2012-03-03	12.0000	Joe
7	7	2012-03-04	10.0000	Beth
8	8	2012-03-04	18.0000	Sam
9	9	2012-03-04	12.0000	Joe
10	10	2012-03-04	11.0000	Beth
11	11	2012-03-05	14.0000	Sam
12	12	2012-03-06	17.0000	Beth
13	13	2012-03-06	19.0000	Joe
14	14	2012-03-07	13.0000	Beth
15	15	2012-03-07	16.0000	Sam



	OrderDate	CustomerNa...	CumDist	PctRa...
1	2012-03-02	Beth	0.2	0
2	2012-03-04	Beth	0.6	0.25
3	2012-03-04	Beth	0.6	0.25
4	2012-03-06	Beth	0.8	0.75
5	2012-03-07	Beth	1	1
6	2012-03-01	Joe	0.2	0
7	2012-03-02	Joe	0.4	0.25
8	2012-03-03	Joe	0.6	0.5
9	2012-03-04	Joe	0.8	0.75
10	2012-03-06	Joe	1	1
11	2012-03-01	Sam	0.2	0
12	2012-03-02	Sam	0.4	0.25
13	2012-03-04	Sam	0.6	0.5
14	2012-03-05	Sam	0.8	0.75
15	2012-03-07	Sam	1	1

Time- Intelligence OLAP Operations

In a modern data warehouse environment, the following operations are the "bread and butter" of time-series analysis:

Running Totals (Cumulative Sums):

Calculating the sum of values from the start of a period to the current row.

Moving Averages (Sliding Windows):

Smoothing data by averaging values over a specific number of preceding/following periods.

Period-over-Period Comparisons (Growth):

Using offsets to compare the current value against a previous day, month, or year.

Year-to-Date (YTD) & Quarter-to-Date (QTD):

Cumulative totals that reset at specific time boundaries.

First/Last Value Identification:

Identifying the first or most recent transaction value within a specific time partition.

Running Totals (Cumulative Sales)

Company-wide Cumulative Revenue – Tracks the growth of total revenue across the entire history of AdventureWorks.

```
SELECT
    OrderDate,
    TotalDue,
    SUM(TotalDue) OVER (ORDER BY OrderDate) AS RunningTotal
FROM Sales.SalesOrderHeader;
```

Running Totals (Cumulative Sales)

Cumulative Sales per Territory – This partitions the data, meaning the running total **resets** for each new Territory ID.

```
SELECT
```

```
    TerritoryID,
```

```
    OrderDate,
```

```
    TotalDue,
```

```
    SUM(TotalDue) OVER (PARTITION BY TerritoryID ORDER BY OrderDate)
```

```
    AS TerritoryRunningTotal
```

```
FROM Sales.SalesOrderHeader;
```

Moving Averages (Smoothing Trends)

30-Day Moving Average of Sales – Provides a smoothed average of daily sales using the 29 preceding days and the current day.

```
SELECT
    OrderDate,
    SUM(TotalDue) AS DailySales,
    AVG(SUM(TotalDue)) OVER (
        ORDER BY OrderDate
        ROWS BETWEEN 29 PRECEDING AND CURRENT ROW
    ) AS MovingAvg30Day
FROM Sales.SalesOrderHeader
GROUP BY OrderDate;
```

Moving Averages (Smoothing Trends)

3-Month Moving Average (Aggregated) – Calculates the trend based on monthly totals rather than individual orders.

```
WITH MonthlySales AS (  
    SELECT EOMONTH(OrderDate) AS MonthEnd, SUM(TotalDue) AS MonthlyTotal  
    FROM Sales.SalesOrderHeader  
    GROUP BY EOMONTH(OrderDate)  
)  
SELECT  
    MonthEnd, MonthlyTotal,  
    AVG(MonthlyTotal) OVER (  
        ORDER BY MonthEnd  
        ROWS BETWEEN 2 PRECEDING AND CURRENT ROW  
    ) AS MovingAvg3Month  
FROM MonthlySales;
```

Period-over-Period (Growth Analysis)

Month-over-Month (MoM) Sales Change – Compares this month's total revenue to the previous month:

WITH MonthlySales AS (

SELECT YEAR(OrderDate) as Yr, MONTH(OrderDate) as Mo, SUM(TotalDue)
AS Sales

FROM Sales.SalesOrderHeader

GROUP BY YEAR(OrderDate), MONTH(OrderDate)

)

SELECT

Yr, Mo, Sales,

LAG(Sales) OVER (ORDER BY Yr, Mo) AS PreviousMonthSales,

(Sales - LAG(Sales) OVER (ORDER BY Yr, Mo)) / LAG(Sales) OVER (ORDER BY
*Yr, Mo) * 100 AS GrowthPct*

FROM MonthlySales;

Period-over-Period (Growth Analysis)

Comparing to the Same Month Last Year (YoY) – Uses the offset parameter of LAG (12) to compare the current month to exactly one year ago:

```
WITH MonthlySales AS (  
    SELECT YEAR(OrderDate) as Yr, MONTH(OrderDate) as Mo, SUM(TotalDue)  
    AS Sales  
    FROM Sales.SalesOrderHeader  
    GROUP BY YEAR(OrderDate), MONTH(OrderDate)  
)  
SELECT  
    Yr, Mo, Sales,  
    LAG(Sales, 12) OVER (ORDER BY Yr, Mo) AS SalesLastYearSameMonth  
FROM MonthlySales;
```

YTD (Year-To-Date) Aggregations

Basic Sales YTD – Calculates total sales for the year, resetting at each new year.

```
SELECT
    OrderDate,
    TotalDue,
    SUM(TotalDue) OVER (
        PARTITION BY YEAR(OrderDate)
        ORDER BY OrderDate
    ) AS SalesYTD
FROM Sales.SalesOrderHeader;
```

YTD (Year-To-Date) Aggregations

Product-Level YTD Sales – Tracks how a specific product is performing within the current calendar year.

```
SELECT
```

```
    ProductID, OrderDate, LineTotal,
```

```
    SUM(LineTotal) OVER (
```

```
        PARTITION BY ProductID, YEAR(OrderDate)
```

```
        ORDER BY OrderDate
```

```
    ) AS ProductYTD
```

```
FROM Sales.SalesOrderDetail sod
```

```
    INNER JOIN Sales.SalesOrderHeader soh ON sod.SalesOrderID =  
    soh.SalesOrderID;
```

First and Last Value (Anchor Analysis)

Comparison to First Sale of the Year – Shows how the current sale compares to the "opening" sale of that specific year.

```
SELECT
    OrderDate, TotalDue,
    FIRST_VALUE(TotalDue) OVER (
        PARTITION BY YEAR(OrderDate)
        ORDER BY OrderDate
    ) AS FirstSaleOfYear
FROM Sales.SalesOrderHeader;
```

First and Last Value (Anchor Analysis)

Identifying Customer's Last Purchase Amount – Retrieves the value of the most recent purchase made by a customer prior to the current one.

```
SELECT
    CustomerID, OrderDate, TotalDue,
    LAST_VALUE(TotalDue) OVER (
        PARTITION BY CustomerID
        ORDER BY OrderDate
        ROWS BETWEEN UNBOUNDED PRECEDING AND 1 PRECEDING
    ) AS LastPurchaseAmount
FROM Sales.SalesOrderHeader;
```

Querying OLTP – OLAP Perspective

Sum up

Case

Pivot/UnPivot

Window

OVER clause

Ranking

*ROW_NUMBER, RANK / DENSE_RANK,
NTILE*

Positions

LAG, LEAD, FIRST_VALUE, LAST_VALUE

Percentiles

CUME_DIST, PERCENT_RANK

Common table expression

A photograph of an outdoor dining table set for a meal. The table is rectangular, made of light-colored wood, and is covered with a white tablecloth. It is set with white plates, glasses, and cutlery. There are several small potted plants and flowers on the table, including a large white hydrangea in the foreground. The table is surrounded by white chairs with wooden frames. The background is filled with dense green foliage and trees. The text "Common table expression" is overlaid in white, sans-serif font, centered on the image.

WITH

The WITH clause preceding a SELECT statement allows you to name the relations created by the subqueries contained within it.

The DBMS optimises the execution of such a query, for example by implementing the subquery as a materialised view or a temporary table.

Subqueries defined within the WITH clause can then be used in the query, making it more readable.

CTEs allows you to break down large, nesting-heavy queries into logical building blocks.

Readability: Replaces deeply nested subqueries with a sequential "top-down" approach.

Reusability: You can reference the same CTE multiple times within the main query.

Scope: It is temporary and disappears immediately after the query finishes executing.

WITH

The SELECT statement with a WITH clause takes the following form:

```
WITH cte_name [ ( column_name [,...n] ) ]  
AS  
(  
    -- CTE Query Definition  
    SELECT ...  
)  
-- Statement using the CTE  
SELECT * FROM cte_name;
```

WITH

Single-use Scope:

A CTE is only valid for the **one statement** immediately following its definition.

You cannot reference it three lines later in a separate query.

Column Names:

Every column in the CTE query must have a unique name.

If you use a function like SUM() or AVG(), you **must** provide an alias

e.g., SUM(Total) AS GrandTotal

WITH

```
WITH CustomerSales AS (  
    SELECT  
        CustomerID,  
        SUM(TotalDue) AS TotalSpent  
    FROM Sales.SalesOrderHeader  
    GROUP BY CustomerID  
)  
SELECT CustomerID, TotalSpent  
FROM CustomerSales  
WHERE TotalSpent > 5000  
ORDER BY TotalSpent DESC;
```

WITH

```
WITH ProductTotals AS (  
    SELECT ProductID, SUM(LineTotal) AS SalesAmount  
    FROM Sales.SalesOrderDetail  
    GROUP BY ProductID  
)  
OverallStats AS (  
    SELECT AVG(SalesAmount) AS AverageSales  
    FROM ProductTotals  
)  
SELECT  
    p.ProductID, p.SalesAmount, o.AverageSales  
FROM ProductTotals p, OverallStats o  
WHERE p.SalesAmount > o.AverageSales;
```

WITH – recursive CTEs

A Recursive CTE is a specialized version of a Common Table Expression that references itself.

It is the standard SQL solution for querying hierarchical data—structures where items have parent-child relationships, such as organizational charts, file systems, or Bill of Materials (BOM).

WITH

```
WITH CTE_Tree(employee_id, manager_id) AS (  
  -- Anchor member  
  SELECT employee_id, manager_id FROM employees WHERE  
  manager_id IS NULL  
  UNION ALL  
  -- Recursive member.  
  SELECT t.employee_id, t.manager_id FROM employees t  
  INNER JOIN CTE_Tree ct ON ct.employee_id = t.manager_id  
)  
SELECT * FROM CTE_Tree;
```

WITH

```
WITH CTE_Tree(employee_id,  
manager_id) AS (
```

```
-- Anchor member
```

```
    SELECT employee_id, manager_id  
    FROM employees WHERE  
    manager_id IS NULL
```

```
UNION ALL
```

```
-- Recursive member.
```

```
    SELECT t.employee_id, t.manager_id  
    FROM employees t
```

```
    INNER JOIN CTE_Tree ct ON  
    ct.employee_id = t.manager_id
```

```
)
```

```
SELECT * FROM CTE_Tree;
```

To prevent an infinite loop, a recursive CTE must follow a strict three-part structure:

The Anchor Member:

The "base" query that returns the starting point of the hierarchy (e.g., the CEO or the root folder).

The Recursive Member:

The query that references the CTE name.

It joins the CTE back to the base table to find the "next level" of data.

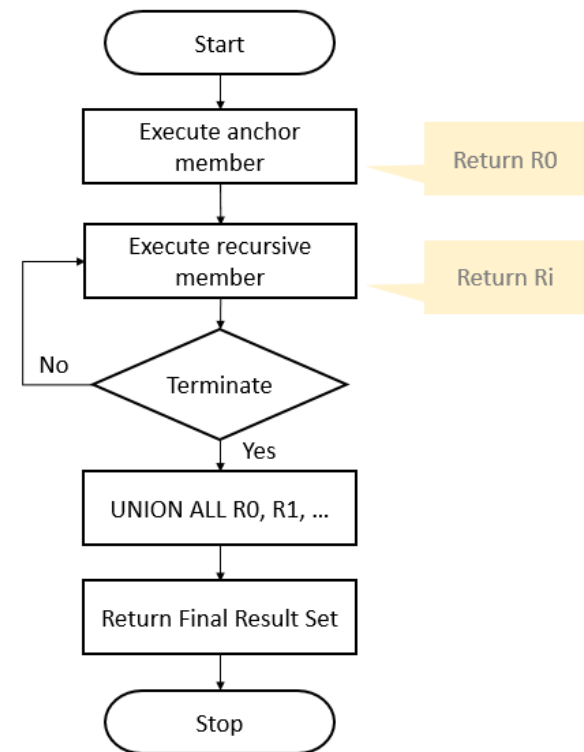
The Termination Check:

Usually implicit in the JOIN logic.

When the recursive member returns no more rows (e.g., an employee has no subordinates), the recursion stops.

WITH

```
WITH CTE_Tree(employee_id, manager_id) AS (  
  
  -- Anchor member  
  SELECT employee_id, manager_id FROM  
    employees WHERE manager_id IS NULL  
  
  UNION ALL  
  
  -- Recursive member.  
  SELECT t.employee_id, t.manager_id  
    FROM employees t  
    INNER JOIN CTE_Tree ct ON  
      ct.employee_id = t.manager_id  
)  
SELECT * FROM CTE_Tree;
```



WITH

SQL Server processes a recursive CTE in these logical steps:

Step 1: Execute the Anchor Member

create the initial result set - T_0 .

Step 2: Execute the Recursive Member

using T_0 as input to produce T_1 .

Step 3: Repeat the process

using T_n to produce $T_{\{n+1\}}$ until a step produces an empty set.

Step 4: Apply a UNION ALL

combine all result sets ($T_0, T_1 \dots T_n$).

WITH

```
WITH cte_numbers(n, weekday)
AS (
    -- anchor
    SELECT 0, DATENAME(DW, 0)
    UNION ALL
    -- recursive
    SELECT n + 1, DATENAME(DW, n + 1)
    FROM cte_numbers
    WHERE n < 6
)
SELECT weekday
FROM cte_numbers;
```

weekday
Monday
Tuesday
Wednesday
Thursday
Friday
Saturday
Sunday

WITH

Using paths:

```
WITH CTE_Tree(employee_id, manager_id, lvl,
path) AS (
    SELECT employee_id, manager_id, 1 as lvl,
    TO_CHAR(employee_id)
    FROM employees WHERE manager_id IS NULL
    UNION ALL
    SELECT t.employee_id, t.manager_id, lvl+1, ct.path || '-'
    || t.employee_id as path
    FROM employees t INNER JOIN CTE_Tree ct ON
    ct.employee_id = t.manager_id
)
SELECT * FROM CTE_Tree;
```

EMPLOYEE_ID	MANAGER_ID	LVL	PATH
1	(null)	1	100
2	101	2	100-101
3	102	2	100-102
4	114	2	100-114
5	120	2	100-120
6	121	2	100-121
7	122	2	100-122
8	123	2	100-123
9	124	2	100-124
10	145	2	100-145
11	146	2	100-146
12	147	2	100-147
13	148	2	100-148
14	149	2	100-149
15	201	2	100-201
16	108	3	100-101-108
17	200	3	100-101-200
18	203	3	100-101-203
19	204	3	100-101-204
20	205	3	100-101-205
21	103	3	100-102-103

WITH

CTE can lead to infinite recursion – it is advisable to apply a restriction:

```
WITH CTE_Tree(employee_id, manager_id, lvl, path) AS (  
    SELECT employee_id, manager_id, 1 as lvl, TO_CHAR(employee_id)  
        FROM employees WHERE manager_id IS NULL  
    UNION ALL  
    SELECT t.employee_id, t.manager_id, lvl+1, ct.path || '-' || t.employee_id  
        as path  
        FROM employees t INNER JOIN CTE_Tree ct ON ct.employee_id = t.manager_id  
        WHERE lvl < 10  
)  
SELECT * FROM CTE_Tree;
```

WITH

MAXRECURSION Hint:

By default, SQL Server limits recursion to 100 levels.

If your data is deeper (or you have a circular reference), use `OPTION (MAXRECURSION n)` at the end of the query.

Setting it to 0 allows infinite recursion.

Union Rules:

The Recursive Member must be connected to the Anchor by a `UNION ALL`.

A standard `UNION` (which performs a distinct check) is not allowed.

Aggregates:

You cannot use `GROUP BY`, `SUM()`, or `DISTINCT` within the Recursive Member itself.

WITH

SQL Server

```
WITH cte_name ( column_name [ ,...n ] )
```

```
AS (
```

```
    CTE_query_definition -- Anchor member is defined.
```

```
    UNION ALL
```

```
    CTE_query_definition -- Recursive member is defined referencing cte_name.
```

```
)
```

```
SELECT * FROM cte_name
```

```
    -- Statement using the CTE
```

```
[OPTION (MAXRECURSION X)]
```

```
    -- Limit recursion to X (query hint)
```

DBMS support

	BigQuery 2025-10-01	Db2 (LUW) 12.1.2	MariaDB 12.0.2	MySQL 9.5.0	Oracle DB 23.26.0	PostgreSQL 18	SQL Server 2022	SQLite 3.50.0
With clause	✓ ^a	✓	✓	✓	✓	✓	✓	✓
With recursive clause	✓ ^b	✓ ^c	✓	✓	✓ ^d	✓	✓ ^d	✓
Over (...)	✓	✓	✓	✓	✓	✓	✓	✓

With

Good practices – WITH Clause

- Do not nest – use CTEs

- Define CTEs in a logical order

- Name your CTEs and columns inside CTEs

- Do not overcomplicate recursion

- Document – state goals of each CTE

- Test your CTE



Merge Interval

Cost: 0 %



Sort
(Top N Sort)

Cost: 0 %



Compute Scalar

Cost: 0 %



Concatenation

Cost: 0 %



Clustered Index Seek (Clustered)

[SalesOrder].[PK_SalesOrder]

Cost: 100 %

Execution plans

Very brief introduction to execution plans in SQL Server using SQMS

SQL Execution plan

What is an SQL Execution Plan?

An execution plan (or query plan) is the sequence of steps that the database plans to take to execute a query.

When you write an SQL query, you specify what you want to be done, such as the columns to see and the tables to get data from – declarative programming

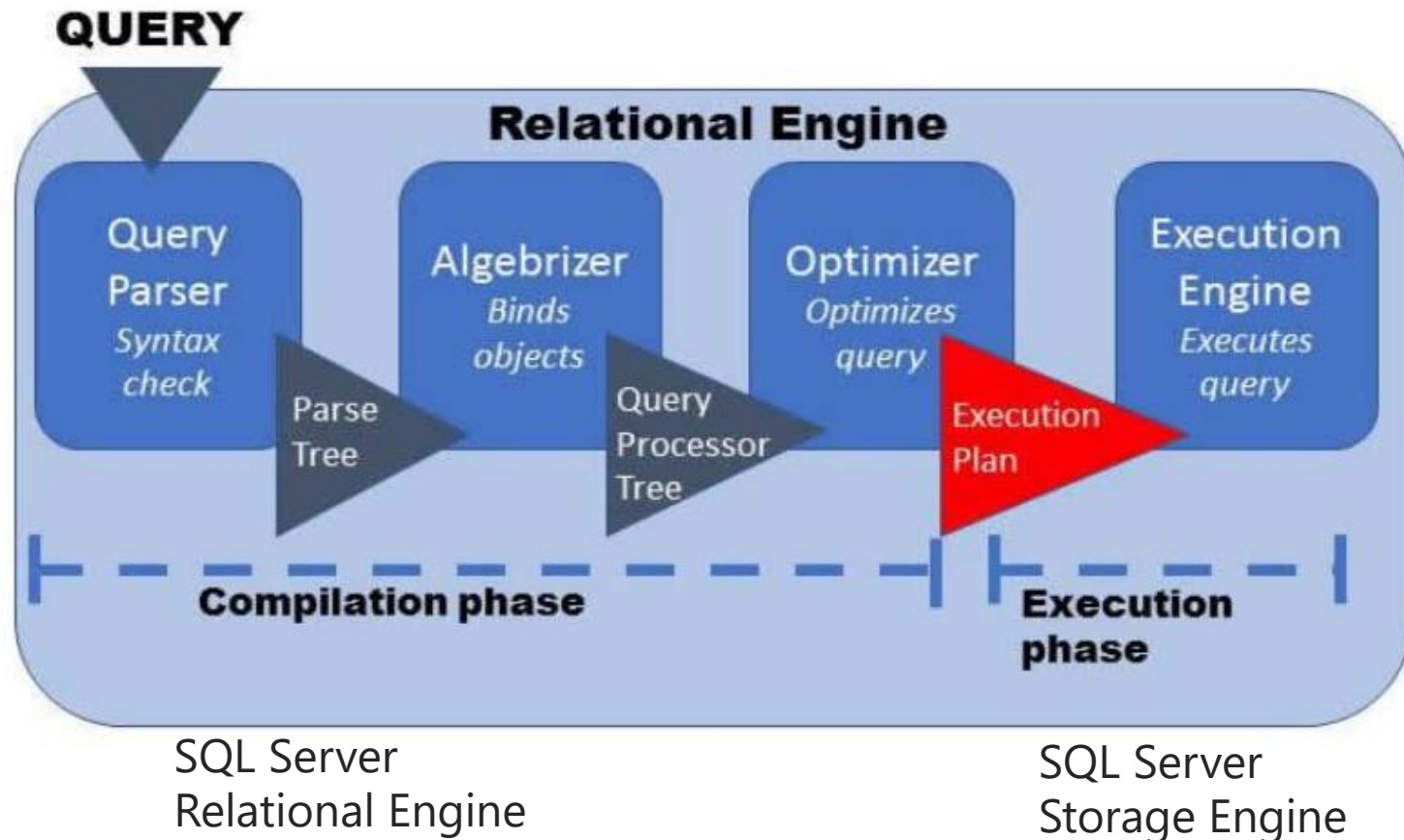
Reveal which tables and indexes a query accessed

in which order, how they were accessed, what types of joins were used, how much data was retrieved initially, and at what point filtering and sorting occurred.

Show how aggregations were performed

how calculated columns were derived, how and where foreign keys were accessed, and more.

What Happens When a Query is Submitted?



Compilation phase

Query parsing

make sure that query has no syntax errors

Result – parsing tree

Query binding

resolve the names of the different database objects referenced by the submitted query against the system catalog

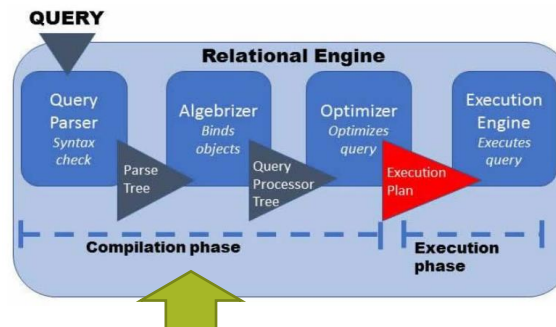
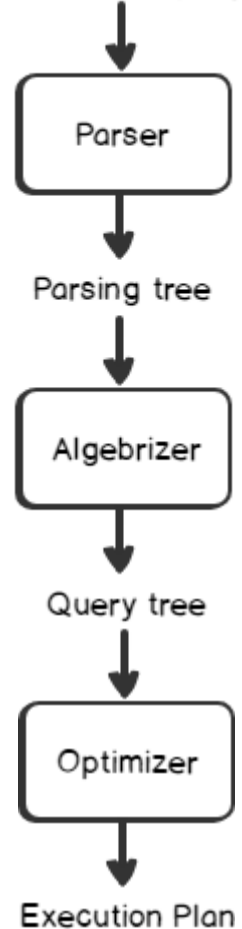
Result – query tree

Query optimization

creating the most efficient plan to execute the provided query

Result – execution plan

Submitted query



Cost-based optimization

The total cost of each candidate execution plan

calculated by assigning a weight for each operation, then summing the cost of all these operations together.

The full cost-based optimization process takes three inputs:

The Query processor tree

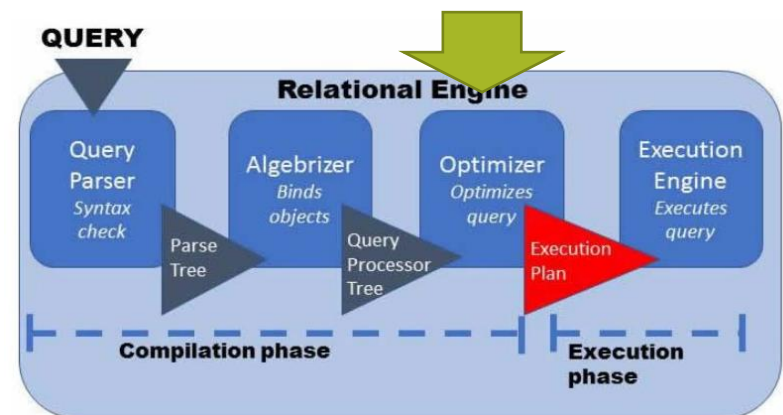
gives the optimizer knowledge of the logical query structure and of the underlying tables and indexes.

Statistics

index and column statistics give the optimizer an understanding of volume and distribution of data in the underlying data structures.

Constraints

the primary keys, enforced and trusted referential constraints, and any other types of constraints in place on the tables and columns that make up the query, tell the optimizer the limits on possible data stored within the tables referenced



Execution phase

After choosing the most efficient SQL Server Execution Plan

it will be stored in the Plan Cache memory storage

generating the execution plan is an expensive process

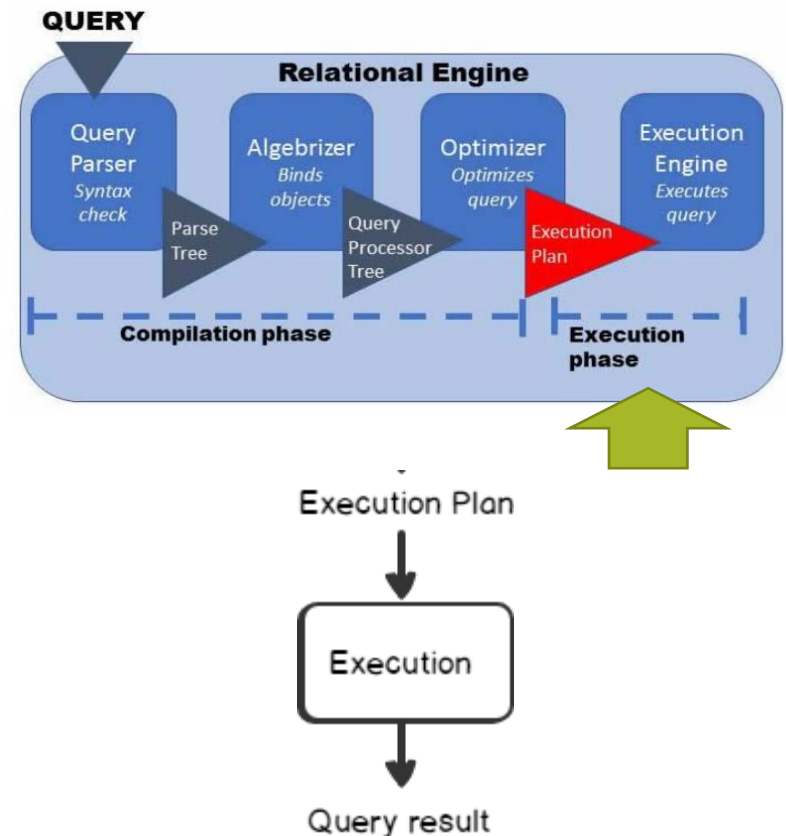
Query execution phase

The query execution engine executes the query per the instructions set out in the execution plan.

At runtime, the execution engine cannot change the optimizer's plan.

under certain circumstances force a plan to be recompiled

Interleaved execution



Execution phase

The importance of statistics

We don't want the optimizer to read all the data in all the tables referenced in a query each time it tries to generate a plan

the optimizer relies on statistics

The estimated cost of an execution plan depends largely on its cardinality estimations

The plan cache and plan reuse

Each plan has an associated „age“

Some related TSQL statements:

DBCC FREEPROCCACHE

will clear the cache for all databases on the server

DBCC FREEPROCCACHE(<planhandle>)

remove a specific plan from the plan cache

get handle from sys.dm_exec_query_stats

*ALTER DATABASE SCOPED
CONFIGURATION CLEAR
PROCEDURE_CACHE*

remove all plans for a single database

Execution phase

For each submitted query, the optimizer looks for a matching QueryHash value among the plans in the plan cache.

If found, it performs a detailed comparison of the SQL text of the submitted query and SQL text associated with the cached plan.

If they match exactly (including spaces and carriage returns) this returns the plan_handle, a value that uniquely identifies the plan in memory.

If we submit ad hoc queries to SQL Server and use hard-coded literal values then, for most of those queries, SQL Server will be forced to complete the full optimization process and compile a new plan each time.

```
SELECT p.ProductID ,  
       p.Name AS ProductName ,  
       pi.Shelf ,  
       l.Name AS LocationName  
FROM   Production.Product p  
       INNER JOIN Production.ProductInventory AS pi  
           ON pi.ProductID = p.ProductID  
       INNER JOIN Production.Location AS l  
           ON l.LocationID = pi.LocationID  
WHERE  l.Name = 'Paint';  
GO
```

Two plans in SSMS

estimated plans

is generated by parsing the submitted query as an estimate of how the query will be executed, without being executed

If there exists in the plan cache a plan that exactly matches the submitted query text, then the optimizer simply returns that cached plan.

If there is no match, the optimizer performs the optimization process and returns the new plan

actual plans

is generated by executing the submitted query, displaying the actual steps that followed while executing the query

If there is a cached plan that exactly matches the submitted query text, then the optimizer simply passes the cached plan to the execution engine, which executes it, and adds the requested runtime values to the displayed plan.

If there is no cached plan, the optimizer produces a new plan, places it in the cache and passes it on for execution and, again, we see the plan with runtime information.

Two plans in SSMS

estimated plans

Execution plan

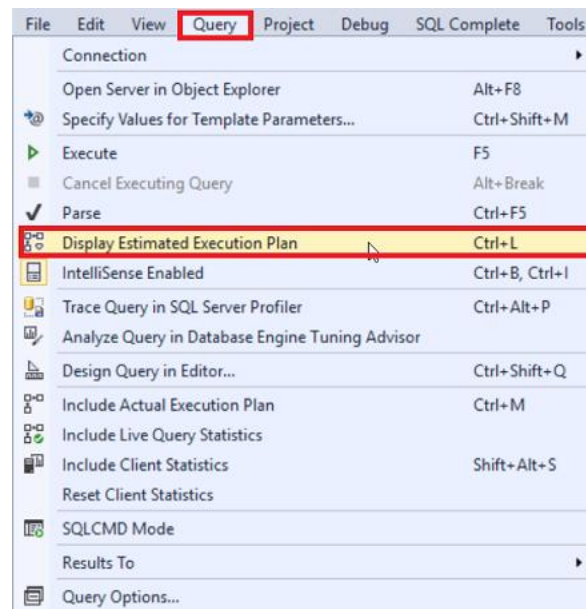
With estimates

actual plans

Execution plan

With run-time statistics

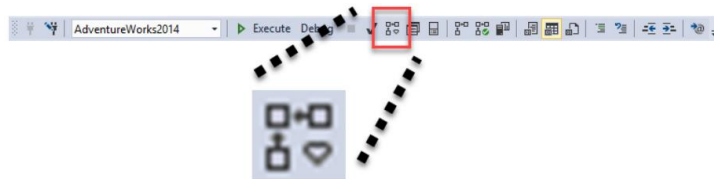
With estimates



Two plans in SSMS

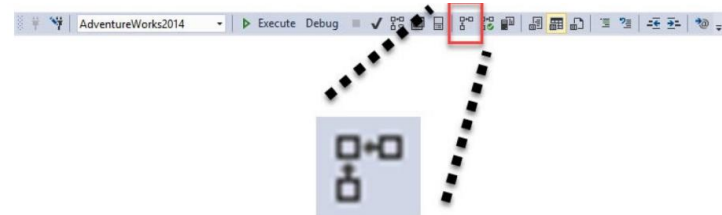
Capturing graphical plans in SSMS

Display Estimated Execution Plan
use the keyboard shortcut -
CTRL+L for estimated within SSMS



Capturing graphical plans in SSMS

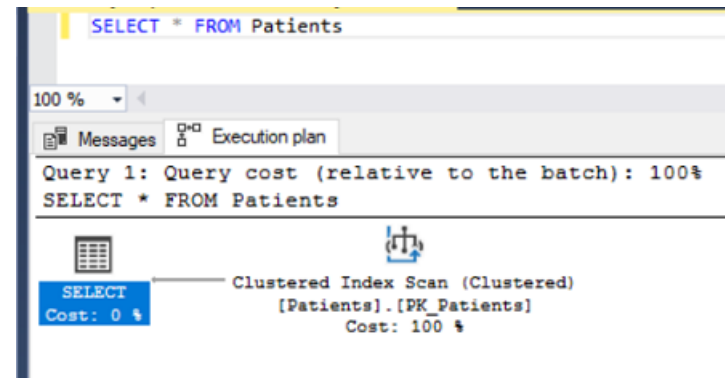
Display Include Actual Execution Plan
use the keyboard shortcut -
CTRL+M for actual within SSMS



Two plans in SSMS

Estimated Execution Plan

for the submitted query will be displayed without the need to execute it



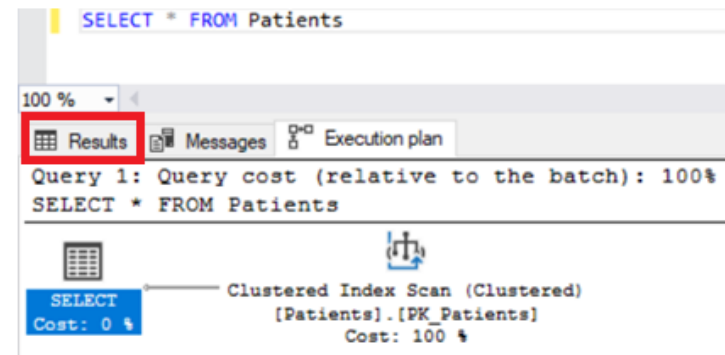
Actual Execution Plan

cannot be displayed directly

requires the submitted query to be executed in order to be displayed

option turned ON for a query

Actual Execution Plan will be displayed as Execution plan

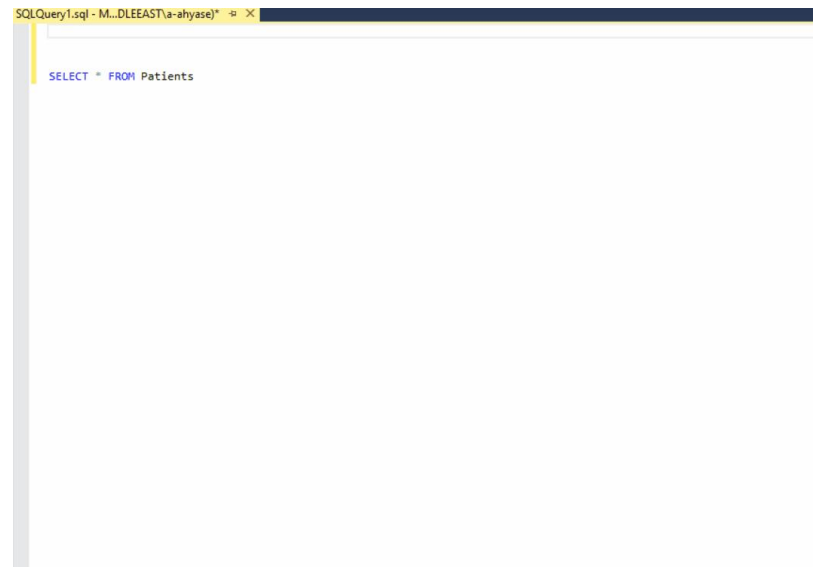
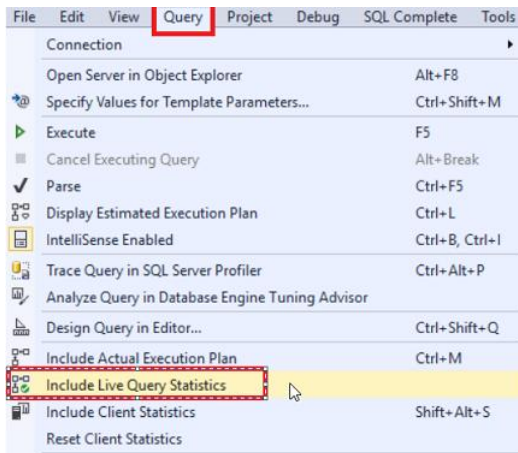


Live

Live Query Statistics

run the query and monitor the run-time execution statistics of the query and how the data flows between the operators

SQL Server 2016+



Other formats

[illegible]

The screenshot shows the SQL Server Enterprise Manager interface. In the left pane, the 'Messages' tab is selected, showing a message from Microsoft SQL Server 2005 XML Showplan. The message text is: 'ShowPlanXML xmlns="http://schemas.microsoft.com/sqlserver/2004/07/showplan" Version="1.5" Build="13.0.1601.5"'. In the right pane, the 'ShowplanXML' node is selected. A red arrow points from the 'ShowplanXML' node to the 'Show Execution Plan As...' menu option. Another red arrow points from the 'Show Execution Plan As...' menu option to the 'Show Execution Plan XML...' menu option.

SET SHOWPLAN_XML ON
GO
SELECT * FROM sys.databases
GO

Microsoft SQL Server 2005 XML Showplan
1 ShowPlanXML xmlns="http://schemas.microsoft.com/sqlserver/2004/07/showplan" Version="1.5" Build="13.0.1601.5"

Save Execution Plan As...
Show Execution Plan XML...
Compare Showplans
Analyze Actual Execution Plan
Find Node
Edit Query Text...
Zoom In
Zoom Out
Custom Zoom...
Zoom to Fit
Properties

ShowPlanXML xmlns="http://schemas.microsoft.com/sqlserver/2004/07/showplan" Version="1.5" Build="13.0.1601.5"
<batch>
<statements>
<statement StatementText="SELECT * FROM sys.databases" StatementId="1" StatementCompId="1" StatementType="SELECT" RetrievedFromCache="true" StatementSubTree=<StatementSetOptions QUOTED_IDENTIFIER="true" ARITHABORT="true" CONCAT_NULL_YIELDS_NULL="true" ANSI_NULLS="true" ANSI_PADDING="true" ANSI_WARNINGS="true" StatementSetOptions>
<queryPlan NodeId="0" PhysicalOp="Compute Scalar" LogicalOp="ParallelPlan" CachePlanSize="976" CompileTime="32" CompileCPU="32" CompileMemory="4688" Warnings=<Warnings>
<WarningConvertConversion="Cardinality Estimate" Expression="CONVERT_IMPLICIT(sql_variant,DBPROM,dtcol,<id>)">
</WarningConvert>
</Warnings>
<MemoryGrantInfo SerialRequiredMemory="0" SerialDesiredMemory="0">
<MemoryGrantInfo>
<OptimizersHardwareDependentProperties EstimatedAvailableMemoryGrant="836375" EstimatedPagesCached="418189" EstimatedAvailableDegreeOfParallelism="4">
</OptimizersHardwareDependentProperties>
<QueryNodeId="0" PhysicalOp="Compute Scalar" LogicalOp="Compute Scalar" EstimatedRows="1" EstimatedIO="20" EstimateCPU="1e-007" AvgRowSize="1176" EstimateOutputList=<ColumnReference Database="master" Schema="sys" Table="sysdatabases" Alias="(d)" Column="Id">
</ColumnReference>

Using SHOWPLAN options

Before you run your query, run one of the following statements.

The statement must be the only statement in the batch, i.e. you cannot execute another statement at the same time:

SHOWPLAN:

SET SHOWPLAN_TEXT ON

SET SHOWPLAN_ALL ON

SET SHOWPLAN_XML ON

STATISTICS

SET STATISTICS PROFILE ON

SET STATISTICS XML ON

These are connection options

you only need to run this once per connection

Once you are done you can turn this option off with the following statement:

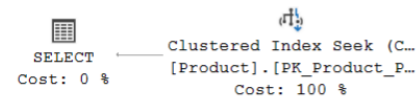
SET <<option>> OFF

Simple example

Sample query

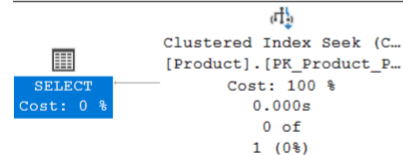
```
select Name
from Production.Product
where ProductID = 203
```

```
select Name from Production.Product where ProductID = 203
```



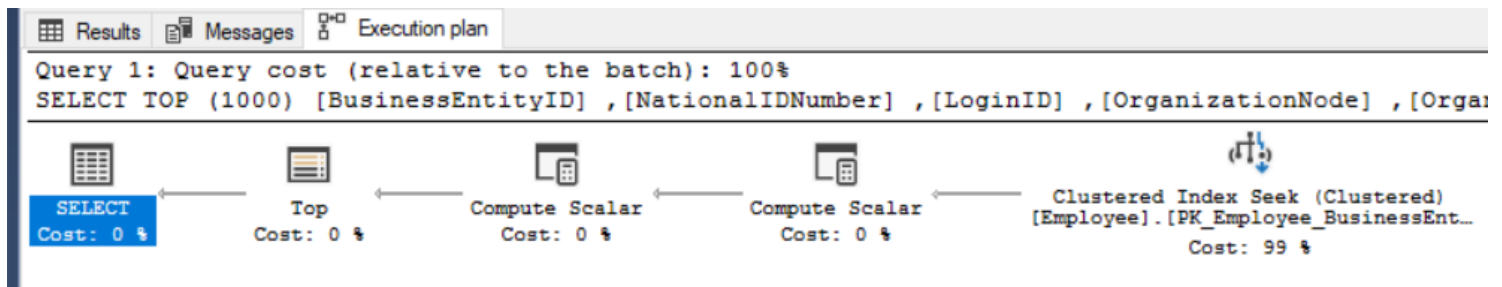
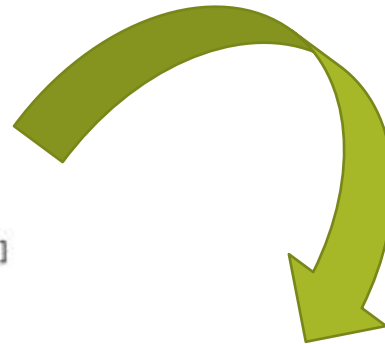
Query 1: Query cost (relative to the batch): 100%

```
SELECT [Name] FROM [Production].[Product] WHERE [ProductID]=@1
```

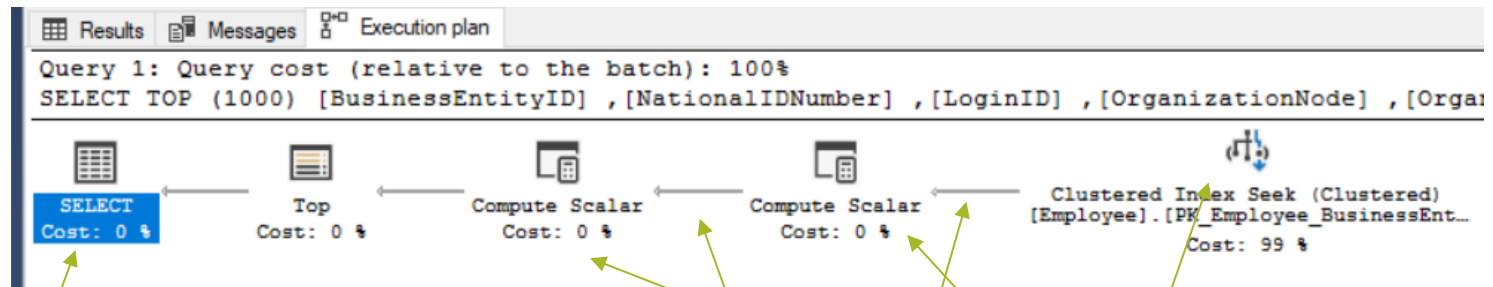


Simple example

```
SELECT TOP (1000) [BusinessEntityID]
, [NationalIDNumber]
, [LoginID]
, [OrganizationNode]
, [OrganizationLevel]
, [JobTitle]
, [BirthDate]
, [MaritalStatus]
, [Gender]
, [HireDate]
, [SalariedFlag]
, [VacationHours]
, [SickLeaveHours]
, [CurrentFlag]
, [rowguid]
, [ModifiedDate]
FROM [AdventureWorks2016CTP3].[HumanResources].[Employee]
WHERE BusinessEntityID > 208
```



Components



Return

HumanResources.Employee
Columns
BusinessEntityID (PK, FK, int, not null)
NationalIDNumber (nvarchar(15), not null)
LoginID (nvarchar(256), not null)
OrganizationNode (hierarchyid, null)
OrganizationLevel (Computed, smallint, null)
JobTitle (nvarchar(50), not null)
BirthDate (date, not null)
MaritalStatus (nchar(1), not null)
Gender (nchar(1), not null)
HireDate (date, not null)
SalariedFlag (Flag(bit), not null)
VacationHours (smallint, not null)
SickLeaveHours (smallint, not null)
CurrentFlag (Flag(bit), not null)
rowguid (uniqueidentifier, not null)
ModifiedDate (datetime, not null)

Operators

implements a specific algorithm designed to perform a specialized task.

Arrows

Thickness represents the amount of data

Estimated

Estimated operator costs

Best thought of as "cost units"

based on the mathematical calculations of anticipated CPU and I/O within the optimizer.

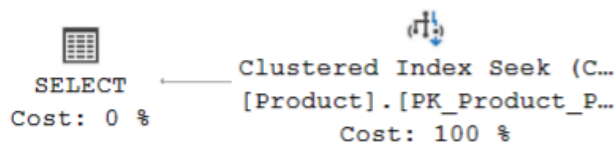
Are useful as measures

don't represent real-world measures of actual CPU and I/O.

There is generally a correlation between high estimated cost within the plan, and higher actual performance costs

still just estimated values.

```
select Name from Production.Produ
```



Clustered Index Seek (Clustered)	
Scanning a particular range of rows from a clustered index.	
Physical Operation	Clustered Index Seek
Logical Operation	Clustered Index Seek
Estimated Execution Mode	Row
Storage	RowStore
Estimated Operator Cost	0.0032831 (100%)
Estimated I/O Cost	0.003125
Estimated Subtree Cost	0.0032831
Estimated CPU Cost	0.0001581
Estimated Number of Executions	1
Estimated Number of Rows to be Read	1
Estimated Number of Rows for All Executions	1
Estimated Number of Rows Per Execution	1
Estimated Row Size	61 B
Ordered	True
Node ID	0
Object	
[AdventureWorks2014].[Production].[Product].[PK_Product_ProductID]	
Output List	
[AdventureWorks2014].[Production].[Product].Name	
Seek Predicates	
Seek Keys[1]: Prefix: [AdventureWorks2014].[Production].[Product].ProductID = Scalar Operator(CONVERT_IMPLICIT(int,[@1],0))	

Estimated

Estimated operator costs

Best thought of as "cost units"

*based on the mathematical
calculations of anticipated CPU and
I/O within the optimizer.*

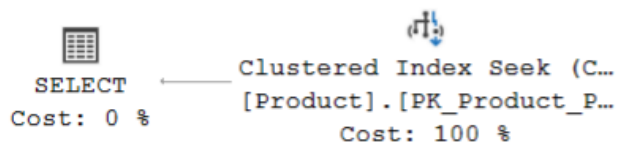
Are useful as measures

*don't represent real-world
measures of actual CPU and I/O.*

There is generally a correlation
between high estimated cost
within the plan, and higher actual
performance costs

still just estimated values.

```
select Name from Production.Produ
```



Clustered Index Seek (Clustered)	
Misc	
Defined Values	[AdventureWorks2014].[Production].[Product].*
Description	Scanning a particular range of rows from a clus
Estimated CPU Cost	0.0001581
Estimated Execution Mode	Row
Estimated I/O Cost	0.003125
Estimated Number of Executions	1
Estimated Number of Rows for All Executio	1
Estimated Number of Rows Per Execution	1
Estimated Number of Rows to be Read	1
Estimated Operator Cost	0.0032831 (100%)
Estimated Rebinds	0
Estimated Rewinds	0
Estimated Row Size	61 B
Estimated Subtree Cost	0.0032831
Forced Index	False
ForceScan	False
ForceSeek	False
Logical Operation	Clustered Index Seek
Node ID	0
NoExpandHint	False
Object	[AdventureWorks2014].[Production].[Product].*
Ordered	True
Output List	[AdventureWorks2014].[Production].[Product].*
Parallel	False
Physical Operation	Clustered Index Seek
Scan Direction	FORWARD
Seek Predicates	Seek Keys[1]: Prefix: [AdventureWorks2014].[Pr
Storage	RowStore
TableCardinality	504

Actual

Actual execution plan costs

```
SELECT [Name] FROM [Production].[Product] WHERE [ProductID]=@1
```

		Clustered Index Seek (C...
SELECT		[Product].[PK_Product_P...
Cost: 0 %	Cost: 100 %	
	0.000s	
	0 of	
	1 (0%)	

Clustered Index Seek (Clustered)	
Scanning a particular range of rows from a clustered index.	
Physical Operation	Clustered Index Seek
Logical Operation	Clustered Index Seek
Actual Execution Mode	Row
Estimated Execution Mode	Row
Storage	RowStore
Actual Number of Rows for All Executions	0
Actual Number of Batches	0
Estimated I/O Cost	0.003125
Estimated Operator Cost	0.0032831 (100%)
Estimated Subtree Cost	0.0032831
Estimated CPU Cost	0.0001581
Estimated Number of Executions	1
Number of Executions	1
Estimated Number of Rows for All Executions	1
Estimated Number of Rows Per Execution	1
Estimated Number of Rows to be Read	1
Estimated Row Size	61 B
Actual Rebinds	0
Actual Rewinds	0
Ordered	True
Node ID	0
Object	
[AdventureWorks2014].[Production].[Product].[PK_Product_ProductID]	
Output List	
[AdventureWorks2014].[Production].[Product].Name	
Seek Predicates	
Seek Keys[1]: Prefix: [AdventureWorks2014].[Production]. [Product].ProductID = Scalar Operator(CONVERT_IMPLICIT(int,[@1],0))	

Actual

Actual execution plan costs

Actual Time Statistics	
Actual Elapsed CPU Time (ms)	0
All threads	0
Actual Elapsed Time (ms)	0
All threads	0

```
SELECT [Name] FROM [Production].[Product] WHERE [ProductID]=@1
```

		Clustered Index Seek (C...
		[Product].[PK_Product_P...
SELECT	Cost: 100 %	
Cost: 0 %	0.000s	
	0 of	
	1 (0%)	

Clustered Index Seek (Clustered)		
Misc	Actual	Property Pages
		Row
Actual I/O Statistics		
Actual Number of Batches	0	
Actual Number of Rows for All Executions	0	
Actual Rebinds	0	
Actual Rewinds	0	
Actual Time Statistics		
Defined Values		[AdventureWorks2014].[Production].[Product].N
Description		Scanning a particular range of rows from a clus
Estimated CPU Cost	0.0001581	
Estimated Execution Mode	Row	
Estimated I/O Cost	0.003125	
Estimated Number of Executions	1	
Estimated Number of Rows for All Executio	1	
Estimated Number of Rows Per Execution	1	
Estimated Number of Rows to be Read	1	
Estimated Operator Cost	0.0032831 (100%)	
Estimated Rebinds	0	
Estimated Rewinds	0	
Estimated Row Size	61 B	
Estimated Subtree Cost	0.0032831	
Forced Index	False	
ForceScan	False	
ForceSeek	False	
Logical Operation	Clustered Index Seek	
Node ID	0	
NoExpandHint	False	
Number of Executions	1	
Object		[AdventureWorks2014].[Production].[Product].I
Ordered	True	
Output List		[AdventureWorks2014].[Production].[Product].N
Parallel	False	
Physical Operation	Clustered Index Seek	
Scan Direction	FORWARD	
Seek Predicates		Seek Keys[1]: Prefix: [AdventureWorks2014].[Pr
Storage	RowStore	
TableCardinality	504	

Properties window

Operator details

Properties window for other operators reveals information specific to the action of that operator

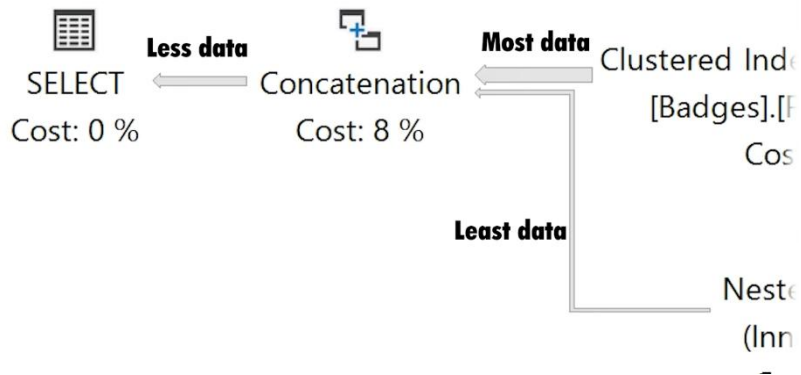
the first operator offers a lot of information about the plan itself and its generation

includes information such as the time, CPU and memory

The screenshot displays the SQL Server Enterprise Manager interface. The main window shows a query plan for 'Query 1: Query cost (relative to the batch) - SELECT [d].[Department ID], [d].[Name], [d].[G...]' with a cost of 0. The plan consists of a single operator: 'Clustered Index Scan (Clustered) [Department].[PK_Department_Departm...]' with a cost of 100. The 'Properties' window is open on the right, showing details for the selected 'SELECT' operator.

SELECT	
Misc	
Cached plan size	24 KB
CardinalityEstimationModelVersion	140
CompileCPU	0
CompileMemory	128
CompileTime	0
DatabaseContextSettingsId	1
Degree of Parallelism	1
Estimated Number of Rows	2
Estimated Operator Cost	0 (0%)
Estimated Subtree Cost	0.0032996
MemoryGrantInfo	
Optimization Level	TRIVIAL
OptimizerHardwareDependentProperties	
OptimizerStatsUsage	
Parameter List	
ParentObjectId	0
QueryHash	0xD72909107A423269
QueryPlanHash	0x79F269FD306EAFB7
QueryTimeStats	
RetrievedFromCache	true
SecurityPolicyApplied	False
Set Options	
ANSI_NULLS	True, ANSI
Statement	SELECT [d].[Department
StatementParameterizationType	2
StatementSqlHandle	0x0900D07771465026ED1

Flow



Arrow size represents

the estimated number of rows
*output from the source operator in
estimated execution plans,*

the number of rows
*read by the source operator in
actual execution plans.*

What is Relational Algebra?

Relational algebra

An algebra whose operands are relations or variables that represent relations.

Operators are designed to do the most common things that we need to do with relations in a database.

Five primitive operators of Codd's algebra are

the selection, the projection, the Cartesian product (also called the cross product or cross join), the set union, and the set difference.

We also have

intersection, other products and joins, renaming, aggregates, etc.



Reminder

Operands

variables or values from which new values can be constructed.

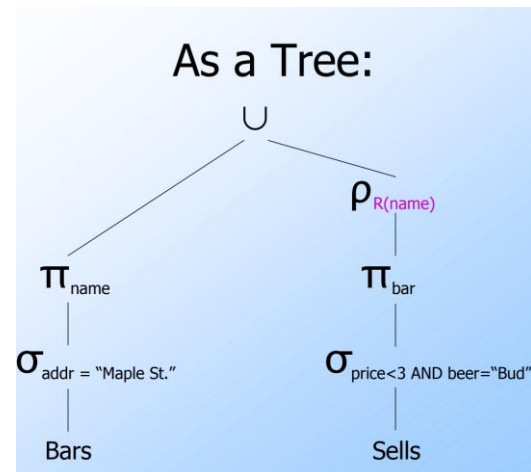
Operators

symbols denoting procedures that construct new values from given values

Algebra

Operands + operators

Set and multiset



Terms in an SQL Server Execution Plan

There are a few terms used in the SQL Server execution plans you should be aware of:

Index Scan and Clustered Index Scan

This will read the entire index in order.

Index Seek and Clustered Index Seek

This will perform a B-tree traversal and find matching rows.

Key Lookup (Clustered)

This will find a single row from a clustered index.

RID Lookup

This will find a single row from a table (no clustered index)

RID is a physical address of the row within the data file – FileID, PageID, SlotID.

Table Scan

This will read all rows and columns in the table. Also called a Full Table Scan. It's an expensive operation.

Hash Match

Joins two tables by creating a hash table.

Nested Loops

Joins two tables by getting the result from one table and matching it to the other table.

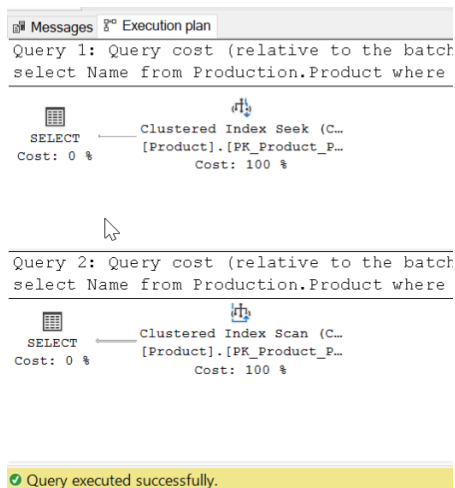
Merge Join

Joins two tables that have been sorted by matching rows together.

Sort

Sorts the result of your query using the ORDER BY clause.

Operators



A graphical execution plan displays three distinct types of operator:

Physical Operators (and their associated logical operations)

the only type of operator you'll see in an actual execution plan.

Cursor Operators

represent T-SQL cursor operations

Language Elements

represent T-SQL language elements

such as ASSIGN, DECLARE, IF, WHILE, and so on

Operators

Logical Operators

describe the relational algebraic operation used to process a statement

describe conceptually what operation needs to be performed.

E.g. JOIN

Physical Operators

implement the operation described by logical operators

each physical operator is an object or routine that performs an operation.

E.g. MERGE JOIN, HASH MATCH JOIN, ADAPTIVE JOIN

The physical operators initialize, collect data, and close.

They all share a common interface that is used when they are called by other operators:

Init():

operator initialize

second and later calls to Init() during execution of the same plan request the operator to prepare to re-start.

GetNext():

return a single row (or batch) of data.

The operators uses internal data structures to keep track of state, so they can continue where they left off when receiving the next GetNext() call.

and the number of times it is called appears as ActualRows

Close():

Clean up any memory it may have used and shut itself down.

An operator never receives more than one Close() call during execution of a single plan.

Example – merge join

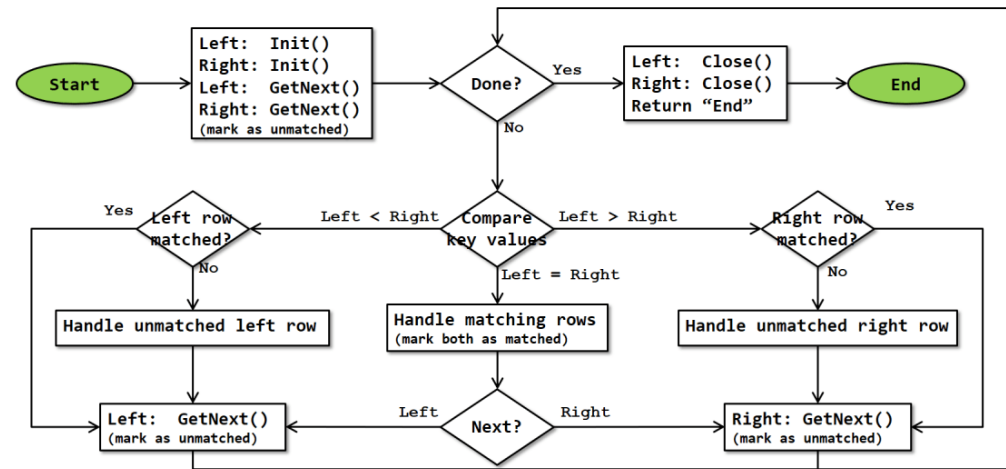
The Merge Join operator is one of four operators that join data from two input streams into a single combined output stream.

has two inputs

called the left and right input.

In a graphical execution plan, the left input is displayed on the top.

Merge Join requires all input data to be sorted by the join columns.



Cursor

```
DECLARE TerritoryName CURSOR FOR

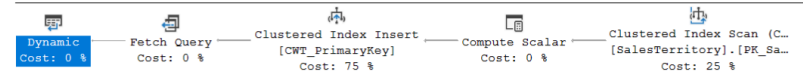
SELECT SalesTerritory.Name FROM [Sales].[SalesTerritory]
WHERE SalesTerritory.[Group] LIKE '%America%'

OPEN TerritoryName
FETCH NEXT FROM TerritoryName

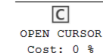
WHILE @@FETCH_STATUS = 0
BEGIN
    FETCH NEXT FROM TerritoryName
END

CLOSE TerritoryName
DEALLOCATE TerritoryName
```

Query 1: Query cost (relative to the batch): 100%
DECLARE TerritoryName CURSOR FOR SELECT SalesTerritory.Name FROM [Sales].[SalesTerritory]



Query 2: Query cost (relative to the batch): 0%
OPEN TerritoryName



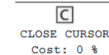
Query 3: Query cost (relative to the batch): 0%
FETCH NEXT FROM TerritoryName



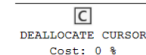
Query 4: Query cost (relative to the batch): 0%
WHILE @@FETCH_STATUS = 0



Query 5: Query cost (relative to the batch): 0%
END CLOSE TerritoryName



Query 6: Query cost (relative to the batch): 0%
DEALLOCATE TerritoryName



Language

```
▣ DECLARE @tmp_key int  
  SET @tmp_key = 0
```

```
Query 1: Query cost (relative to the batch): 0%  
DECLARE @tmp_key int SET @tmp_key = 0
```

```
T-SQL  
  ASSIGN  
Cost: 0 %
```

Reading plans

We generally read execution plans from right to left, following the data flow arrows

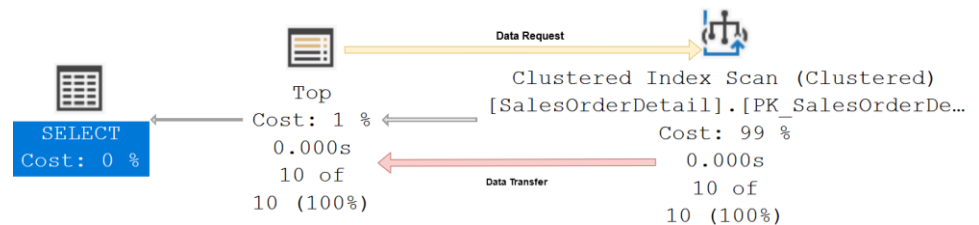
but that it is equally valid, and frequently helpful, to read from left to right

Internally, SQL Server executes the plan from left to right.

Each operator requests data from the next operator.

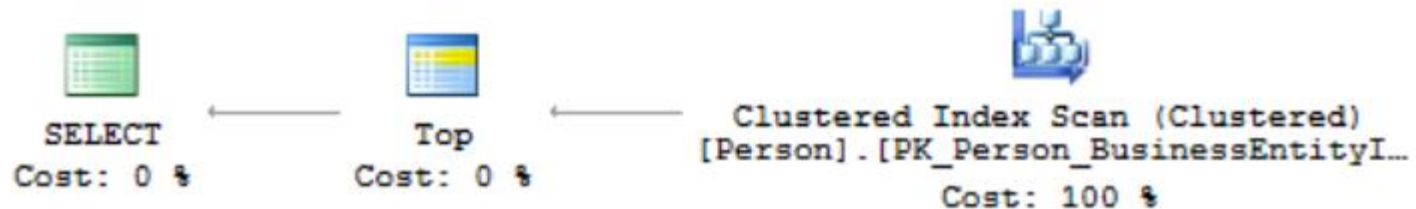
If you are interested in operator's physical orders, you should read an execution plan from left to right

```
SELECT TOP (10) [SalesOrderID]
[SalesOrderDetailID]
[CarrierTrackingNumber]
[OrderQty]
[ProductID]
[SpecialOfferID]
[ModifiedDate]
FROM [AdventureWorks2019].[Sales].[SalesOrderDetail] where [UnitPrice]>1000
```



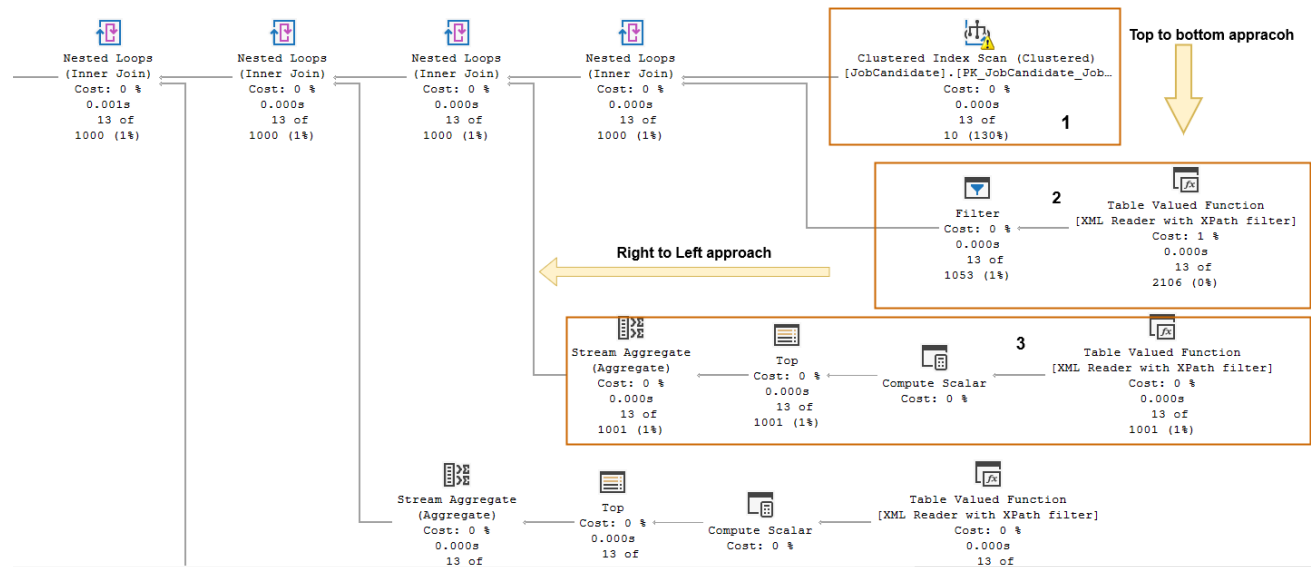
Reading plans

```
SELECT TOP(1) Title, FirstName, LastName  
FROM Person.Person;
```



Reading plans

If multiple branches merge in an execution plan, you should follow the right to left, top to bottom approach.



Tooltips and additional properties

Index Seek (NonClustered)

[Badges].[IX].[PK]

Scan a particular range of rows from a nonclustered index.

Physical Operation	Index Seek
Logical Operation	Index Seek
Actual Execution Mode	Row
Estimated Execution Mode	Row
Storage	RowStore
Number of Rows Read	661
Actual Number of Rows	661
Actual Number of Batches	
Estimated I/O Cost	0.037939
Estimated Operator Cost	0.0477302 (0%)
Estimated CPU Cost	0.009290
Estimated Subtree Cost	0.047230
Estimated Number of Executions	
Number of Executions	
Estimated Number of Rows	8303.0
Estimated Number of Rows to be Read	8303.0
Estimated Row Size	63
Actual Rebinds	
Actual Rewinds	
Ordered	True
Node ID	

Object

```
[StackOverflow2013].[dbo].[Badges].[IX_Badges] [b]
```

Output List

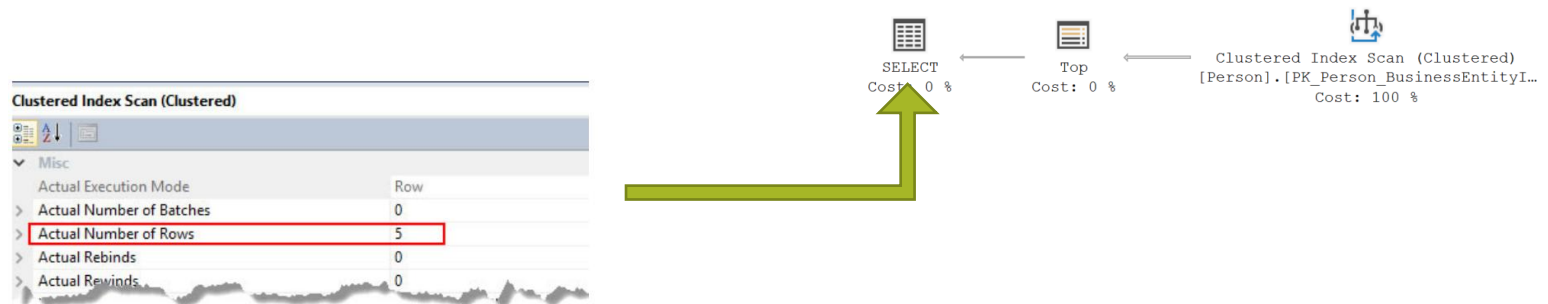
```
[StackOverflow2013].[dbo].[Badges].Name,  
[StackOverflow2013].[dbo].[Badges].UserId,  
[StackOverflow2013].[dbo].[Badges].Date
```

Seek Predicates

```
Seek Keys[1]: Start: [StackOverflow2013].[dbo].[Badges].Date  
>= Scalar Operator('2010-01-01 00:00:00.000'), End:  
[StackOverflow2013].[dbo].[Badges].Date < Scalar Operator  
( '2010-01-07 00:00:00.000')
```

```
SELECT TOP 5 )
    BusinessEntityID ,
    PersonType ,
    NameStyle ,
    Title ,
    FirstName ,
    LastName ,
    ModifiedDate
FROM Person
WHERE ModifiedDate >= '20130601'
AND ModifiedDate <= CURRENT_TIMESTAMP ;
```

Listing 2-1



Operators

Type of plan depends a lot on the nature of the workload.

For an OLTP workload you will hope to see a lot of Index Seek and Nested Loops operators, characteristic of frequent queries that return relatively small amounts of data.

Type of plan depends a lot on the nature of the workload.

For a BI system, you are likely to see more Index Scans, since these are often more efficient when reading a large proportion of data in a table, and Merge Join or Hash Match joins, which are join algorithms that become more efficient when joining larger data streams.

Operators

Reading data	Combining data	Grouping and ordering data
Table/Index Scan	Nested Loops	Sort
Index Seek	Merge Join	Stream Aggregate
Lookup	Hash Match	Hash Match (Aggregate)
Constant Scan	Adaptive Join	Window Aggregate
	Sequence	Segment
	Concatenation	Window Spool
	Switch	

List of all operators

<https://docs.microsoft.com/en-us/sql/relational-databases/showplan-logical-and-physical-operators-reference>

general information only

<https://sqlserverfast.com/epr/operator-list/>

with block diagrams of implementation

Manipulating data	Modifying data	Performance
Compute Scalar	Table/Index Insert	Bitmap
Filter	Table/Index Update	Spools
Top	Table/Index Delete	Parallelism
Sequence Project	Table/Index Merge	
	Assert	
	Split	
	Collapse	

Scan / Seek

(Clustered) Index Scan operator

retrieves all rows from the (non)clustered index specified in the Argument column.

If an optional WHERE:() predicate appears in the Argument column, only those rows that satisfy the predicate are returned.

Index Scan is a logical and physical operator.

(Clustered) Index Seek operator

uses the seeking ability of indexes to retrieve rows from a (non)clustered index.

The Argument column contains the name of the nonclustered index being used.

It also contains the SEEK:() predicate.

only those rows that satisfy the SEEK:() predicate.

It optionally may include a WHERE:() predicate,

evaluate against all rows that satisfy the SEEK:() predicate (it does not use the indexes to do this).

If the Argument column contains the ORDERED clause,

the rows must be returned in the order in which the nonclustered index has sorted them

If the ORDERED clause is not present

storage engine searches the index in the optimal way

Index Seek is a logical and physical operator.

Lookup

Key Lookup operator

occurs when the query optimizer chooses to use a non-clustered index to locate rows in a table that has a clustered index

The argument column contains the name of the clustered index and the clustering key used to look up the row in the clustered index.

Key Lookup is always accompanied by a Nested Loops operator.

The use of a Key Lookup operator in a query plan indicates that the query might benefit from performance tuning.

For example, query performance might be improved by adding a covering index.

Join data operators

There are four operators that join data from two input streams into a single combined output stream.

Nested loops

is most effective join operator when the outer input has a low estimated cardinality, and the inner input has a low estimated cost per execution

Merge Join

ideal for joining data streams that are sorted by the join key

Hash Match

most effective for joining large unsorted sets

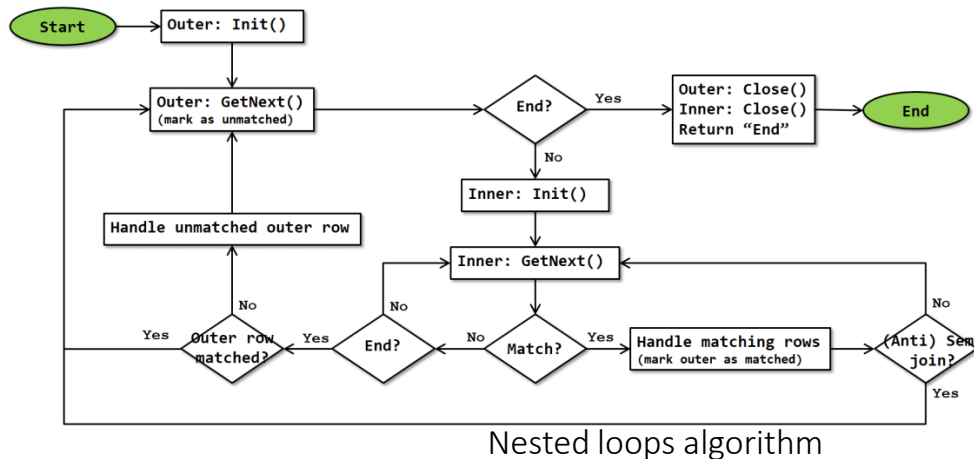
Adaptive Join

which can be used when the optimizer finds viable plans for both Nested Loops and Hash Match and wants to postpone the final choice until run time.

In the plan You will see:

ActualRebinds (reevaluation due to change)

ActualRewinds (reuse due to no change)



Sort

Sort operator

Is a blocking operator
sorts all incoming rows.

The Argument column contains either
*a DISTINCT ORDER BY:() predicate if
duplicates are removed by this operation,
an ORDER BY:() predicate with a comma-
separated list of the columns being sorted.*

The columns are prefixed with the value

ASC if the columns are sorted in
ascending order

DESC if the columns are sorted in
descending order

Sort is a logical and physical
operator.

Top N Sort

retrieves all available data, sorts the
data, and returns only top N rows

the number of rows to be returned are
limited

Distinct Sort

Retreives all available data, sorts the
data, while returning skips adjacent rows
with same values

Operators

Streaming versus blocking operators

A streaming operator creates output data at the same time as it receives the input.

In other words, it will pass on rows to the next operator as soon as it has performed its task on that row

Blocking operators must gather the entire set of input data and then perform their work on the entire data set, before passing on any rows.

Some operators are only semi-blocking

must complete only part of their work before releasing the first row.

For example, the join operator Hash Match first processes all rows from its first input, but then processes and returns rows from the second input as it reads them

Simple example

The Clustered Index Scan is a streaming operator

- passes on rows as they are read from the index
- a scan indicates that it will read all rows in the table, or index, until all rows are processed

- When it finds a row that falls in the required date range, it passes that row on to the next operator – a Sort.

The Sort operator reorders data, representing here the ORDER BY statement in the query

- The Sort operator is a blocking operator.

- This logical operation is a Top N Sort because of the TOP operation in the query.

- It must read every row from its child operator, in this case over 13K qualifying rows, sort them according to the specified criteria, ModifiedDate, and then pass on the top five rows

```
SELECT TOP ( 5 )
    BusinessEntityID ,
    PersonType ,
    NameStyle ,
    Title ,
    FirstName ,
    LastName ,
    ModifiedDate
FROM
    Person.Person
WHERE
    ModifiedDate >= '20130601'
    AND ModifiedDate <= CURRENT_TIMESTAMP ;
```

Listing 2-1

Figure 2-1 shows the resulting execution plan.



After adding:
Order by ModifiedDate

Aggregate

Stream Aggregate operator

- groups rows by one or more columns and then calculates one or more aggregate expressions returned by the query.

- requires input ordered by the columns within its groups

 - optimizer will use a Sort operator prior to this operator if the data is not already sorted due to a prior Sort operator or due to an ordered index seek or scan.*

- columns in the GROUP BY predicate are listed in the Argument column,

- aggregate expressions are listed in the Defined Values column

Stream Aggregate is a physical operator.

Stream Aggregate is the most effective of the two aggregation operators

- extra sorts increases the total plan cost.

- In such cases, the optimizer tends to choose the alternative aggregation operator, Hash Match

 - with an aggregate creates a temporary hash table in memory in which it stores the results of all aggregate computations*

 - can count rows, track minimum and maximum values, calculate a sum, and so on

Concatenation / Compute Scalar

The most common type of data concatenation is through the UNION ALL

Concatenation operator

- always has two or more inputs

- processes each of the inputs in order, from top to bottom, and concatenates them

- is non-blocking

Compute Scalar operator

- is used to compute new values based on other columns in the same row

- These new values are then added as extra column(s) in the output rows.*

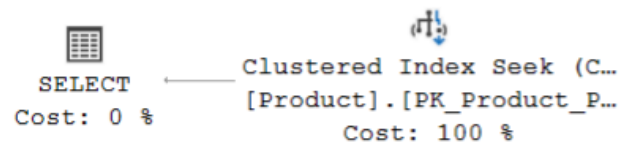
- is non-blocking

Simple example

Sample query

```
SELECT Name  
FROM Production.Product  
WHERE ProductID = 203
```

Query 1: Query cost (relative to the batch): 21%
select Name from Production.Product where ProductID = 203

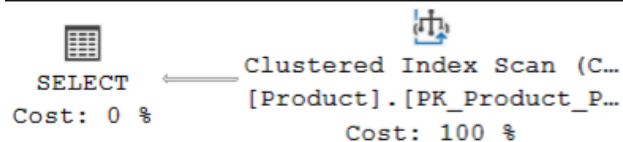


Simple example

Sample query

```
SELECT Name  
FROM Production.Product  
WHERE Color = 'Red'
```

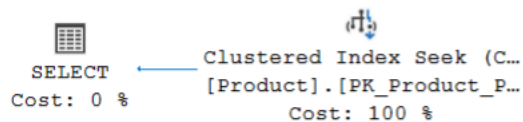
Query 2: Query cost (relative to the batch): 79%
select Name from Production.Product where Color = 'Red'



Simple example

Estimated Number of Executions	1
Estimated Number of Rows for All Executio	1
Estimated Number of Rows Per Execution	1
Estimated Number of Rows to be Read	1

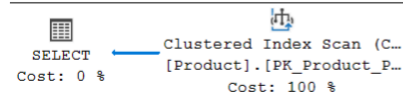
Query 1: Query cost (relative to the batch): 21%
 select Name from Production.Product where ProductID = 203



Estimated Data Size	61 B
Estimated Number of Rows for All Executio	1
Estimated Number of Rows Per Execution	1
Estimated Row Size	61 B

Estimated Number of Executions	1
Estimated Number of Rows for All Executions }	
Estimated Number of Rows Per Execution	38
Estimated Number of Rows to be Read	504

Query 2: Query cost (relative to the batch): 79%
 select Name from Production.Product where Color = 'Red'



Estimated Data Size	2584 B
Estimated Number of Rows for All Executio	38
Estimated Number of Rows Per Execution	38
Estimated Row Size	68 B

Cost

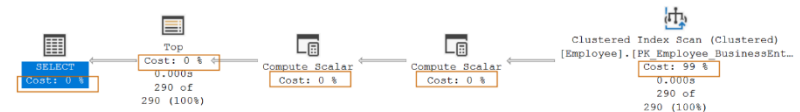
Each operator in a SQL Server execution plan is associated with a cost.

The operator cost is relative to other costs in the execution plan.

Usually, we need to concentrate on the costly operator and tuning the query around it.

In the event of a complicated execution plan, it might be challenging to identify the costly operator from graphical representation

In this case, you can use `SET SHOWPLAN_ALL ON`, and it will provide information in a tabular format.

[illegible]

Cost

Running multiple queries in a batch shows relative cost (%) to the batch

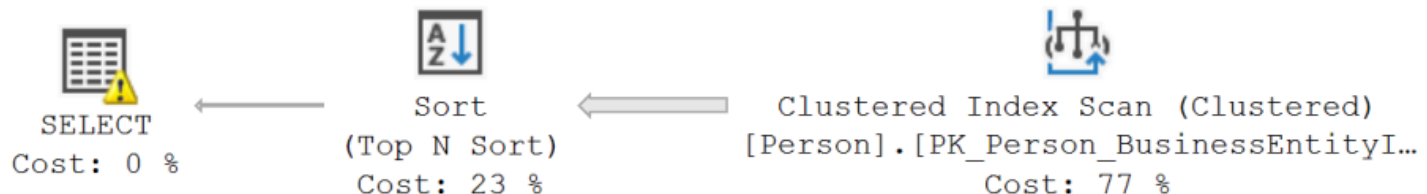
```
select Name from Production.Product where ProductID < 203  
select Name from Production.Product where ProductID < 500
```

Query 1: Query cost (relative to the batch): 35%	
SELECT [Name] FROM [Production].[Product] WHERE [ProductID]<@1	
	Clustered Index Seek (C...
	[Product].[PK_Product_P...
SELECT	Cost: 100 %
Cost: 0 %	0.000s
	4 of
	4 (100%)
Query 2: Query cost (relative to the batch): 65%	
SELECT [Name] FROM [Production].[Product] WHERE [ProductID]<@1	
	Index Scan (NonClustere...
	[Product].[AK_Product_N...
SELECT	Cost: 100 %
Cost: 0 %	0.000s
	176 of
	176 (100%)

Warnings

Within an execution plan, you may see small icons appear on an operator, specifically a yellow or red exclamation mark.

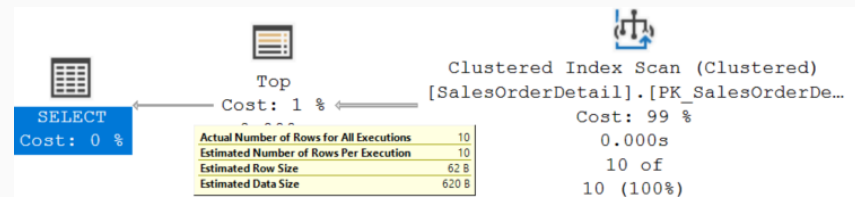
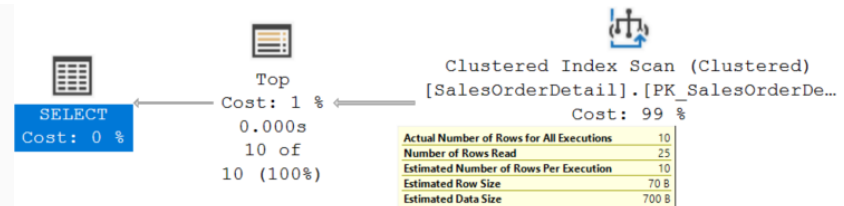
These are warnings.



Simple example

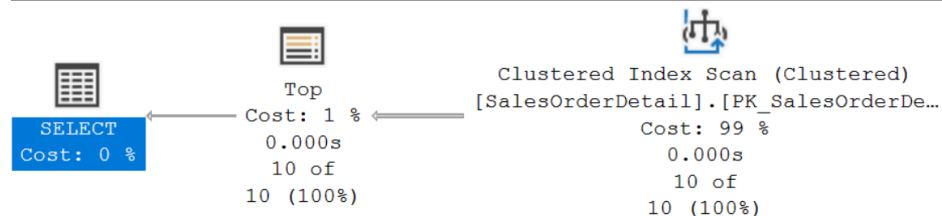
```
SELECT TOP (10) [SalesOrderID]
[SalesOrderDetailID]
[CarrierTrackingNumber]
[OrderQty]
[ProductID]
[SpecialOfferID]
[ModifiedDate]
```

```
FROM [AdventureWorks2019].[Sales].[SalesOrderDetail] where [UnitPrice]>1000
```

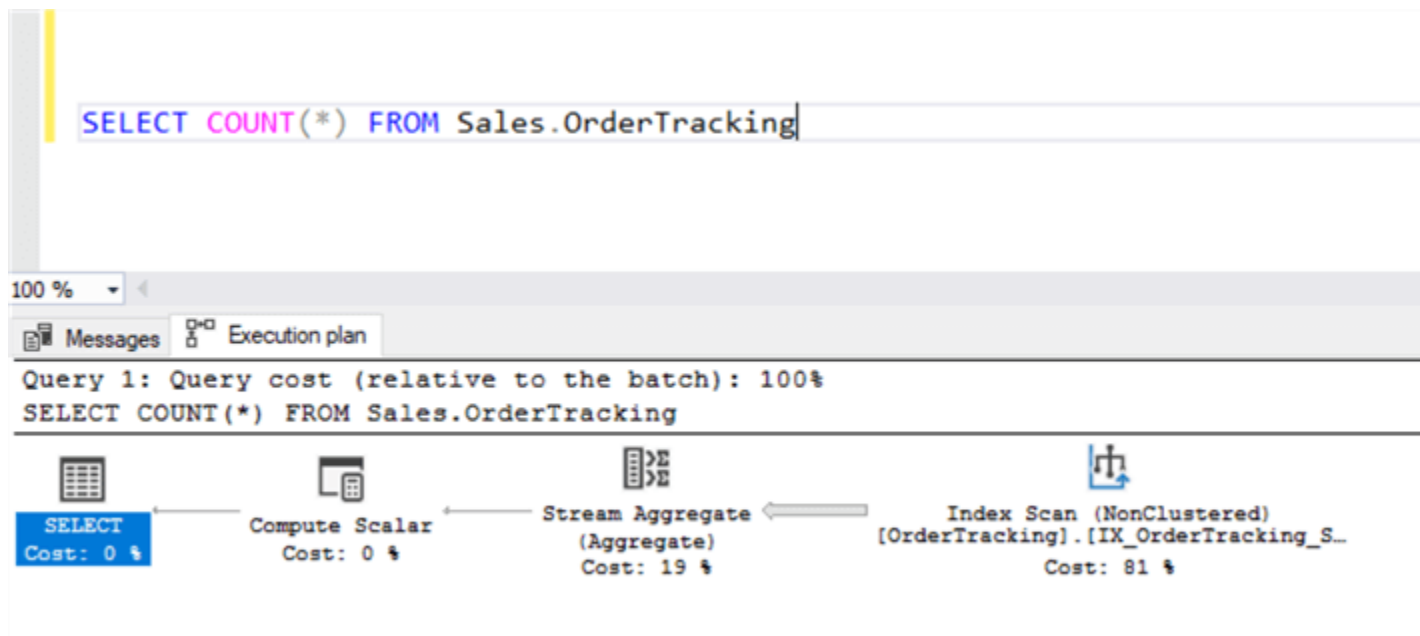


Query 1: Query cost (relative to the batch): 100%

```
SELECT TOP (10) [SalesOrderID] , [SalesOrderDetailID] , [CarrierTrackingNumber] , [OrderQty] , [ProductID] , [SpecialOfferID]
```



Simple example



Cost

Estimated versus actual number of rows

It is very important to remember that all costs you will ever see in a plan are based on cardinality estimations

never on actual row and execution counts.

are only as accurate as the optimizer's cardinality estimations.

One of the first things to check in a plan is to compare estimated and actual row counts

make sure they are within reasonable margins, to confirm the accuracy of the cardinality estimates associated with the estimated costs.



Cost

Having verified that cardinality estimates were accurate, we can look for costliest operators

Operator cost

a means of determining where to focus our initial efforts.

It's often useful to compare the cost of one operator to another within the plan.

we can't compare operator cost within one plan to operator cost within a second plan

Fat pipes

Data flow within an execution plan is defined by the arrows connecting one operator to the next.

frequently referred to as pipes

The thickness of the pipe is based on actual row count when available (actual execution plan), and on estimates otherwise (cached or estimated plan).

A thicker pipe indicates more data being processed;

a thinner pipe indicates less data.

Look out not only for "fat pipes," but also for abrupt transitions in pipe thickness as you read through the execution plan.

For example, a very fat pipe at the beginning of a plan narrowing to a very thin pipe on the left-hand side of the plan suggests that filtering is happening late.

Small pipes that get bigger and bigger suggest that your query is somehow multiplying data.



„Extra” steps

Extra operators

idea of an "extra" operator

Every time you're reading a plan and you see an operator you've never seen, or an operator that you've seen and understand, but can't determine why it's in the spot it's in within the plan, then that is an "extra" operator.

It's an operator that you don't know, or you don't understand why it's affecting the plan.

Scan vs Seek

Read operators

Important ones are the scan and the seek

An indicator of one type of data access

A scan operator (an Index Scan or Table Scan) reads across the pages in an index or a table.

frequently, that a lot of rows are being accessed.

A seek operator uses the structure of an index to identify a starting point, and possibly an ending point, for a targeted scan through the pages of an index.

most of the time, only a small number of rows are being accessed.

Most people have a "**scans bad, seeks good**" mentality.

In fact, neither of these operations is good or bad by definition.

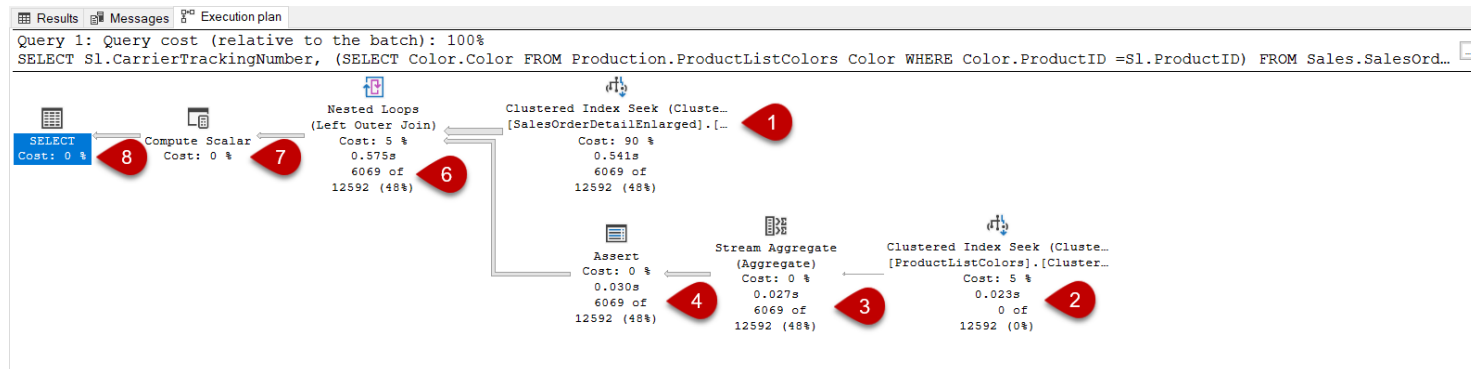
Look out for high-cost scans that retrieve limited data sets

sometimes indicating a missing or poorly structured index

Look out for seeks that retrieve extremely large data sets

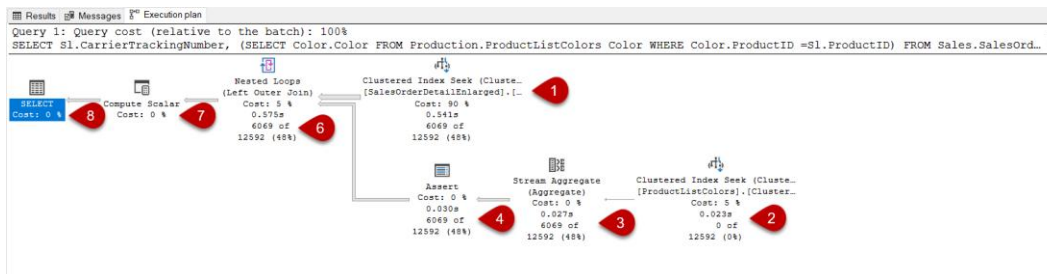
Example

```
SELECT Sl.CarrierTrackingNumber,  
       (SELECT Color. Color  
        FROM Production.ProductListColors Color  
        WHERE Color.ProductID = Sl.ProductID) AS ProductColor  
FROM Sales.SalesOrderDetailEnlarged Sl  
WHERE SalesOrderID > 131336 AND  
       ModifiedDate < '20110601';
```



Example

The clustered index seeks operator shows that the data engine finds the matched rows using the b-tree structure and this is an efficient way to read the data.



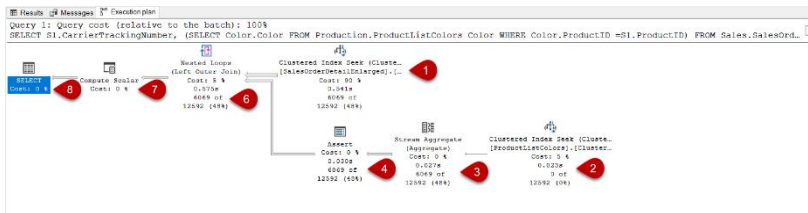
Clustered Index Seek (Clustered)	
Scanning a particular range of rows from a clustered index.	
Physical Operation	Clustered Index Seek
Logical Operation	Clustered Index Seek
Actual Execution Mode	Row
Estimated Execution Mode	Row
Storage	RowStore
Number of Rows Read	4603975
Actual Number of Rows	6069
Actual Number of Batches	0
Estimated Operator Cost	39.9679 (90%)
Estimated I/O Cost	34.8728
Estimated Subtree Cost	39.9679
Estimated CPU Cost	5.09513
Estimated Number of Executions	1
Number of Executions	1
Estimated Number of Rows	12591.6
Estimated Number of Rows to be Read	4631800
Estimated Row Size	48 B
Actual Rebinds	0
Actual Rewinds	0
Ordered	True
Node ID	2
Predicate	
[AdventureWorks2017].[Sales].[SalesOrderDetailEnlarged].	
[ModifiedDate] as [S].[ModifiedDate] < '2011-06-01 00:00:00.000'	
Object	
[AdventureWorks2017].[Sales].[SalesOrderDetailEnlarged].	
[PK_SalesOrderDetailEnlarged_SalesOrderID_SalesOrderDetailID]	
[S]	
Output List	
[AdventureWorks2017].[Sales].	
[SalesOrderDetailEnlarged].CarrierTrackingNumber;	
[AdventureWorks2017].[Sales].	
[SalesOrderDetailEnlarged].ProductID	
Seek Predicates	
Seek Keys[1]: Start: [AdventureWorks2017].[Sales].	
[SalesOrderDetailEnlarged].SalesOrderID > Scalar Operator	
((131336))	

```
SELECT S1.CarrierTrackingNumber,
       (SELECT Color.Color
        FROM Production.ProductListColors Color
        WHERE Color.ProductID = S1.ProductID) AS ProductColor
FROM Sales.SalesOrderDetailEnlarged S1
WHERE SalesOrderID > 131336 AND
       ModifiedDate < '20110601';
```

Example

The second index seek operator which is numbered as 2 tasks is to access ProductListColors and return the Color row according to ProductId.

```
SELECT Sl.CarrierTrackingNumber,  
(SELECT Color.Color  
FROM Production.ProductListColors Color  
WHERE Color.ProductID = Sl.ProductID) AS ProductColor  
FROM Sales.SalesOrderDetailEnlarged Sl  
WHERE SalesOrderID > 131336 AND  
ModifiedDate < '20110601';
```

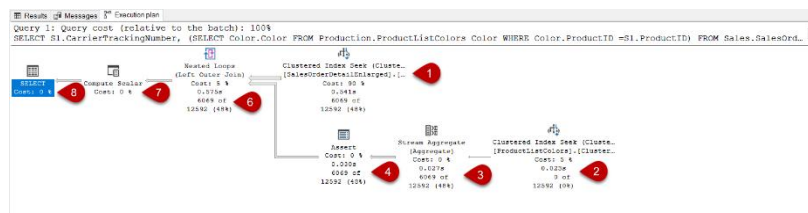


Clustered Index Seek (Clustered)	
Scanning a particular range of rows from a clustered index.	
Physical Operation	Clustered Index Seek
Logical Operation	Clustered Index Seek
Actual Execution Mode	Row
Estimated Execution Mode	Row
Storage	RowStore
Number of Rows Read	6069
Actual Number of Rows	6069
Actual Number of Batches	0
Estimated I/O Cost	0.003125
Estimated Operator Cost	2.00636 (5%)
Estimated CPU Cost	0.0001581
Estimated Subtree Cost	2.00636
Estimated Number of Executions	12591,604
Number of Executions	6069
Estimated Number of Rows	1
Estimated Number of Rows to be Read	1
Estimated Row Size	26 B
Actual Rebinds	0
Actual Rewinds	0
Ordered	True
Node ID	5
Object	
[AdventureWorks2017].[Production].[ProductListColors].	
[IX_Clustered] [Color]	
Output List	
[AdventureWorks2017].[Production].[ProductListColors].Color	
Seek Predicates	
Seek Keys[1]: Prefix: [AdventureWorks2017].[Production].	
[ProductListColors].ProductID = Scalar Operator	
[AdventureWorks2017].[Sales].[SalesOrderDetailEnlarged].	
[ProductID] as [Sl].[ProductID]	

Example

The other operator seems in the execution plan is the stream aggregate operator (3), this operator's task is to perform aggregation operations.

```
SELECT Sl.CarrierTrackingNumber,
       (SELECT Color.Color
        FROM Production.ProductListColors Color
        WHERE Color.ProductID = Sl.ProductID) AS ProductColor
FROM Sales.SalesOrderDetailEnlarged Sl
WHERE SalesOrderID > 131336 AND
       ModifiedDate < '20110601';
```



Stream Aggregate	
Compute summary values for groups of rows in a suitably sorted stream.	
Physical Operation	Stream Aggregate
Logical Operation	Aggregate
Actual Execution Mode	Row
Estimated Execution Mode	Row
Actual Number of Rows	6069
Actual Number of Batches	0
Estimated I/O Cost	0
Estimated Operator Cost	0,01385 (0%)
Estimated Subtree Cost	2,02021
Estimated CPU Cost	0,00000011
Number of Executions	6069
Estimated Number of Executions	12591,604
Estimated Number of Rows	1
Estimated Row Size	34 B
Actual Rebinds	0
Actual Rewinds	0
Node ID	4
Output List Expr1004; Expr1005	

Example – using LIKE

```
Seek Keys[1]:  
  Start: [AdventureWorks2014].[Person].[Person].LastName >= Scalar  
  Operator(N'Jaf'),  
  End: [AdventureWorks2014].[Person].[Person].LastName < Scalar  
  Operator(N'JaG')
```

```
[AdventureWorks2014].[Person].[Person].[LastName] as [p].[LastName] like  
N'Jaf%'
```

Instead of
LIKE 'Jaf%'

in the query,

the optimizer can modify the logic it
uses so that an additional filter has
been added as follows:

Person.LastName >= 'Jaf' AND
Person.LastName < 'JaG'.

Example – insert with constraints

```

create table Test (
  s_id int IDENTITY(1,1) PRIMARY KEY,
  id int,
  name varchar(50),
  moddate date,
  CONSTRAINT l1 check (LEN(name)>2),
  CONSTRAINT l1n CHECK (YEAR(moddate)>2009)
)

create table TestP (
  s_id int IDENTITY(1,1) PRIMARY KEY,
  id int,
  name varchar(50),
  moddate date
)

1 insert into Test(id,name,moddate)
  select SalesOrderID, ProductID, ModifiedDate
  from AdventureWorks2014.sales.SalesOrderDetail

2 insert into TestP(id,name,moddate)
  select SalesOrderID, ProductID, ModifiedDate
  from AdventureWorks2014.sales.SalesOrderDetail
    
```

1

QueryTimeStats	
CpuTime	386
ElapsedTime	469

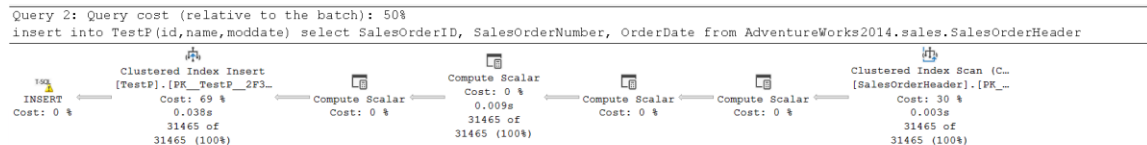
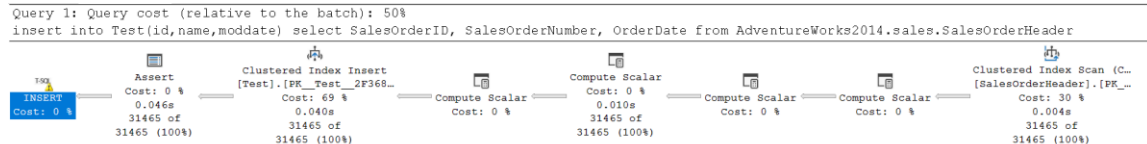
2

QueryTimeStats	
CpuTime	48
ElapsedTime	48

After 6:

QueryTimeStats	
CpuTime	39
ElapsedTime	39

QueryTimeStats	
CpuTime	224
ElapsedTime	224



Example – insert with keys

```
create table TestP (
  id int not null PRIMARY KEY,
  name varchar(50),
  moddate date
)
```

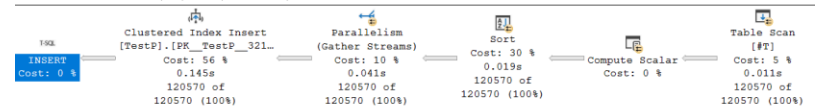
```
create table TestS (
  s_id int IDENTITY(1,1) PRIMARY KEY,
  id int,
  name varchar(50),
  moddate date
)
```

```
create table Test (
  id int,
```

I

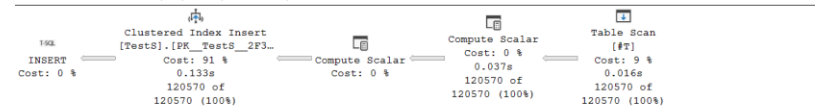
Query 1: Query cost (relative to the batch): 44%

insert into TestP(id,name,moddate) select * from #T



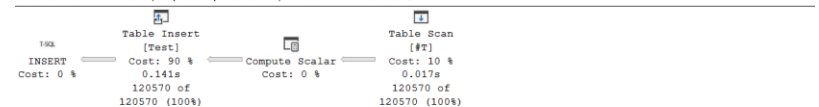
Query 2: Query cost (relative to the batch): 29%

insert into TestS(id,name,moddate) select * from #T



Query 3: Query cost (relative to the batch): 27%

insert into Test(id,name,moddate) select * from #T



Example – insert with keys

```
create table TestP (  
  id int not null PRIMARY KEY,  
  name varchar(50),  
  moddate date  
)  
  
create table TestS (  
  s_id int IDENTITY(1,1) PRIMARY KEY,  
  id int,  
  name varchar(50),  
  moddate date  
)  
  
create table Test (  
  id int,  
  name varchar(50),  
  moddate date  
)
```

I

Degree of Parallelism	8
Estimated Number of Rows for All	0
Estimated Number of Rows Per Ex	120570
Estimated Operator Cost	0 (0%)
Estimated Subtree Cost	7.89292
Memory Grant	18 MB
MemoryGrantInfo	
DesiredMemory	18624
GrantedMemory	18624
GrantWaitTime	0
IsMemoryGrantFeedbackAdjust	NoFirstExecution
LastRequestedMemory	0
MaxQueryMemory	1915192
MaxUsedMemory	7712
RequestedMemory	18624
RequiredMemory	4864
SerialDesiredMemory	14232
SerialRequiredMemory	512
Optimization Level	FULL
OptimizerHardwareDependentPro	
QueryHash	0xE0331627FF8CB7A7
QueryPlanHash	0x52B03023FA06B859
QueryTimeStats	
CpuTime	226
ElapsedTime	152

Example – insert with keys

```
create table TestP (  
  id int not null PRIMARY KEY,  
  name varchar(50),  
  moddate date  
)  
  
create table TestS (  
  s_id int IDENTITY(1,1) PRIMARY KEY,  
  id int,  
  name varchar(50),  
  moddate date  
)  
  
create table Test (  
  id int,  
  name varchar(50),  
  moddate date  
)
```

I

Degree of Parallelism	1
Estimated Number of Rows for All	0
Estimated Number of Rows Per Ex	120570
Estimated Operator Cost	0 (0%)
Estimated Subtree Cost	5.28965
MemoryGrantInfo	
GrantedMemory	0
MaxUsedMemory	0
SerialDesiredMemory	0
SerialRequiredMemory	0
Optimization Level	FULL
OptimizerHardwareDependentPro	
QueryHash	0x0370FC6ECC259434
QueryPlanHash	0xEA73514A32685CA4
QueryTimeStats	
CpuTime	139
ElapsedTime	139

Example – insert with keys

```
create table TestP (  
  id int not null PRIMARY KEY,  
  name varchar(50),  
  moddate date  
)  
  
create table TestS (  
  s_id int IDENTITY(1,1) PRIMARY KEY,  
  id int,  
  name varchar(50),  
  moddate date  
)  
  
create table Test (  
  id int,  
  name varchar(50),  
  moddate date  
)
```

I

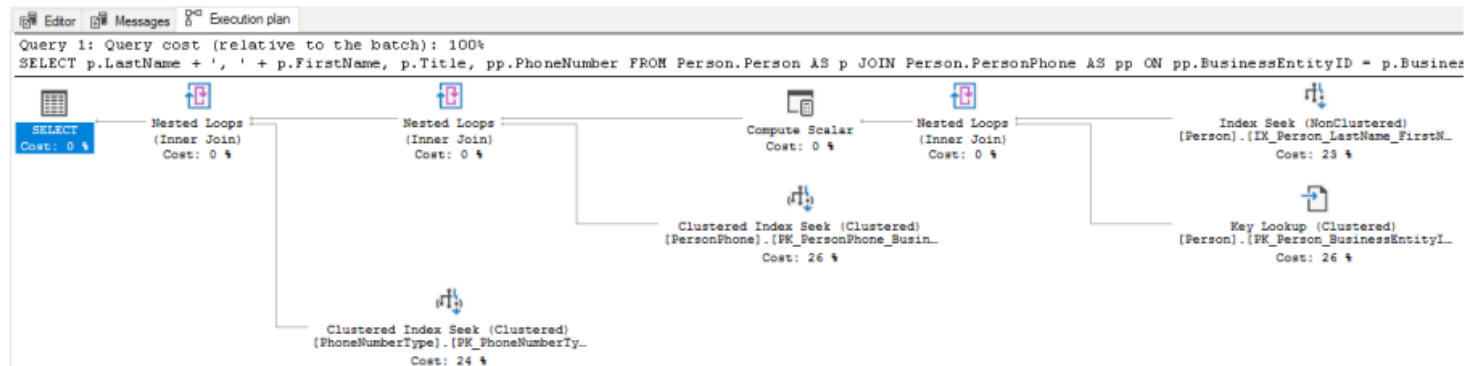
Degree of Parallelism	1
Estimated Number of Rows for All	0
Estimated Number of Rows Per Ex	120570
Estimated Operator Cost	0 (0%)
Estimated Subtree Cost	4.87981
MemoryGrantInfo	
GrantedMemory	0
MaxUsedMemory	0
SerialDesiredMemory	0
SerialRequiredMemory	0
Optimization Level	TRIVIAL
OptimizerHardwareDependentPro	
QueryHash	0x75DFCB2088B5B832
QueryPlanHash	0x25025E35ABCE431A
QueryTimeStats	
CpuTime	147
ElapsedTime	147

Example

```
USE AdventureWorks2014;
GO
SELECT p.LastName + ', ' + p.FirstName,
       p.Title,
       pp.PhoneNumber
FROM Person.Person AS p
     INNER JOIN Person.PersonPhone AS pp
           ON pp.BusinessEntityID = p.BusinessEntityID
     INNER JOIN Person.PhoneNumberType AS pnt
           ON pnt.PhoneNumberTypeID = pp.PhoneNumberTypeID
WHERE pnt.Name = 'Cell'
     AND p.LastName = 'Dempsey';
GO
```

Example

```
USE AdventureWorks2014;
GO
SELECT p.LastName + ', ' + p.FirstName,
       p.Title,
       pp.PhoneNumber
FROM Person.Person AS p
INNER JOIN Person.PersonPhone AS pp
ON pp.BusinessEntityID = p.BusinessEntityID
INNER JOIN Person.PhoneNumberType AS pnt
ON pnt.PhoneNumberTypeID = pp.PhoneNumberTypeID
WHERE pnt.Name = 'Cell'
AND p.LastName = 'Dempsey';
GO
```

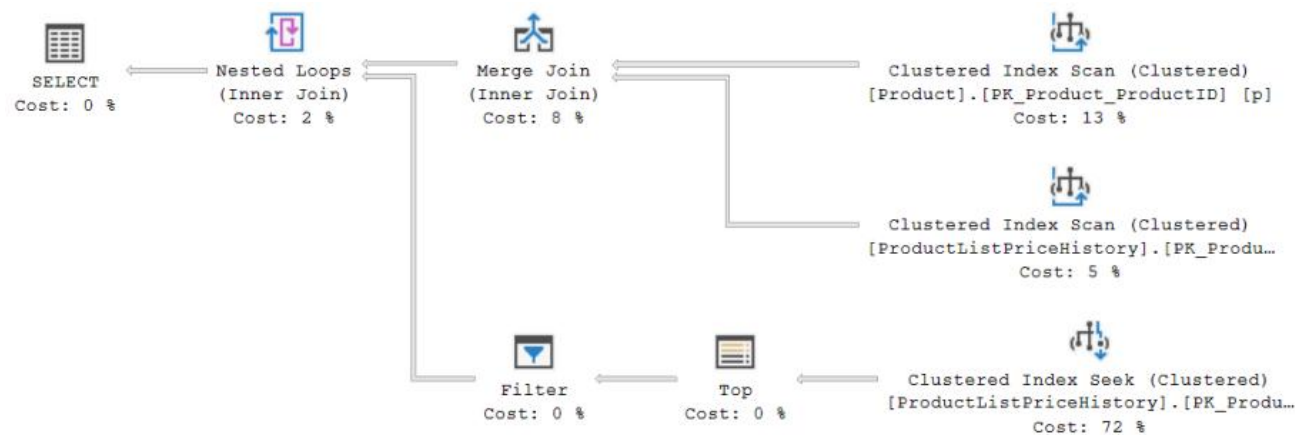


Example

```
SELECT p.Name,
       p.ProductNumber,
       ph.ListPrice
FROM Production.Product AS p
     INNER JOIN Production.ProductListPriceHistory AS ph
     ON p.ProductID = ph.ProductID
        AND ph.StartDate = (
            SELECT TOP (1)
                ph2.StartDate
            FROM Production.
ProductListPriceHistory AS ph2
            WHERE ph2.ProductID = p.ProductID
            ORDER BY ph2.StartDate DESC);
```

Example

```
SELECT p.Name,  
       p.ProductNumber,  
       ph.ListPrice  
FROM Production.Product AS p  
     INNER JOIN Production.ProductListPriceHistory AS ph  
       ON p.ProductID = ph.ProductID  
        AND ph.StartDate = ( SELECT TOP (1)  
                             ph2.StartDate  
                             FROM Production.  
ProductListPriceHistory AS ph2  
                            WHERE ph2.ProductID = p.ProductID  
                             ORDER BY ph2.StartDate DESC);
```



Common pitfalls

Ignoring Indexes

Misusing Wildcard Characters in LIKE Clauses

```
product_name LIKE '%shirt';  
product_name LIKE 'shirt%';
```

Not Using Parameterized Queries

Overusing Subqueries

Not Using LIMIT for Large Result Sets

Using SELECT * as standard

sending data to the application tier

Functions on indexed columns in predicates

```
WHERE LEFT(customer_name, 1) = 'L';  
WHERE customer_name LIKE 'L%';
```

ON before WHERE

Unnecessarily ordering data

DISTINCT

Not very good in optimizing the OR predicates.

SQL Server, Oracle

Paul Weinberg and James Groff and
Andrew Oppel

SQL The Complete Reference, Third
Edition

McGraw-Hill, 2010

Beginning T-SQL 2012

Scott Shaw and Kathi Kellenberger

Apress, 2012

Grant Fritchey

SQL Server Execution Plans

Redgate Publishing 2018

<https://sqlserverfast.com/epr/>

<https://docs.microsoft.com/en-us/sql/relational-databases/showplan-logical-and-physical-operators-reference>

Bibliography

SOURCES